AWS CLOUD | HANDS-ON IMPLEMENTATION GUIDE

Deploy a Three-Tier Web Application on AWS

A secure, highly available and scalable three-tier architecture, built step by step with a custom VPC, EC2, RDS for MySQL, Application Load Balancers and Auto Scaling in the Canada (Central) Region.

VPC | EC2 | RDS MySQL | ALB | Auto Scaling | S3 | IAM | NAT | SSM

PREPARED BY
Darshan Patil

REGION
Canada (Central)
ca-central-1

VERSION
1.0

DATE
October 2026

Project documentation | 15 guided steps | 3 tiers | 2 Availability Zones | 15 AWS services

AWS | CLOUD ENGINEERING PROJECT

THREE-TIER WEB APPLICATION ON AWS

SECURITY | HIGH AVAILABILITY | SCALABILITY

VPC | EC2 | RDS MySQL | ALB | AUTO SCALING | S3 | IAM | SSM

CREATED BY

Darshan Patil

Cloud and DevOps project | October 2026

LinkedIn: https://www.linkedin.com/in/darshan-devops/

GitHub: https://github.com/darshanpatil8799/darshanpatil8799

Document at a Glance

This document records, step by step, how I built and deployed a three-tier web application on AWS in the **Canada (Central)** Region: a public web tier, a private application tier and a private MySQL database tier inside one custom VPC that spans two Availability Zones. Every step lists the exact settings I used, the commands I ran, and screenshots taken from my own AWS console while doing the work, followed by a checkpoint so you always know you are on track.

15	3	2	15
Guided steps	Application tiers	Availability Zones	AWS services

Document Control

Item	Details
Title	Deploy a Three-Tier Web Application on AWS
Prepared by	Darshan Patil
Version / Date	1.0 | October 2026
AWS Region	Canada (Central) ca-central-1, Availability Zones ca-central-1a (cac1-az1) and ca-central-1b (cac1-az2)
Technology stack	Nginx + React (web tier), Node.js + PM2 (app tier), Amazon RDS for MySQL 8.4 (database tier)
Access method	AWS Systems Manager Session Manager: no SSH keys, no bastion host, no public IPs on private servers
Evidence	All screenshots are captured from my own AWS account during the deployment (account identifiers are blurred)

How to Read This Document

Each step opens with a short explanation, followed by numbered actions, settings tables, commands and screenshots. Coloured boxes call out the information that matters most:

WHY THIS STEP?
Explains the purpose of the step before you start.

IMPORTANT
A setting that must be exact, or something that affects your bill.

NOTE
Extra context or an alternative way to do the same thing.

PRO TIP
A shortcut or good habit that saves time.

WARNING
A mistake that can break the deployment or expose data.

CHECKPOINT
A quick test to confirm the step worked before moving on.

Text with a gold highlight, like Three-Tier-Project-VPC, marks a value you will reuse in later steps: names, CIDR blocks and ports.

Contents

Document at a Glance . . . 3
Document Control . . . 3
How to Read This Document . . . 3
Contents . . . 5
Your Journey at a Glance . . . 7
1. Introduction . . . 8
2. Project Overview . . . 9
AWS Components Used . . . 9
Key Terms in Plain English . . . 10
3. Architecture . . . 11
Network Plan . . . 12
Security Group Rules (Traffic Flow) . . . 12
Prerequisites . . . 12
Step 1: Create the VPC and Subnets . . . 13
Create the VPC . . . 13
Create the subnets . . . 14
Enable auto-assign public IP on the web subnets . . . 18
Step 2: Create the Internet Gateway, NAT Gateways and Route Tables . . . 21
Create and attach the internet gateway . . . 21
Create the NAT gateways . . . 22
Create the route tables . . . 23
Edit routes and subnet associations . . . 25
Verify with the resource map . . . 30
Step 3: Create the Security Groups . . . 33
Step 4: Create the DB Subnet Group and the RDS Database . . . 38
Create the DB subnet group . . . 38
Create the database . . . 40
Step 5: Create the IAM Role . . . 44
Step 6: Create an S3 Bucket and Upload the Source Code . . . 47
Create the bucket . . . 47

Upload the application code . . . 49

Step 7: Create the Test App Server and Set It Up . . . 52

Launch the test app server . . . 52

Put your database details in the code . . . 54

Open a terminal with Session Manager . . . 54

Configure the app tier . . . 56

Step 8: Create an AMI of the App Server . . . 61

Step 9: Target Group, Launch Template and Internal Load Balancer (App Tier) . . . 63

A. Create the target group . . . 63

B. Create the launch template . . . 67

C. Create the internal Application Load Balancer . . . 69

Step 10: Create the Test Web Server and Set It Up . . . 73

Point nginx.conf at the internal load balancer . . . 73

Launch the test web server . . . 73

Configure the web server . . . 75

Step 11: Create an AMI of the Web Server . . . 77

Step 12: Target Group, Launch Template and External Load Balancer (Web Tier) . . . 78

A. Create the target group (web tier) . . . 78

B. Create the launch template (web tier) . . . 80

C. Create the external (internet-facing) load balancer . . . 82

Step 13: Create the Auto Scaling Groups . . . 85

A. Auto Scaling group for the app tier . . . 85

B. Auto Scaling group for the web tier . . . 85

Step 14: Access the Application . . . 87

Prove the fault tolerance . . . 87

Step 15: Clean Up the Infrastructure . . . 88

Troubleshooting Quick Guide . . . 89

Appendix A: Resource Inventory . . . 90

Appendix B: Ideas to Harden for Production . . . 91

References and Credits . . . 91

Appendix C: Cost Management . . . 92

Your Journey at a Glance

Four phases, fifteen steps. Every step ends with a quick check, so a mistake is caught early instead of three steps later.

PHASE 1
Network foundation
Steps 1 to 3

VPC, subnets, internet and NAT gateways, route tables, security groups

PHASE 2
Data and access
Steps 4 to 6

RDS MySQL database, IAM role, S3 bucket with the source code

PHASE 3
Servers and traffic
Steps 7 to 12

App and web servers, AMIs, launch templates, target groups, load balancers

PHASE 4
Scale and launch
Steps 13 to 15

Auto Scaling groups, open the application, clean up

1. Introduction

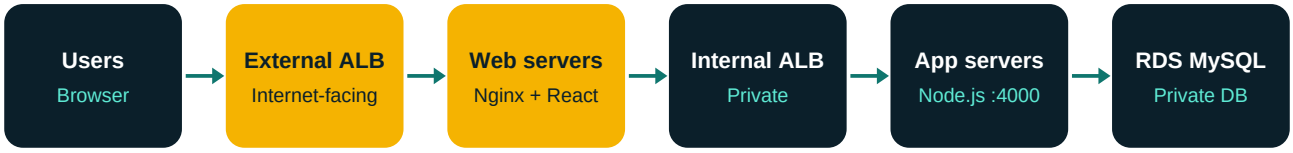
Most real applications must keep working when a single server fails. This project shows how to design for that from the first click. It builds a small, production-style application in which the website, the business logic and the data each live in their own protected layer, while AWS keeps every layer healthy with load balancing and Auto Scaling.

A three-tier architecture splits an application into three layers, each with one job. That separation improves security (a layer can only be reached through the one in front of it), manageability (you can change one layer without touching the others) and flexibility (each layer scales on its own).

Tier	Responsibility	Technology in this project
1. Web tier (presentation)	What the user sees and clicks. Serves pages and forwards API calls.	Nginx serving a React application, on EC2 in public subnets behind an internet-facing load balancer.
2. App tier (logic)	Processes requests, applies business rules, talks to the database.	Node.js API kept alive by PM2 on port 4000, on EC2 in private subnets behind an internal load balancer.
3. Database tier (data)	Stores and returns the application data.	Amazon RDS for MySQL 8.4 in private DB subnets, not publicly accessible.

A user talks only to the web tier. The web tier forwards API calls to the app tier, the app tier reads and writes the database, and the answer travels back along the same path.

How a request travels



Gold = reachable from the internet (public subnets). Dark = private, never directly reachable.

2. Project Overview

The goal was a highly available, secure, scalable and fault-tolerant application on AWS. In practice that translated into four design rules:

- **High availability.** Subnets, load balancers and Auto Scaling groups span two Availability Zones, so losing one zone does not stop the application.
- **Security.** Only the external load balancer faces the internet. App servers and the database sit in private subnets and accept traffic only from the tier in front of them.
- **Scalability.** Auto Scaling groups build servers from launch templates, and load balancers spread requests across whatever is running.
- **Fault tolerance.** Failed health checks make Auto Scaling replace a server automatically, and each tier has at least two servers in different zones.

AWS Components Used

#	AWS service	Role in this project
1	Amazon VPC	The isolated network (10.0.0.0/16) that holds every resource.
2	Amazon EC2	Runs the web and app servers (t3.micro, Amazon Linux 2023).
3	Amazon RDS for MySQL	Managed MySQL 8.4 database for the application data.
4	Amazon S3	Holds the application source code so any server can fetch it.
5	AWS IAM	A role that lets the servers read from S3 without stored credentials.
6	AWS Systems Manager	Session Manager gives a terminal on private servers without SSH keys.
7	Application Load Balancer	One internet-facing ALB for the web tier and one internal ALB for the app tier.
8	Target groups	The list of servers (and their health checks) behind each load balancer.
9	Auto Scaling groups	Keep the wanted number of healthy web and app servers running.
10	Launch templates	The recipe Auto Scaling uses to create each new server.
11	Amazon Machine Images	Saved copies of the configured app server and web server.
12	Internet gateway	The door between the VPC and the internet.
13	NAT gateways	Let private servers reach the internet for downloads while staying unreachable.
14	Route tables	Rules that decide where each subnet sends its traffic.
15	Security groups	Stateful firewalls around every resource.



Key Terms in Plain English

Term	What it means
VPC	Your own private network inside AWS, with an IP range you choose.
Subnet	A slice of the VPC tied to one Availability Zone. Public subnets have a route to the internet gateway; private subnets do not.
Internet gateway / NAT gateway	The internet gateway lets public subnets talk to the internet in both directions. A NAT gateway lets private servers start outbound connections only.
Route table	A list of rules saying where traffic for each destination should go.
Security group	A virtual firewall around a resource. Rules can name another security group as the source.
IAM role	Permissions a server can borrow, so no passwords or keys live on the machine.
AMI and launch template	An AMI is a saved copy of a configured server. A launch template is the recipe (AMI, size, network, role) used to build new ones.
Target group, load balancer, Auto Scaling group	A target group lists the servers a load balancer sends traffic to. The Auto Scaling group keeps the right number of healthy servers registered there.

3. Architecture

The diagram shows how the components connect and how each tier is spread across the two Availability Zones of the Canada (Central) Region.

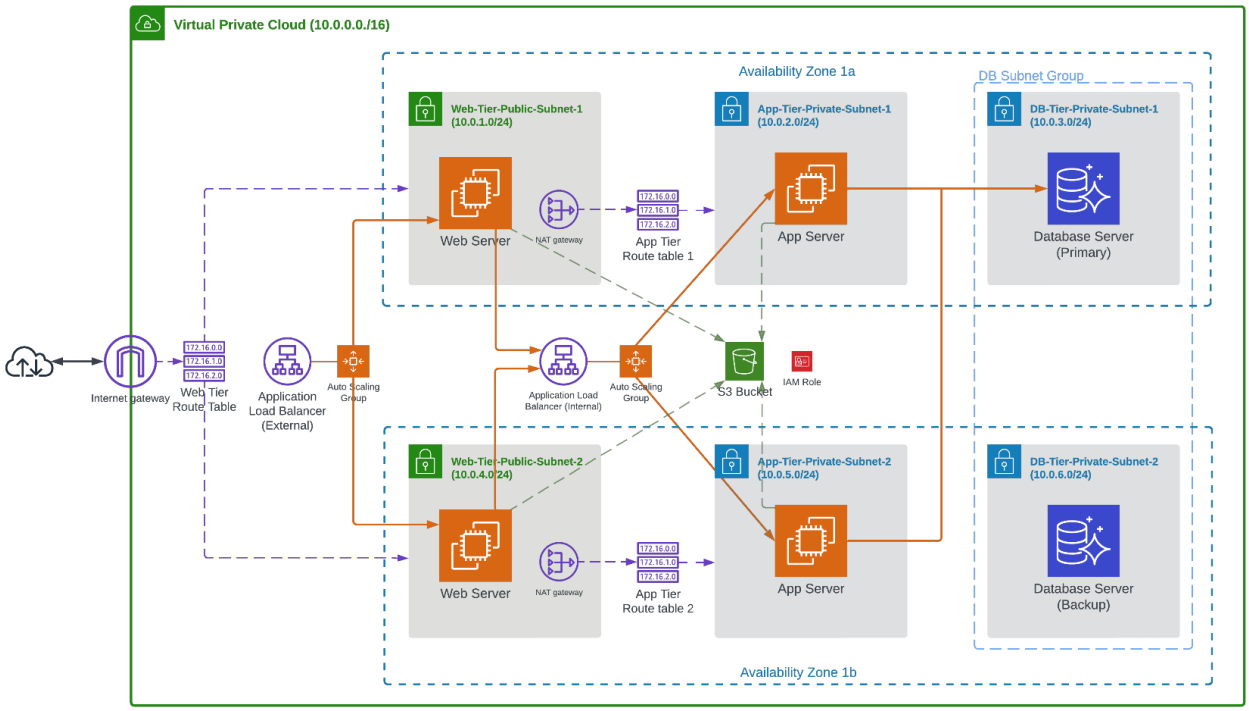


Figure A. VPC 10.0.0.0/16 with public web subnets, private app subnets and private database subnets across ca-central-1a and ca-central-1b.

The web tier lives in public subnets because users must reach it. The app and database tiers are private. All of it sits in a custom VPC, and an S3 bucket carries the source code so that configuring a server is a single copy command.

Network Plan

Subnet	AZ	Type	CIDR	Hosts
Web-Tier-Subnet-1-Public	1a	Public	10.0.1.0/24	Web servers, NAT-GT-1
App-Tier-Subnet-1-Private	1a	Private	10.0.0.0/24	App servers
DB-Tier-Subnet-1-Private	1a	Private	10.0.3.0/24	MySQL database
Web-Tier-Subnet-2-Public	1b	Public	10.0.4.0/24	Web servers, NAT-GT-2
App-Tier-Subnet-2-Private	1b	Private	10.0.5.0/24	App servers
DB-Tier-Subnet-2-Private	1b	Private	10.0.6.0/24	Second DB subnet (DB subnet group)

AZ 1a is **ca-central-1a** and AZ 1b is **ca-central-1b**. The block 10.0.2.0/24 is intentionally unused and kept free for future growth.

Security Group Rules (Traffic Flow)

Security group	Inbound port	Allowed source
External-Load-Balancer-SG	HTTP (80)	0.0.0.0/0 (anyone on the internet)
Web-Tier-SG	HTTP (80)	External-Load-Balancer-SG
Internal-Load-Balancer-SG	HTTP (80)	Web-Tier-SG
App-Tier-SG	Custom TCP (4000)	Internal-Load-Balancer-SG
DB-Tier-SG	MYSQL/Aurora (3306)	App-Tier-SG

WHY THIS STEP?
Each tier accepts traffic only from the tier in front of it. That chain of security-group references is why the database can never be reached straight from the internet.

Prerequisites

- An AWS account that can create VPC, EC2, RDS, IAM, S3 and load balancer resources.
- A web browser, plus a code editor on your PC (I used VS Code) to edit two configuration files.
- One Region for everything. This project uses **Canada (Central) ca-central-1**; resources in different Regions cannot see each other.
- The application source code (a React web tier, a Node.js app tier and an nginx.conf) from the public sample three-tier project listed in References.

COST WARNING
Several services bill by the hour: two NAT gateways, two load balancers, the RDS database and the EC2 servers. Finish the project, capture your screenshots, then follow Step 15 to delete everything.

NOTE
You can name resources however you like as long as you stay consistent. This document uses the names visible in the screenshots.

STEP 01 OF 15 NETWORK FOUNDATION

Create the VPC and Subnets

Draw the map before you build the city

WHY THIS STEP?

Everything else lives inside this private network, so it is built first. The VPC and its six subnets decide which resources can ever talk to each other.

Sign in to the AWS Console, choose **Canada (Central)** in the Region selector and keep it there for the whole project. Use the console search bar to open each service.

Create the VPC

1. Search for **VPC** and open **Your VPCs > Create VPC**.
2. Choose **VPC only** and set the name tag to `Three-Tier-Project-VPC`.
3. Select **IPv4 CIDR manual input** and enter `10.0.0.0/16`. Keep **No IPv6 CIDR block** and tenancy **Default**.
4. Click **Create VPC**.

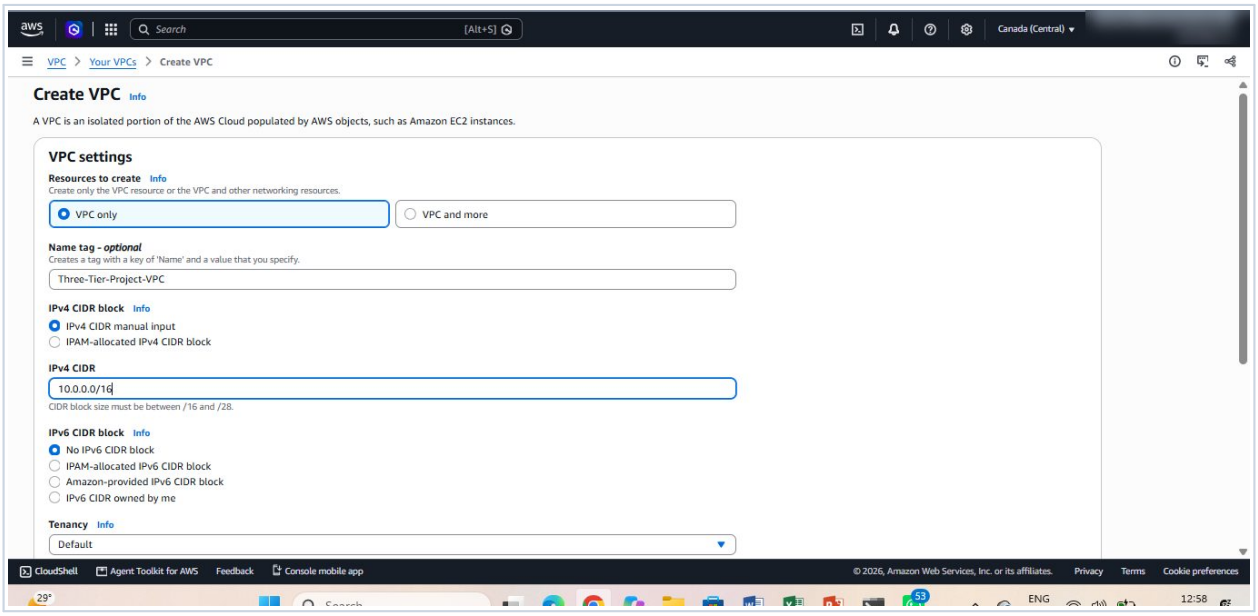


Figure 1. Create VPC: VPC only, name tag Three-Tier-Project-VPC, IPv4 CIDR 10.0.0.0/16.

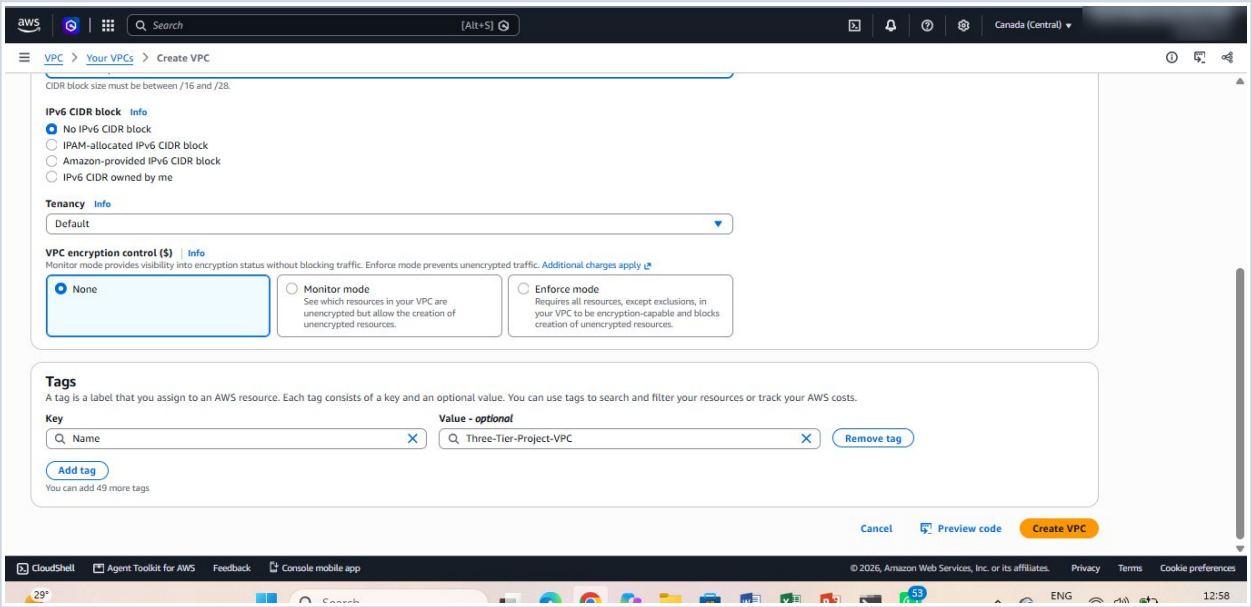


Figure 2. Scroll down, confirm the Name tag and click Create VPC.

Create the subnets

Create six subnets in the new VPC, three per Availability Zone (ca-central-1a and ca-central-1b). Go to **VPC > Subnets > Create subnet**, choose **Three-Tier-Project-VPC** and enter each subnet as below.

Subnet name	Availability Zone	Type	IPv4 CIDR block
Web-Tier-Subnet-1-Public	ca-central-1a	Public	10.0.1.0/24
App-Tier-Subnet-1-Private	ca-central-1a	Private	10.0.0.0/24
DB-Tier-Subnet-1-Private	ca-central-1a	Private	10.0.3.0/24
Web-Tier-Subnet-2-Public	ca-central-1b	Public	10.0.4.0/24
App-Tier-Subnet-2-Private	ca-central-1b	Private	10.0.5.0/24
DB-Tier-Subnet-2-Private	ca-central-1b	Private	10.0.6.0/24

PRO TIP

I created the first four subnets one at a time, then used **Add new subnet** to enter the last two on a single page. Either way works.

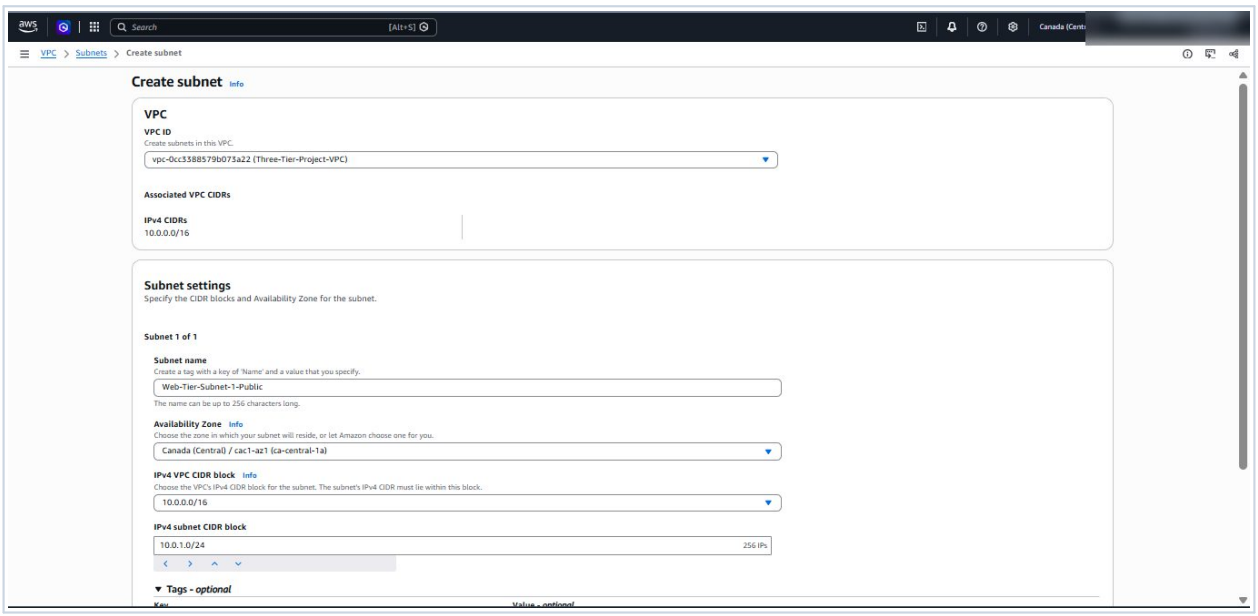


Figure 3. Subnet 1: Web-Tier-Subnet-1-Public in ca-central-1a (10.0.1.0/24).

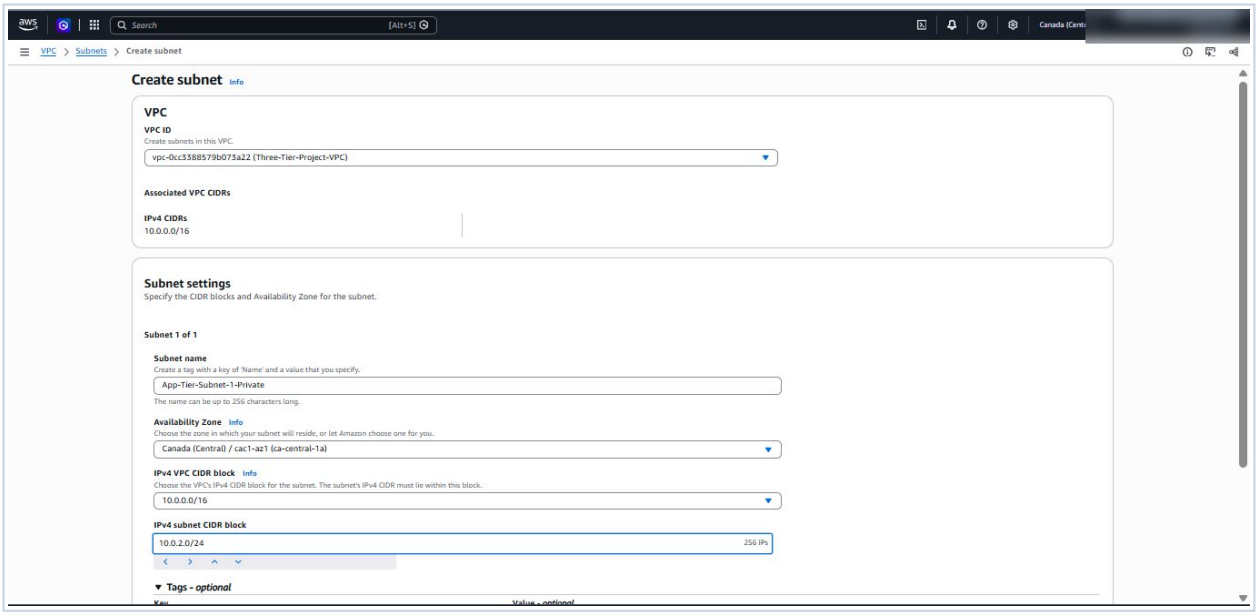


Figure 4. Subnet 2: App-Tier-Subnet-1-Private in ca-central-1a (10.0.0.0/24).

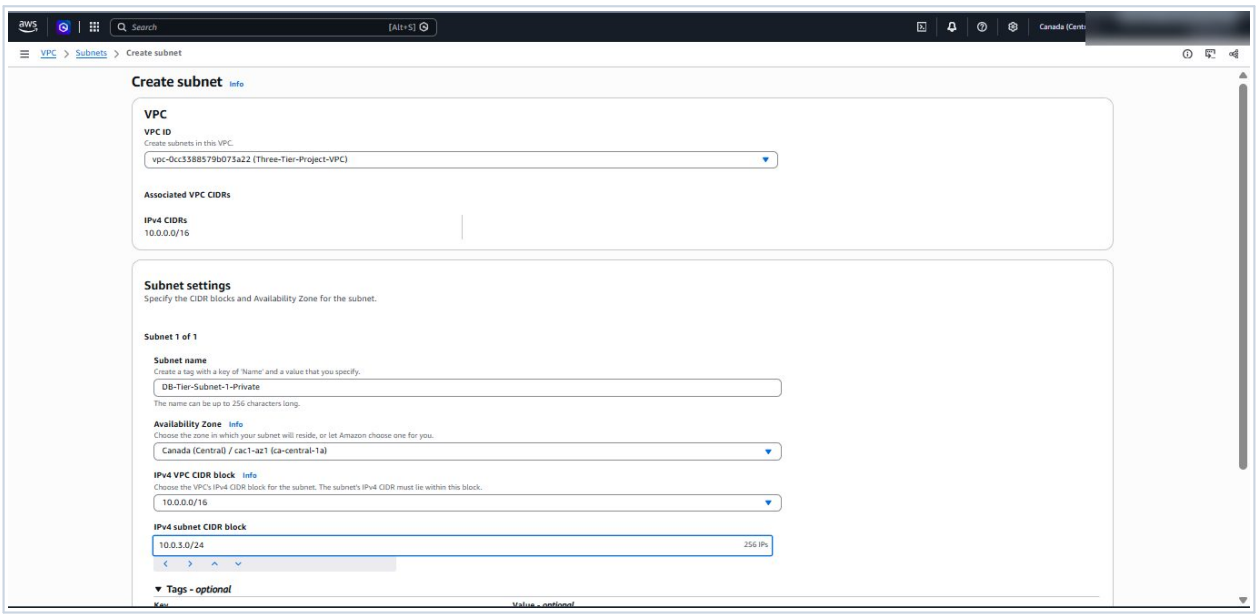


Figure 5. Subnet 3: DB-Tier-Subnet-1-Private in ca-central-1a (10.0.3.0/24).

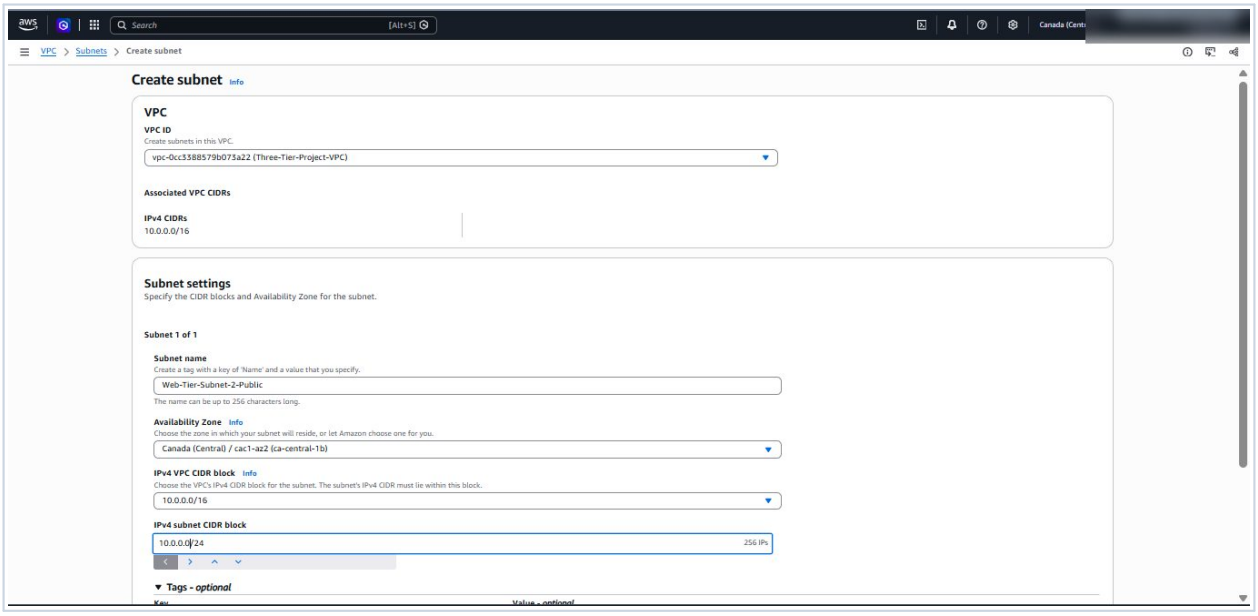


Figure 6. Subnet 4: Web-Tier-Subnet-2-Public in ca-central-1b (10.0.4.0/24).

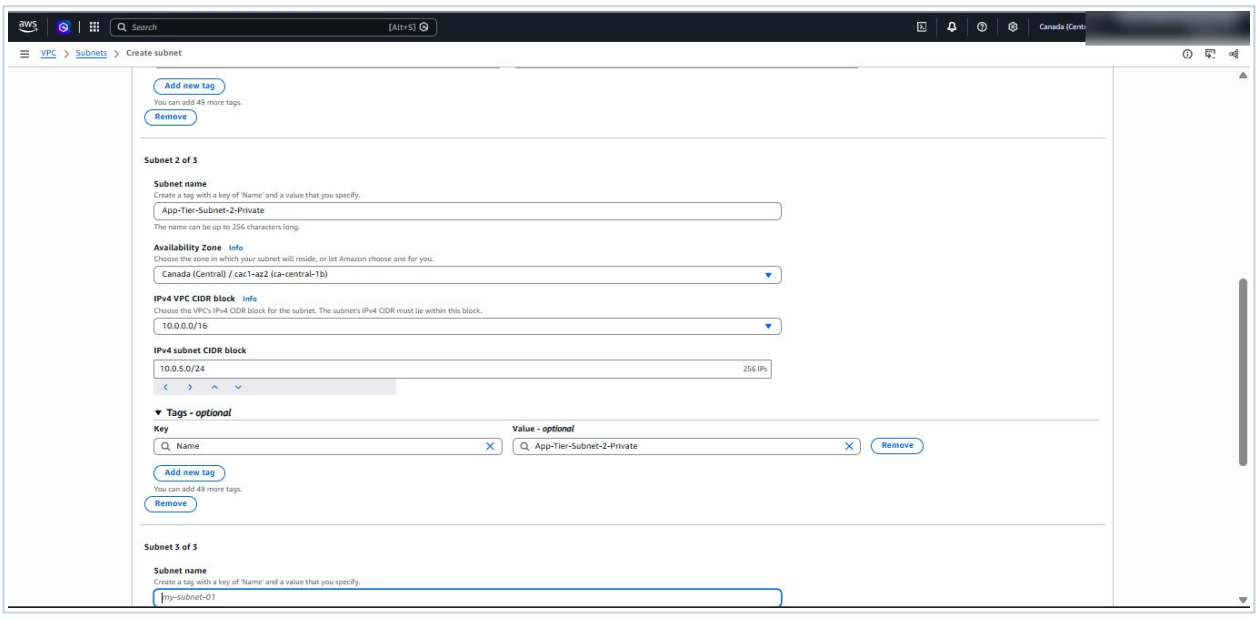


Figure 7. Subnet 5: App-Tier-Subnet-2-Private in ca-central-1b (10.0.5.0/24), added with Add new subnet.

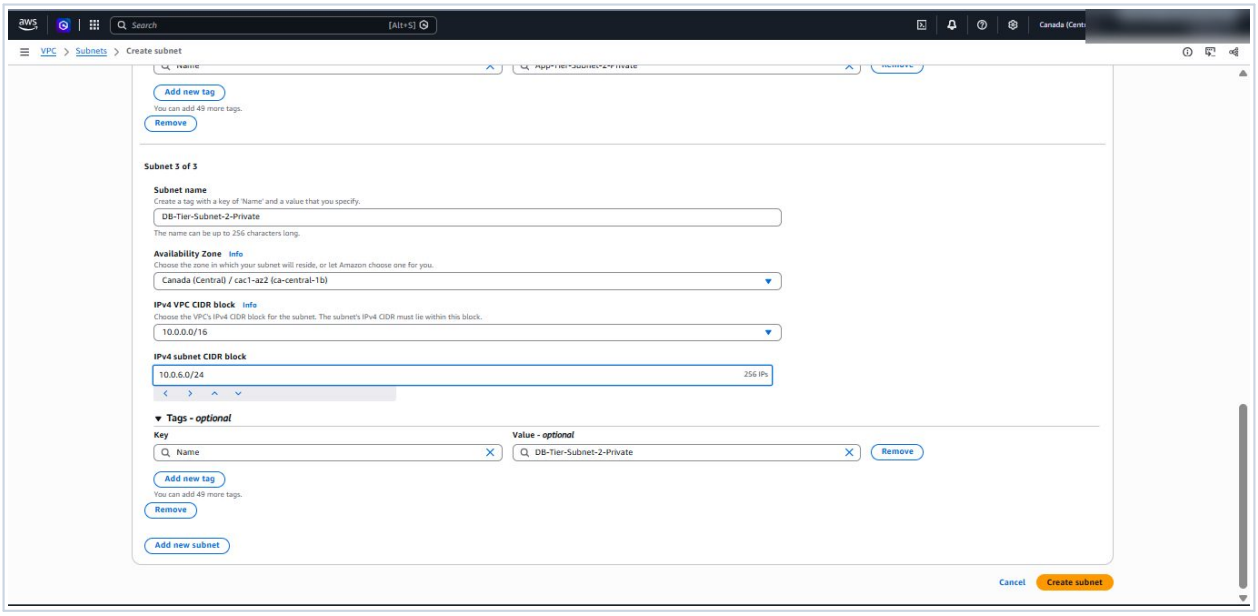


Figure 8. Subnet 6: DB-Tier-Subnet-2-Private in ca-central-1b (10.0.6.0/24). Click Create subnet.

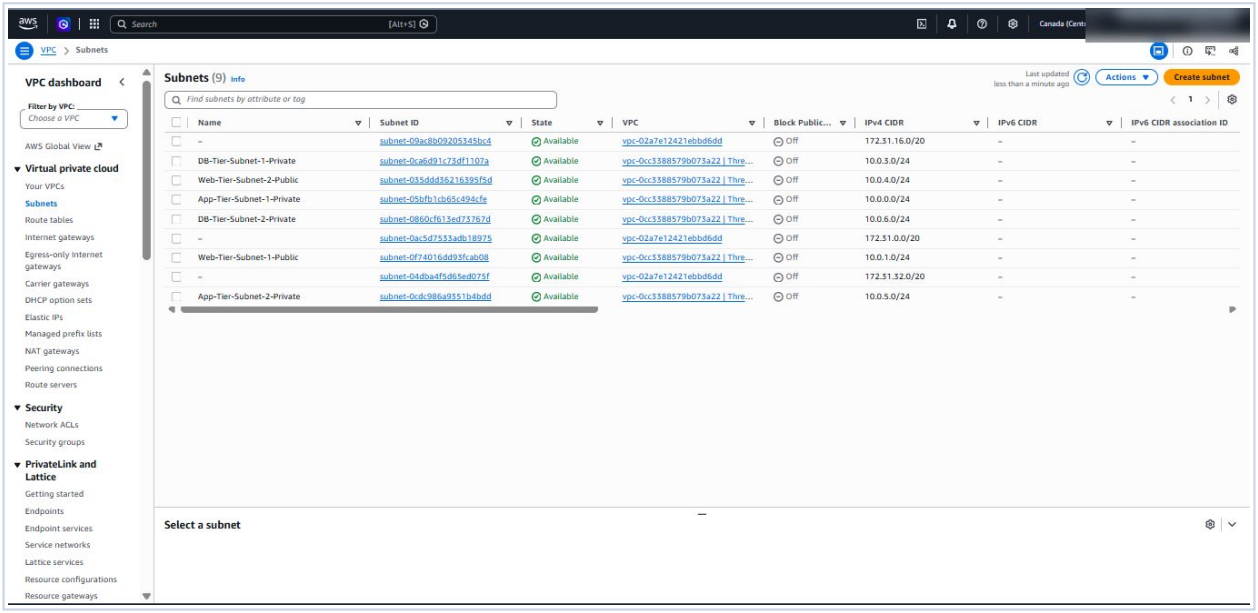


Figure 9. The Subnets list after creation: the six project subnets sit beside the three default-VPC subnets.

Enable auto-assign public IP on the web subnets

1. Select **Web-Tier-Subnet-1-Public** and choose **Actions > Edit subnet settings**.
2. Tick **Enable auto-assign public IPv4 address** and click **Save**.
3. Repeat for **Web-Tier-Subnet-2-Public**.

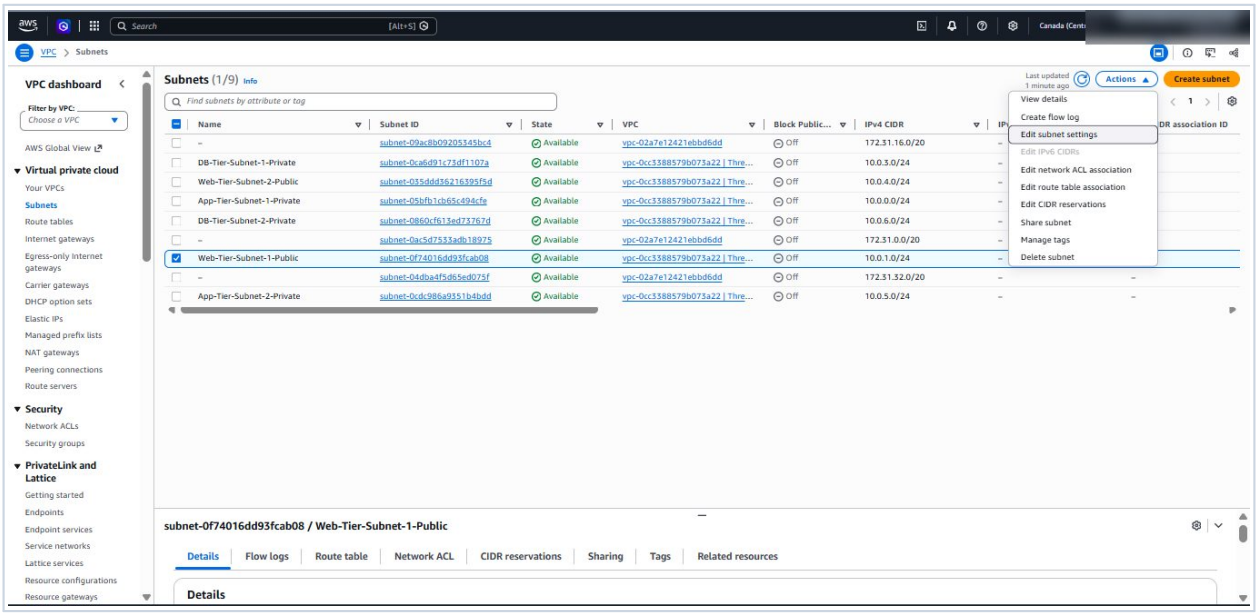


Figure 10. Select Web-Tier-Subnet-1-Public, then Actions > Edit subnet settings.

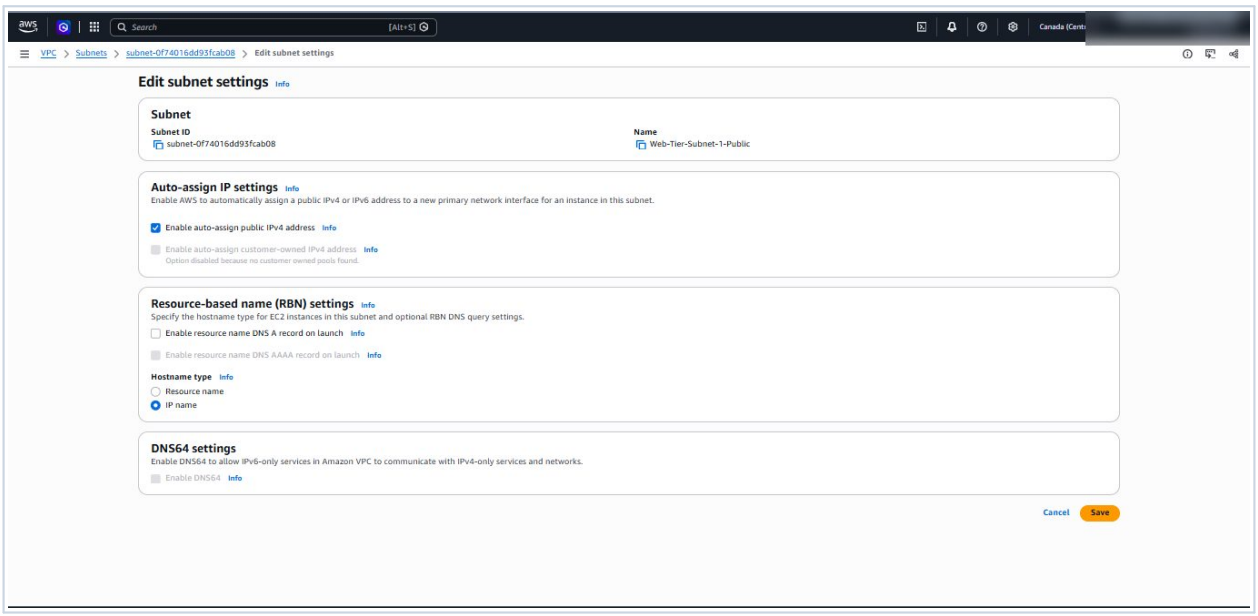


Figure 11. Enable auto-assign public IPv4 address for Web-Tier-Subnet-1-Public and save.

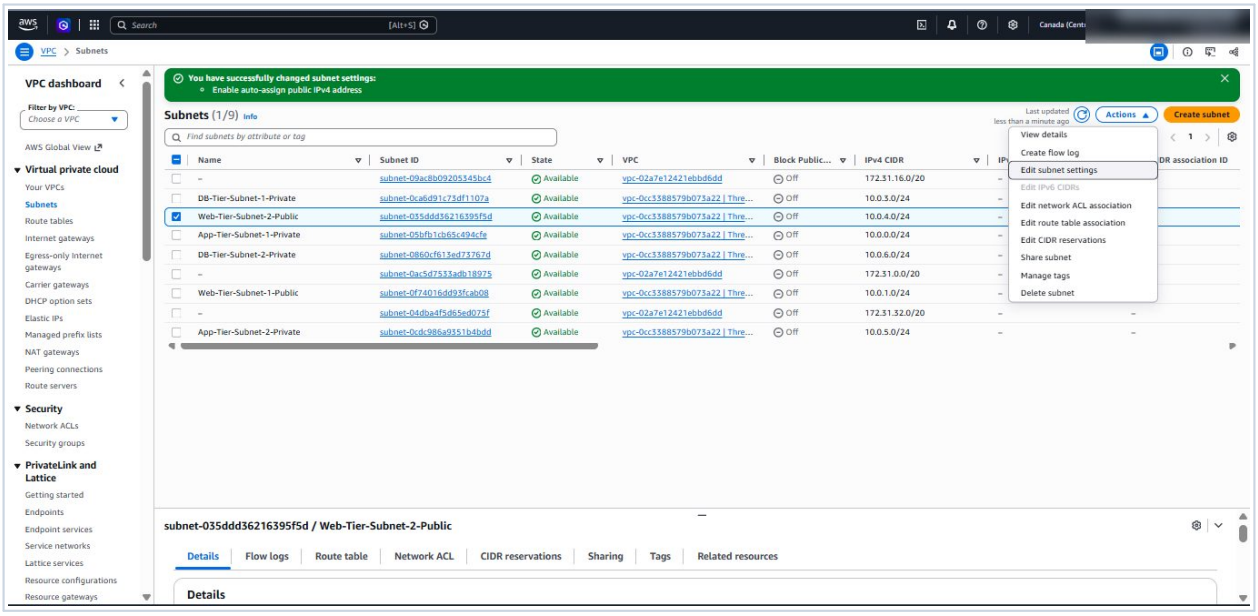


Figure 12. The success banner confirms subnet 1. Now select Web-Tier-Subnet-2-Public and open Edit subnet settings.

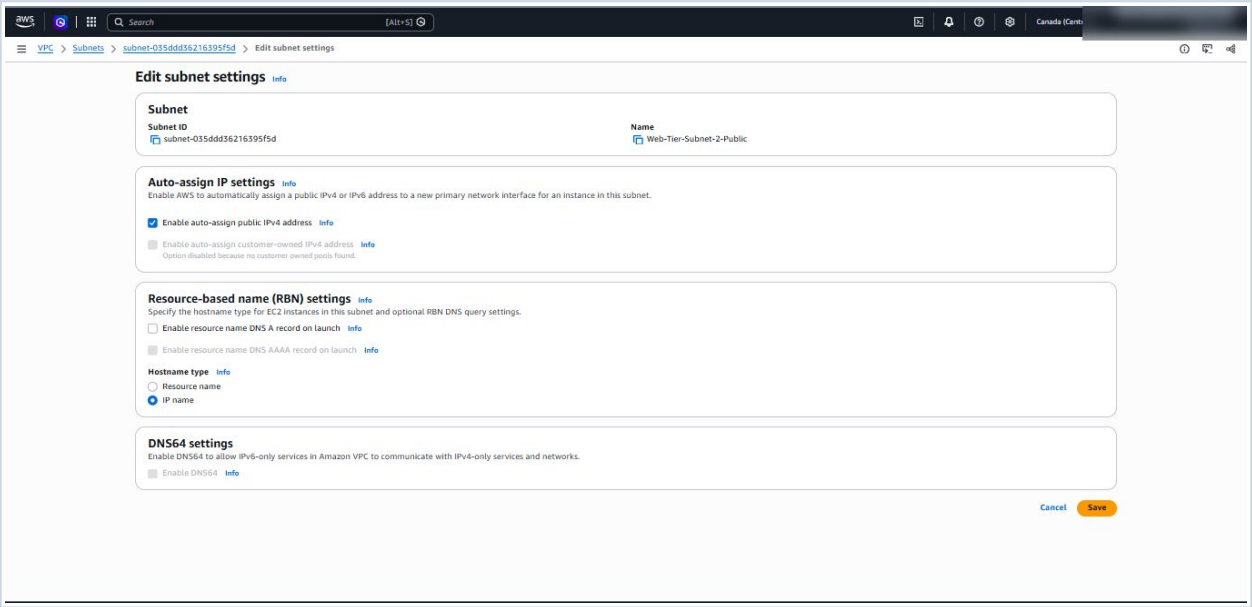


Figure 13. Enable auto-assign public IPv4 address for Web-Tier-Subnet-2-Public and save.

CHECKPOINT

Six subnets exist in Three-Tier-Project-VPC, and both web subnets hand out public IPv4 addresses automatically.

STEP 02 OF 15 NETWORK FOUNDATION

Create the Internet Gateway, NAT Gateways and Route Tables

Open the doors, then lay the roads

WHY THIS STEP?

The web tier needs a path to the internet. The private servers need a way out for software downloads, without letting anyone in. Gateways provide the doors and route tables provide the directions.

Create and attach the internet gateway

1. Go to **VPC > Internet gateways > Create internet gateway**.
2. Set the name tag to Three-Tier-Project-IGW and create it.
3. Choose **Actions > Attach to VPC**, select **Three-Tier-Project-VPC** and click **Attach internet gateway**.

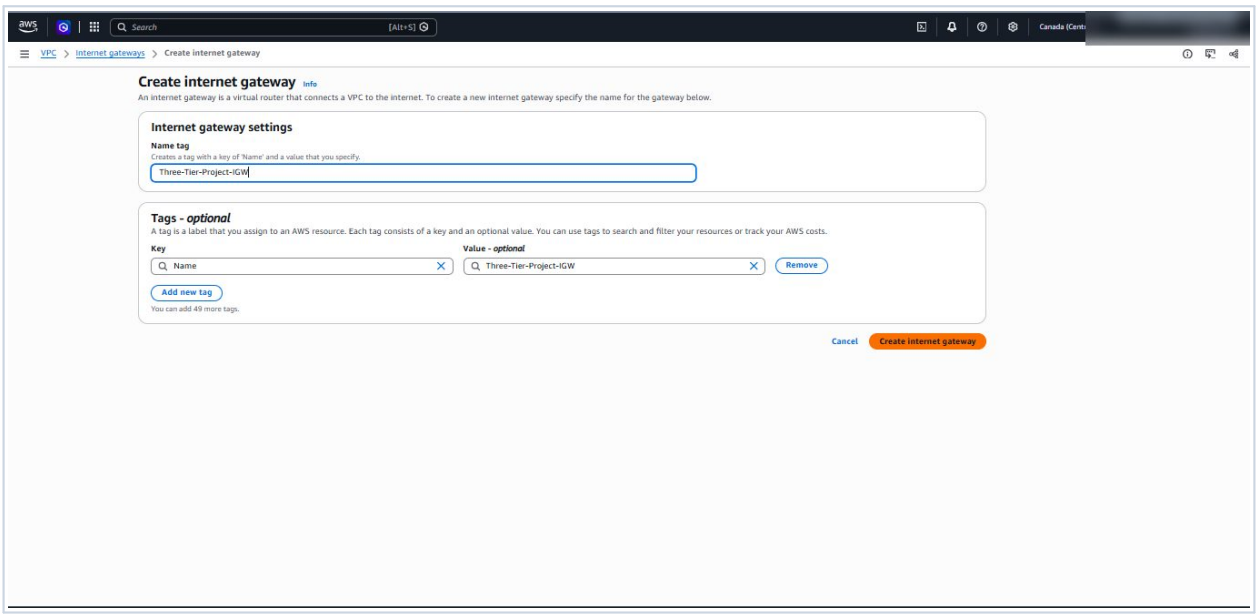


Figure 14. Create internet gateway: name tag Three-Tier-Project-IGW.

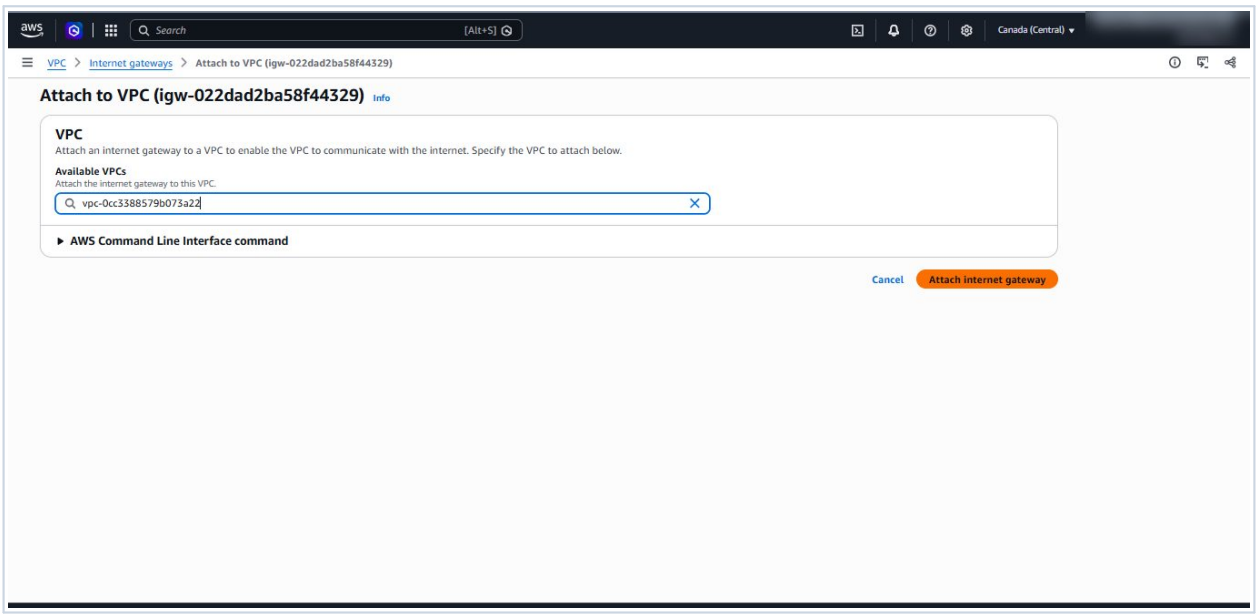


Figure 15. Attach the internet gateway to Three-Tier-Project-VPC.

Create the NAT gateways

Go to **VPC > NAT gateways > Create NAT gateway** and create two gateways. Each one is placed in a *public* web subnet, uses connectivity type **Public** and gets its own Elastic IP (**Allocate Elastic IP**).

Name	Subnet	Connectivity	Elastic IP
NAT-GT-1	Web-Tier-Subnet-1-Public (1a)	Public	Allocate Elastic IP
NAT-GT-2	Web-Tier-Subnet-2-Public (1b)	Public	Allocate Elastic IP

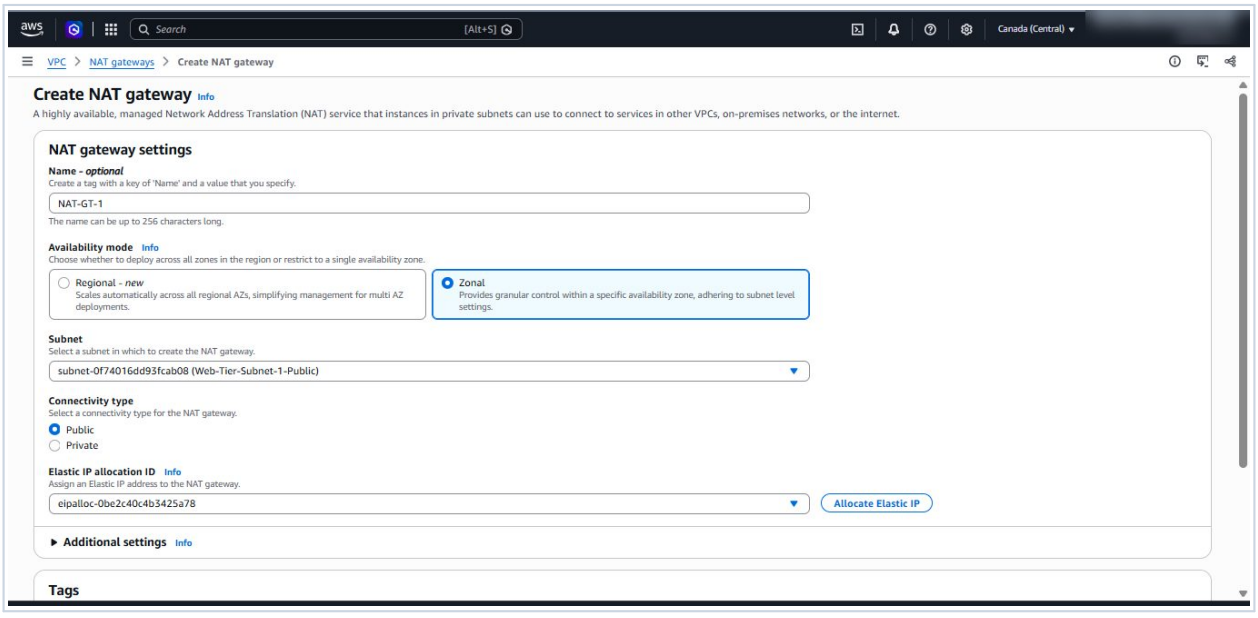


Figure 16. NAT-GT-1 in Web-Tier-Subnet-1-Public, availability mode Zonal, connectivity type Public, with an Elastic IP.

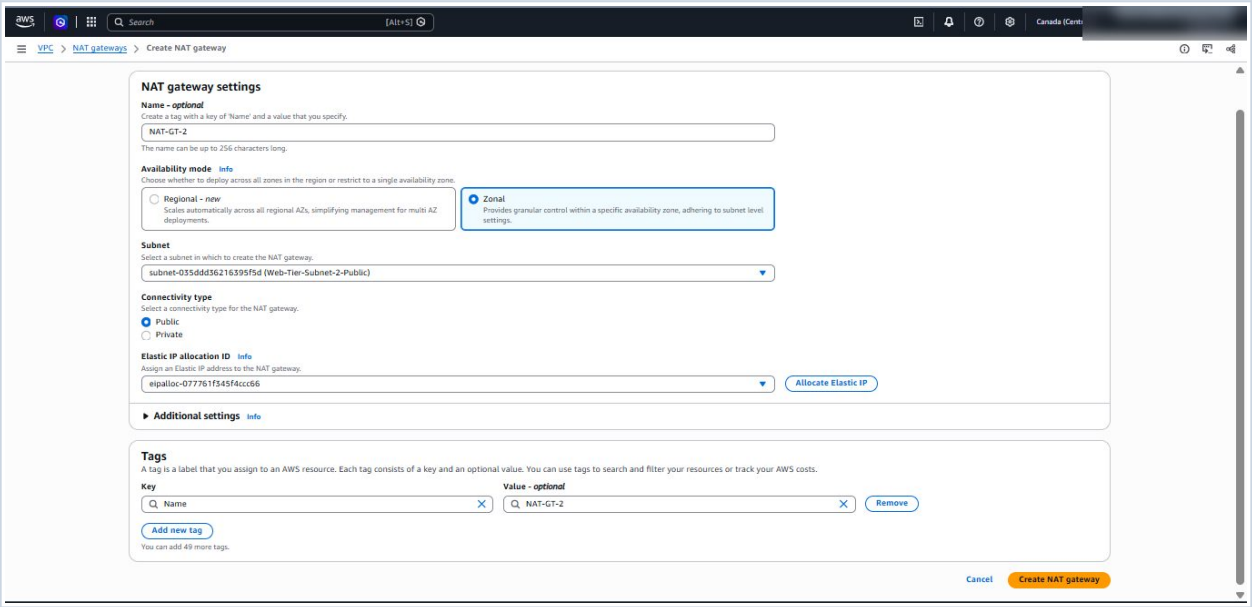


Figure 17. NAT-GT-2 in Web-Tier-Subnet-2-Public, connectivity type Public, with an Elastic IP.

IMPORTANT
A NAT gateway belongs in a **public** subnet. Putting it in a private subnet is the most common reason private servers cannot download packages later.

Create the route tables

Create three route tables in Three-Tier-Project-VPC under **VPC > Route tables > Create route table**: one for the web subnets and one for each app subnet. The database subnets keep the VPC main route table, which only has the local route, so they have no path to the internet at all.

Route table	Default route (0.0.0.0/0)	Associated subnet(s)
Web-Tier-RT	Internet gateway (Three-Tier-Project-IGW)	Web-Tier-Subnet-1-Public, Web-Tier-Subnet-2-Public
App-Tier-RT-1	NAT-GT-1	App-Tier-Subnet-1-Private
App-Tier-RT-2	NAT-GT-2	App-Tier-Subnet-2-Private

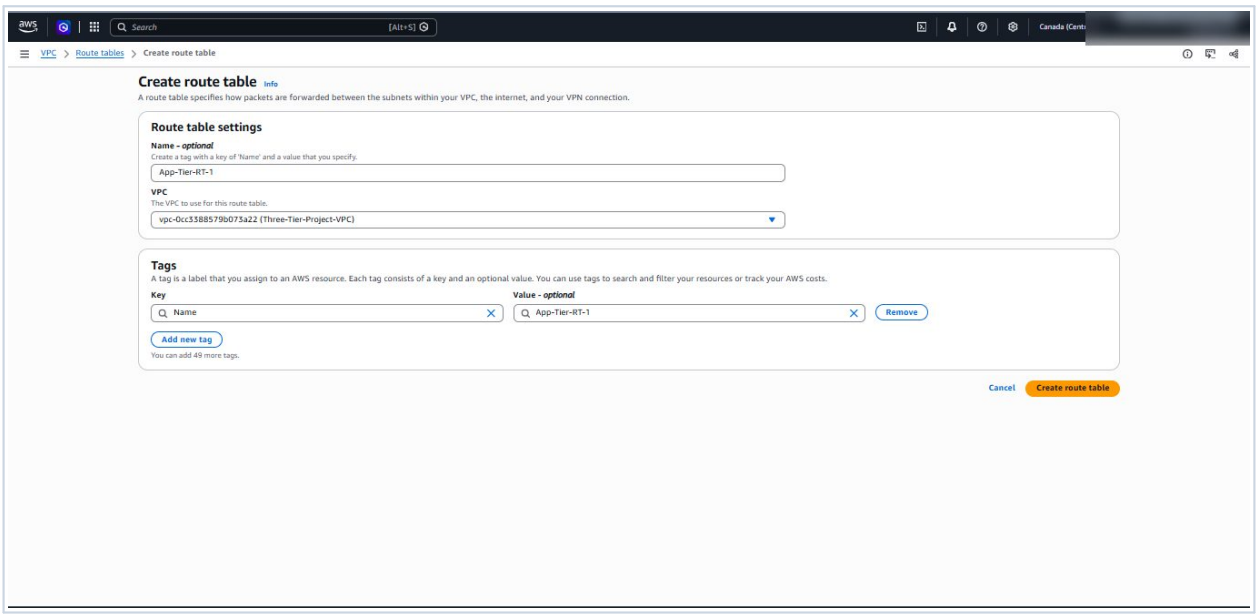


Figure 18. Route table App-Tier-RT-1 in Three-Tier-Project-VPC.

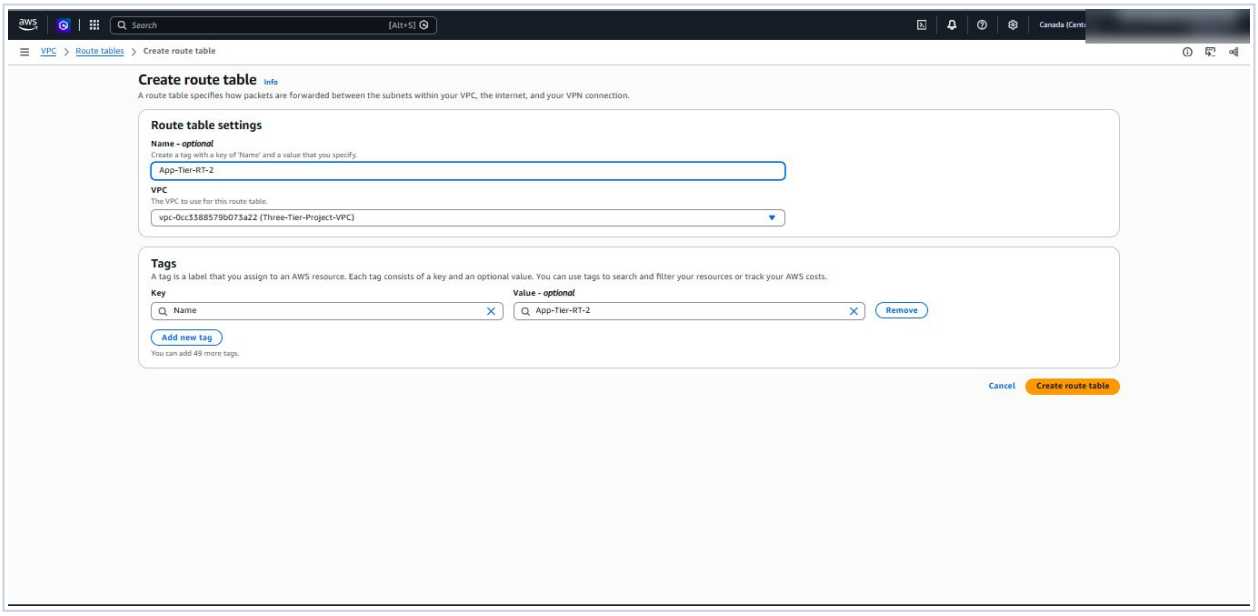


Figure 19. Route table App-Tier-RT-2 in Three-Tier-Project-VPC.

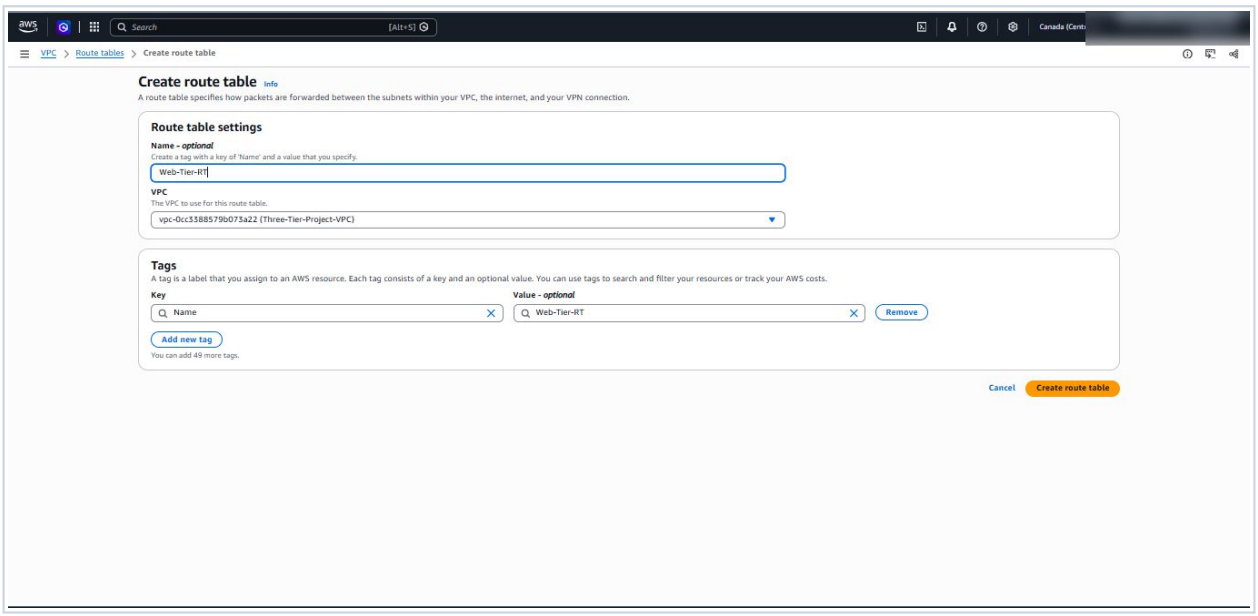


Figure 20. Route table Web-Tier-RT in Three-Tier-Project-VPC.

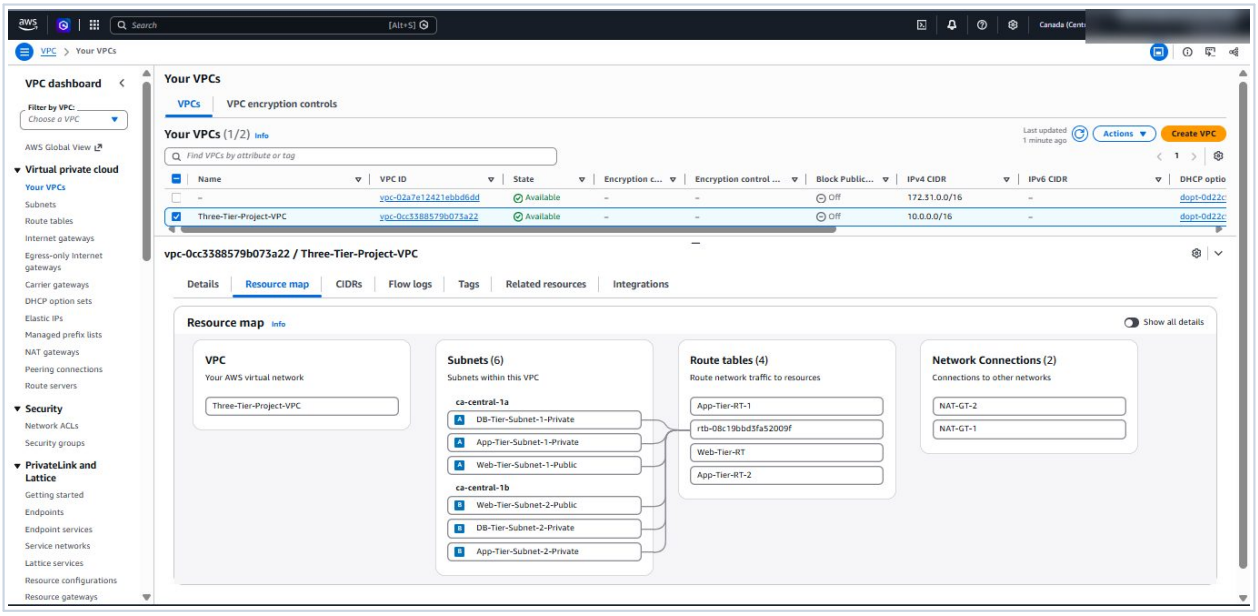


Figure 21. The VPC resource map right after the three tables exist: nothing is routed yet.

Edit routes and subnet associations

For each table: select it, choose **Actions > Edit routes**, add **0.0.0.0/0** with the right target, save, then choose **Actions > Edit subnet associations** and tick the subnets from the table above.

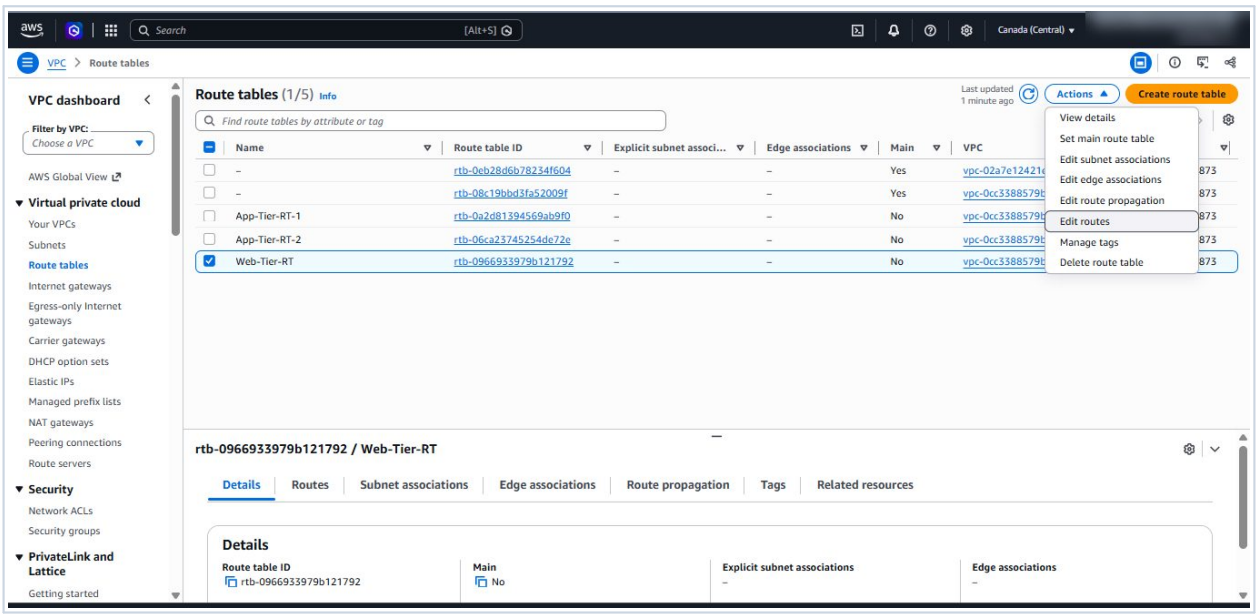


Figure 22. Select Web-Tier-RT and open Actions > Edit routes.

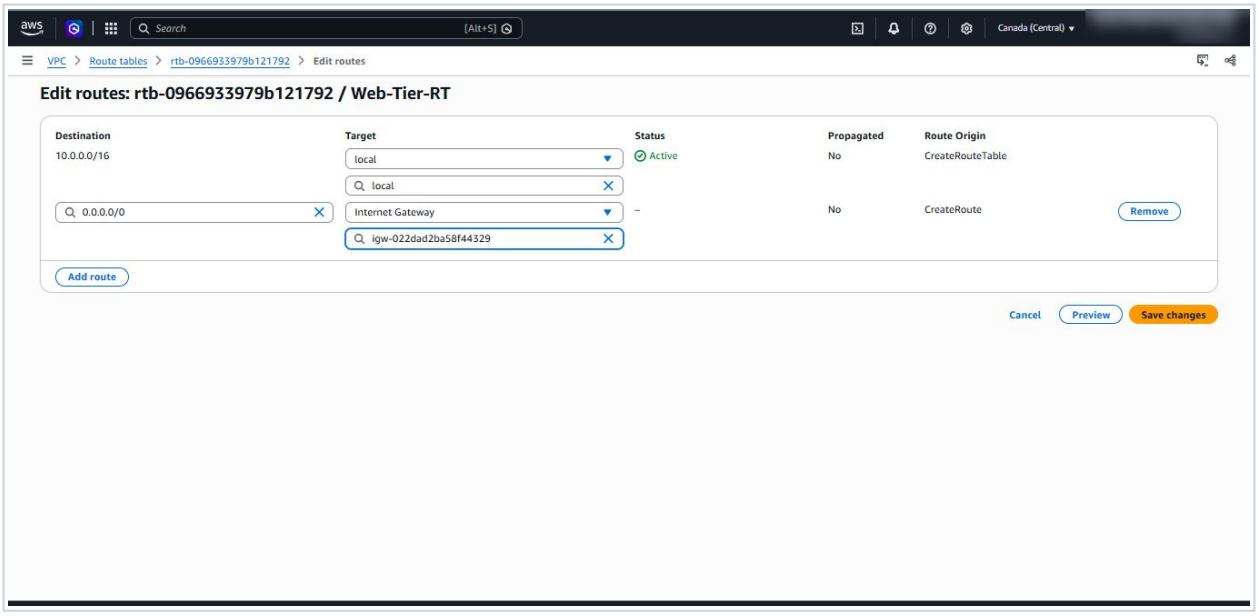


Figure 23. Web-Tier-RT: add 0.0.0.0/0 with the internet gateway as the target and save.

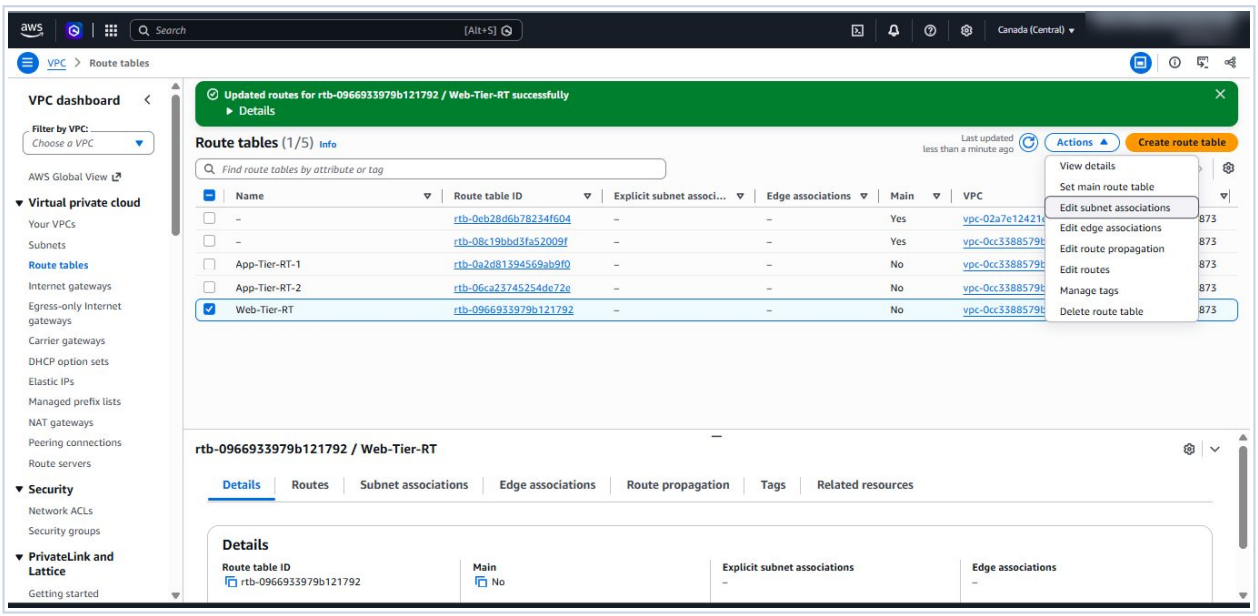


Figure 24. Routes saved for Web-Tier-RT. Next: Actions > Edit subnet associations.

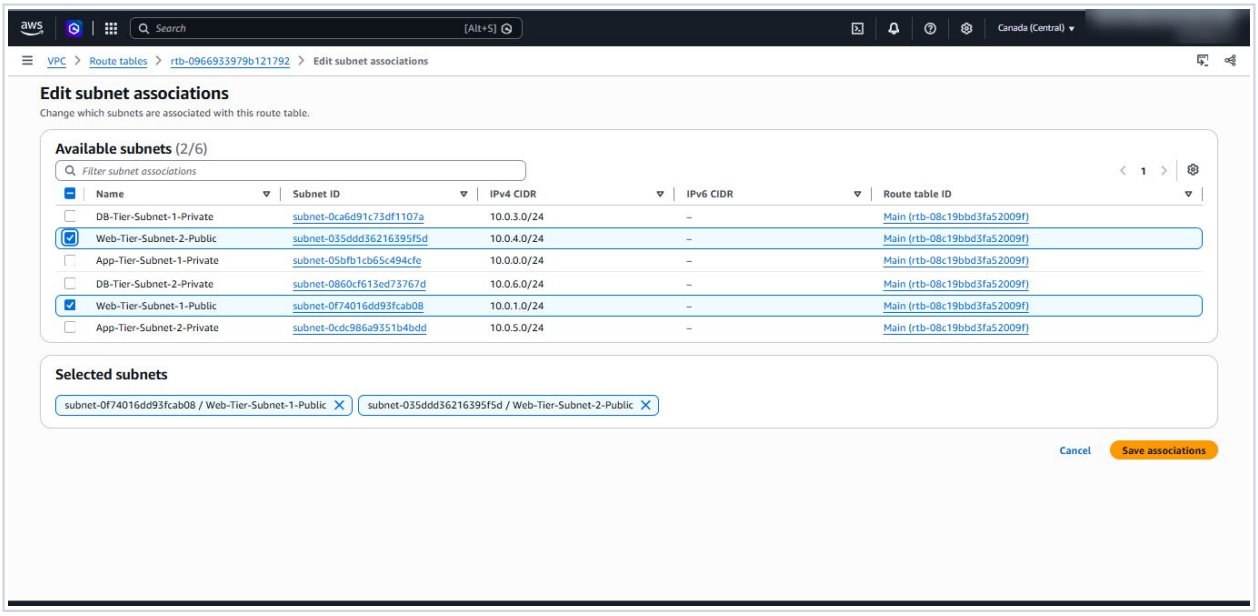


Figure 25. Associate both public web subnets with Web-Tier-RT.

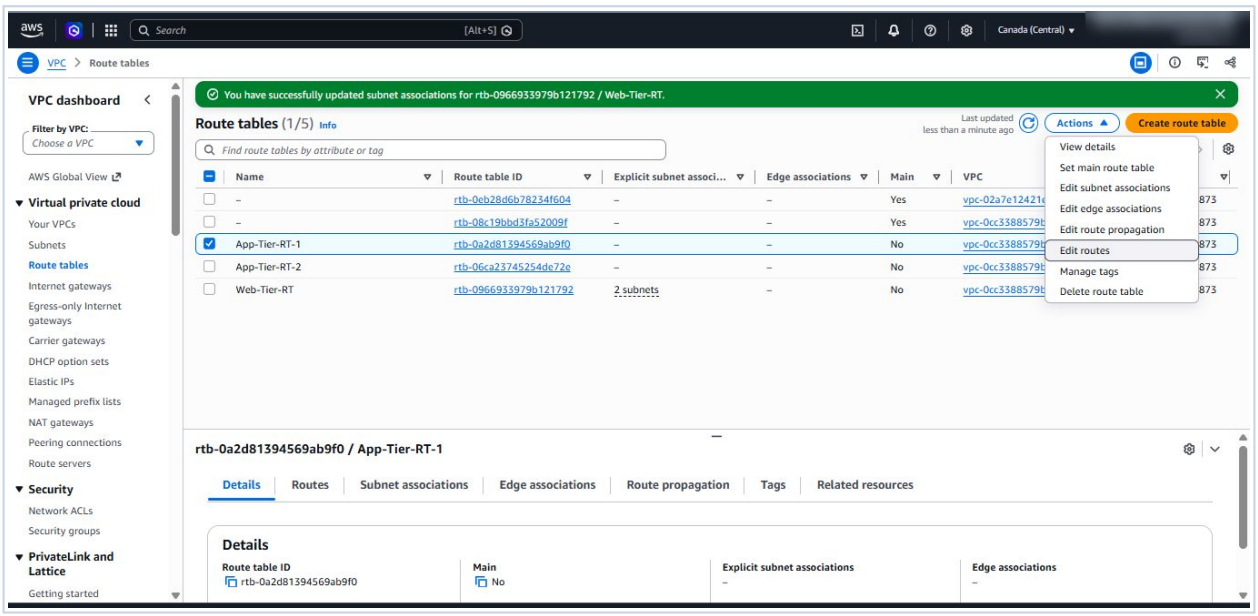


Figure 26. Associations saved. Now select App-Tier-RT-1 and open Edit routes.

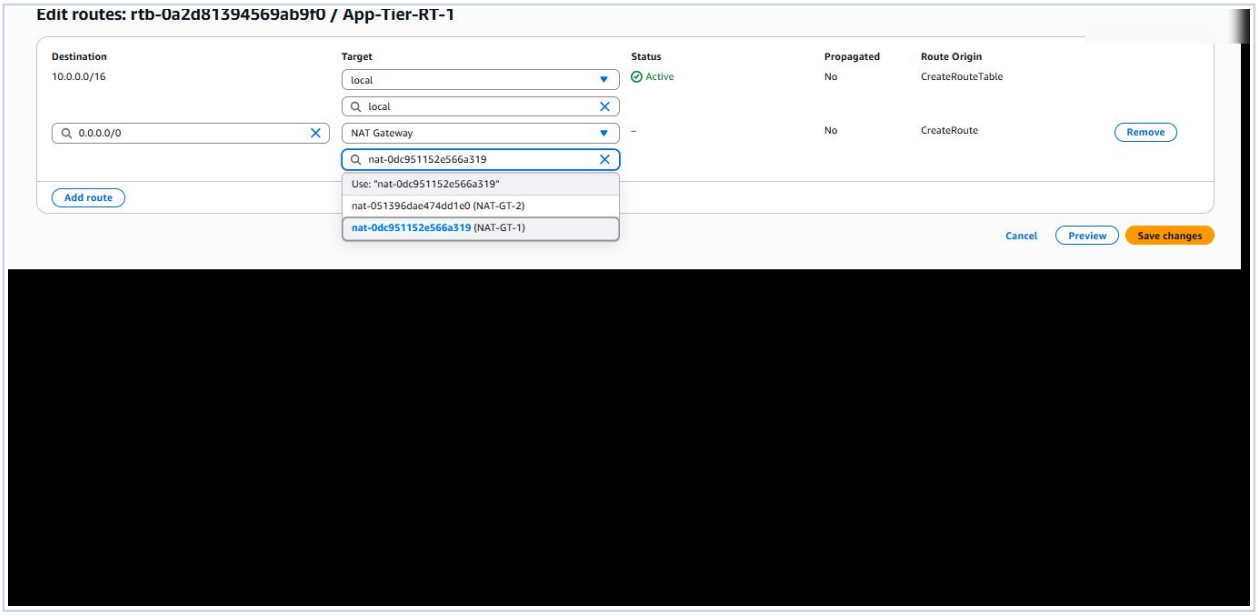


Figure 27. App-Tier-RT-1: add 0.0.0.0/0 and choose the NAT gateway in the same Availability Zone (NAT-GT-1).

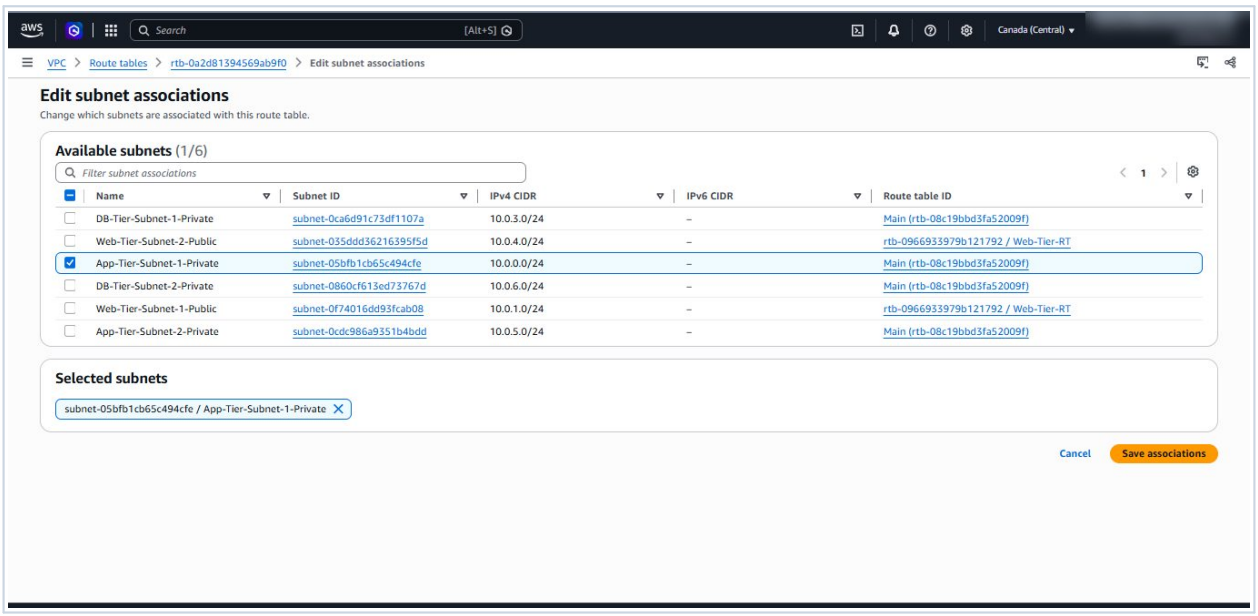


Figure 28. Associate App-Tier-Subnet-1-Private with App-Tier-RT-1.

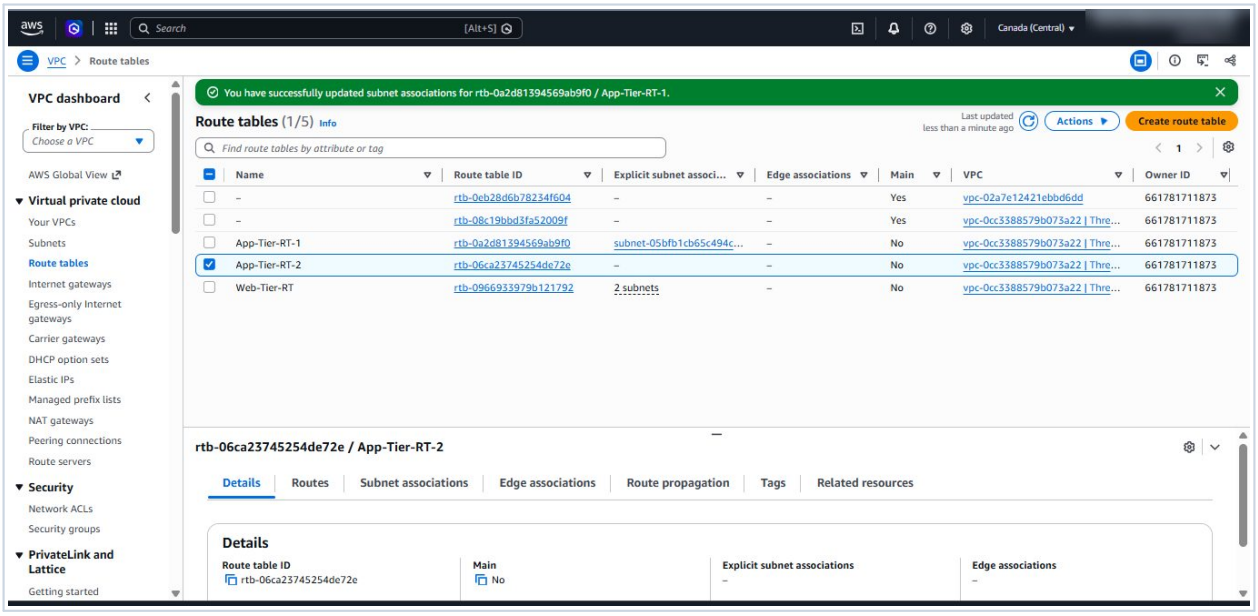


Figure 29. Associations saved for App-Tier-RT-1. Now select App-Tier-RT-2.

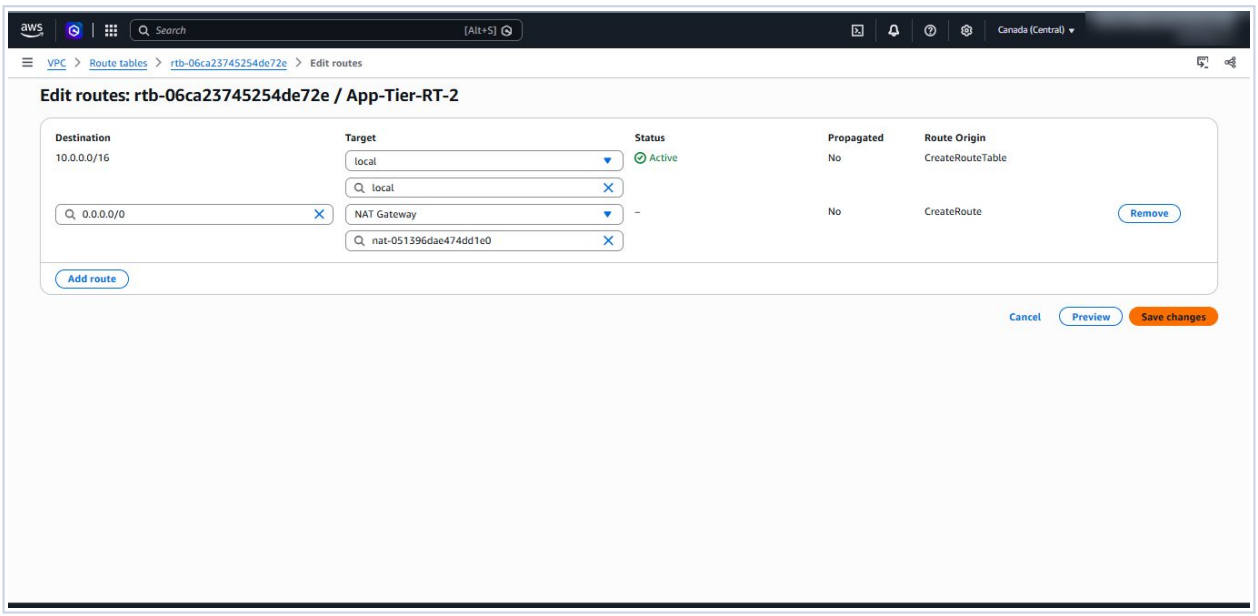


Figure 30. App-Tier-RT-2: add 0.0.0.0/0 with NAT-GT-2 as the target and save.

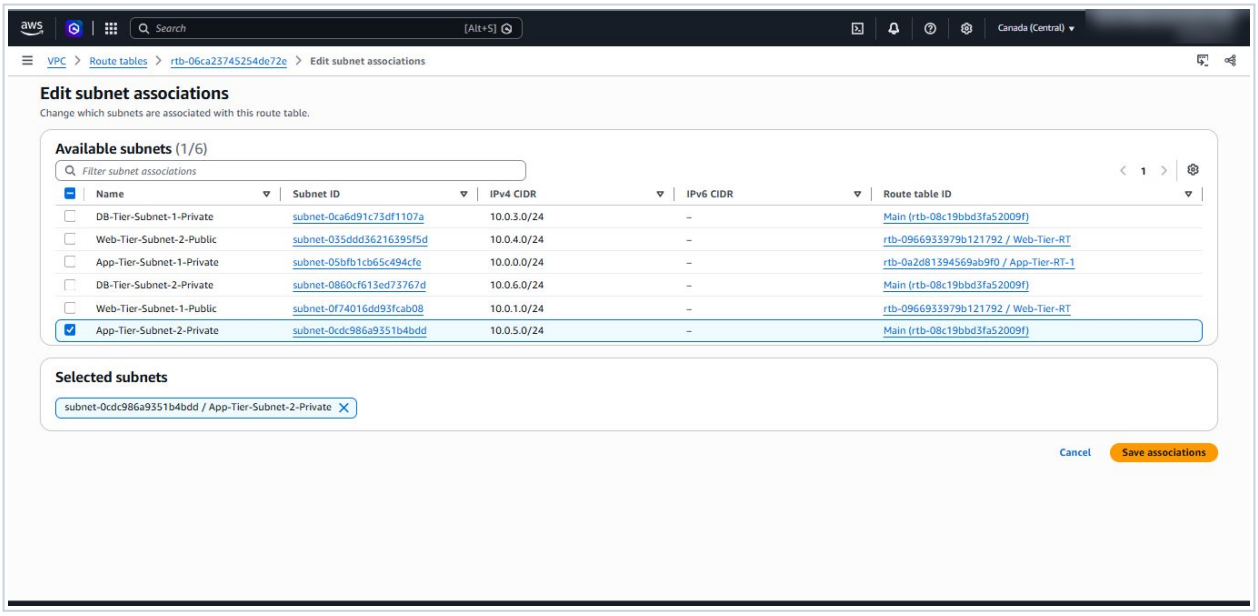


Figure 31. Associate App-Tier-Subnet-2-Private with App-Tier-RT-2.

Verify with the resource map

Open **Your VPCs**, select Three-Tier-Project-VPC and click the **Resource map** tab. Every subnet should line up with its route table, and every route table with its gateway.

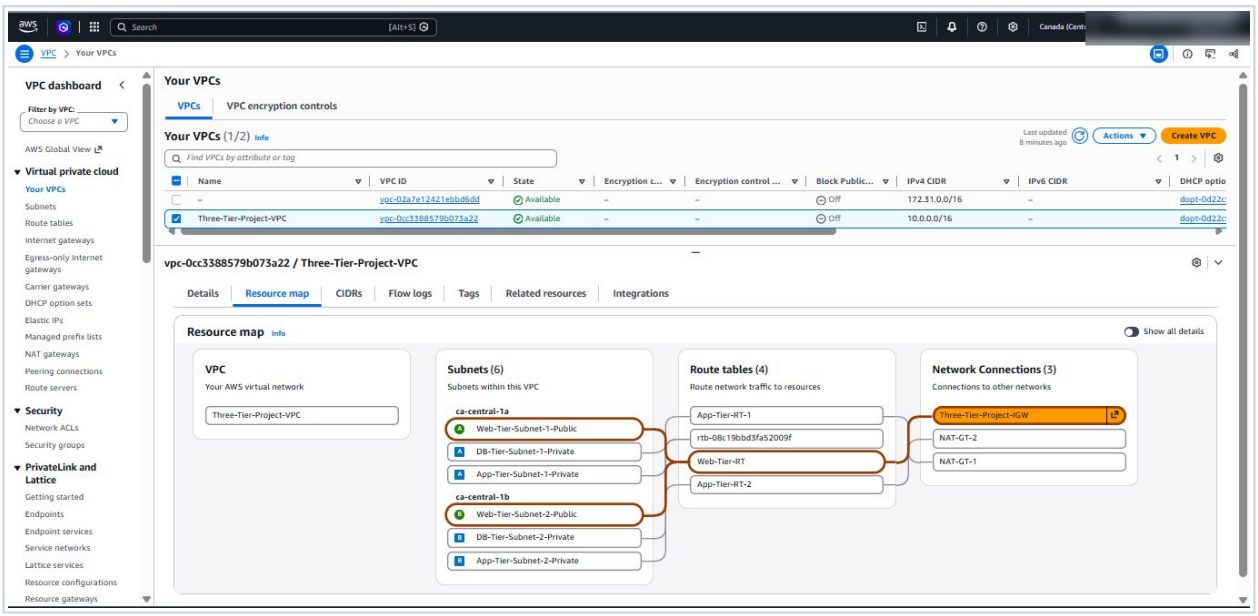


Figure 32. Resource map: confirm that each subnet is attached to the route table you intended.

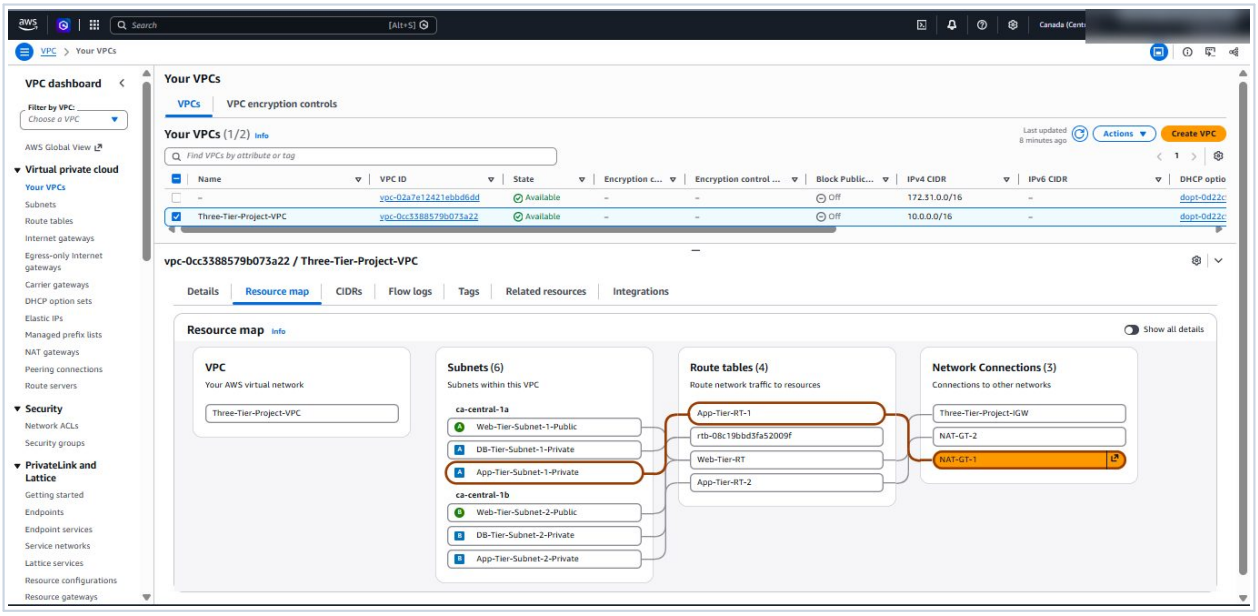


Figure 33. Resource map with the NAT gateway connection highlighted for one app subnet.

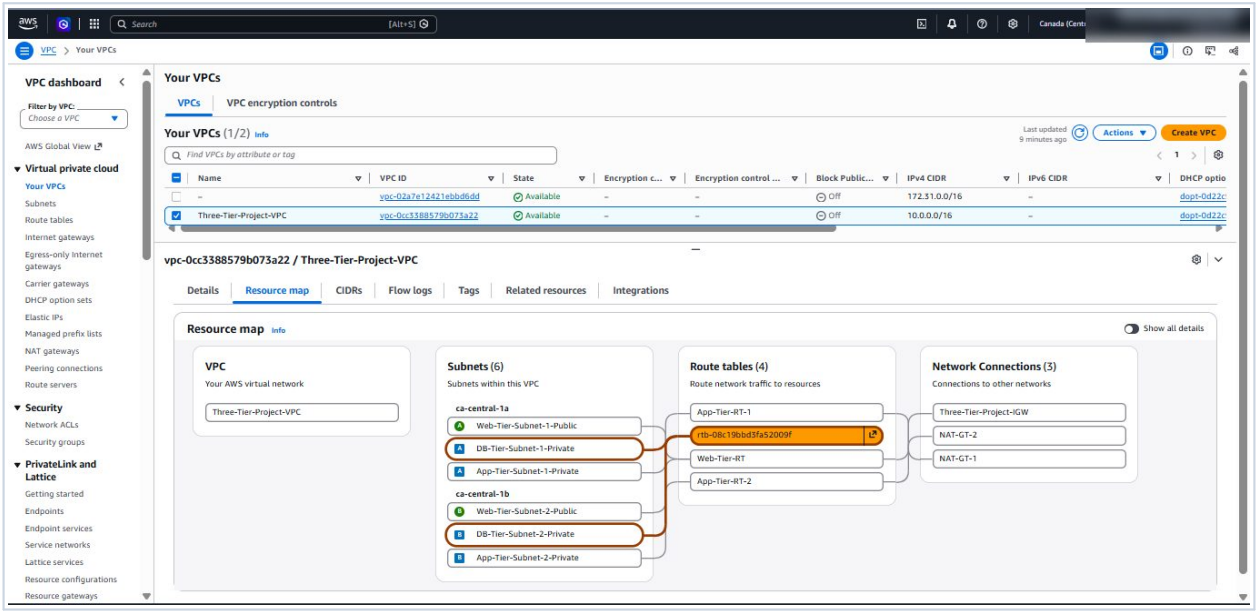


Figure 34. Resource map: final check of subnets, route tables and network connections.

CHECKPOINT

Web subnets route to the internet gateway, each app subnet routes to the NAT gateway in its own zone, and the database subnets have local routing only.

STEP 03 OF 15

NETWORK FOUNDATION

Create the Security Groups

Post a guard at every door

WHY THIS STEP?

Each tier should accept traffic only from the tier in front of it. Security groups make that rule real, so the database cannot be reached from the internet.

Create five security groups in Three-Tier-Project-VPC. Go to **VPC > Security groups > Create security group**, enter the name (I reused it as the description), choose the VPC and add the single inbound rule from the table.

Security group	Inbound type (port)	Source
External-Load-Balancer-SG	HTTP (80)	0.0.0.0/0 (Anywhere-IPv4)
Web-Tier-SG	HTTP (80)	External-Load-Balancer-SG
Internal-Load-Balancer-SG	HTTP (80)	Web-Tier-SG
App-Tier-SG	Custom TCP (4000)	Internal-Load-Balancer-SG
DB-Tier-SG	MYSQL/Aurora (3306)	App-Tier-SG

IMPORTANT

Create the groups in this order. A security group can only be chosen as a source after it exists. For rows 2 to 5, set **Source type** to **Custom** and pick the previous tier's security group, not an IP range.

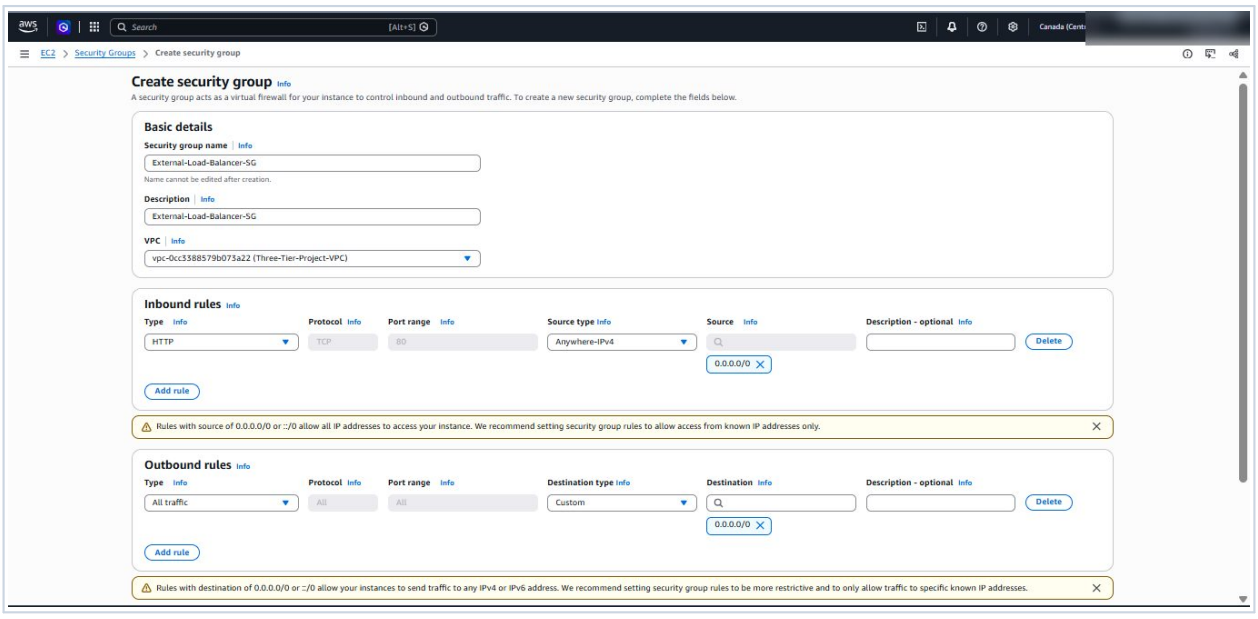


Figure 35. External-Load-Balancer-SG: inbound HTTP (80) from 0.0.0.0/0.

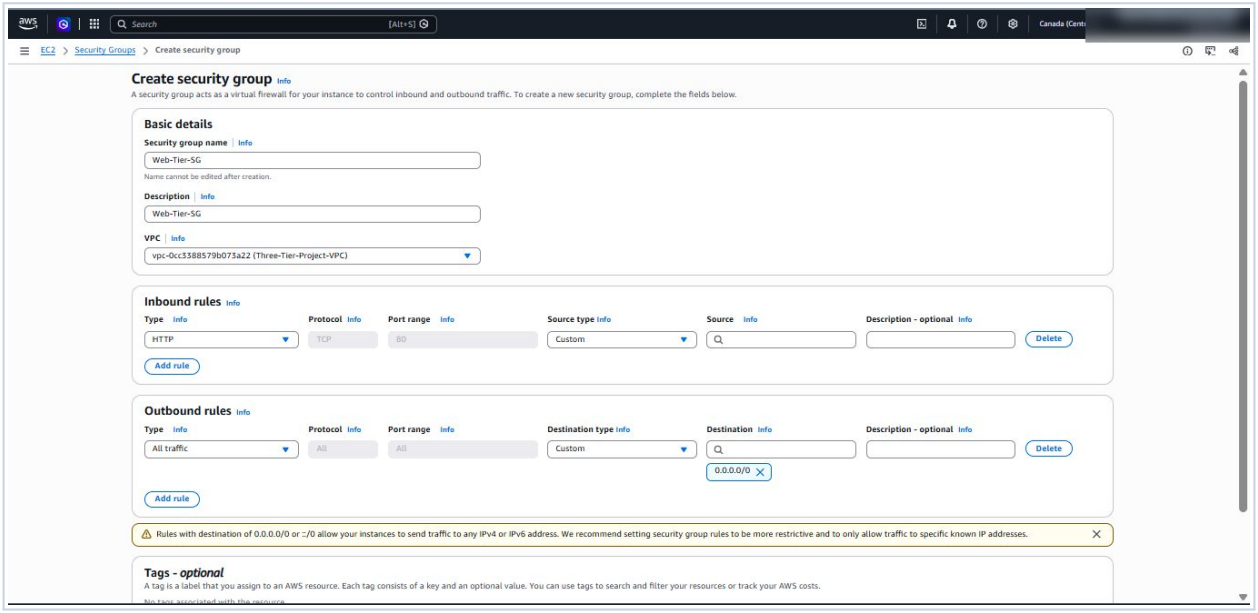


Figure 36. Web-Tier-SG: name, description and VPC, with inbound HTTP (80) from External-Load-Balancer-SG.

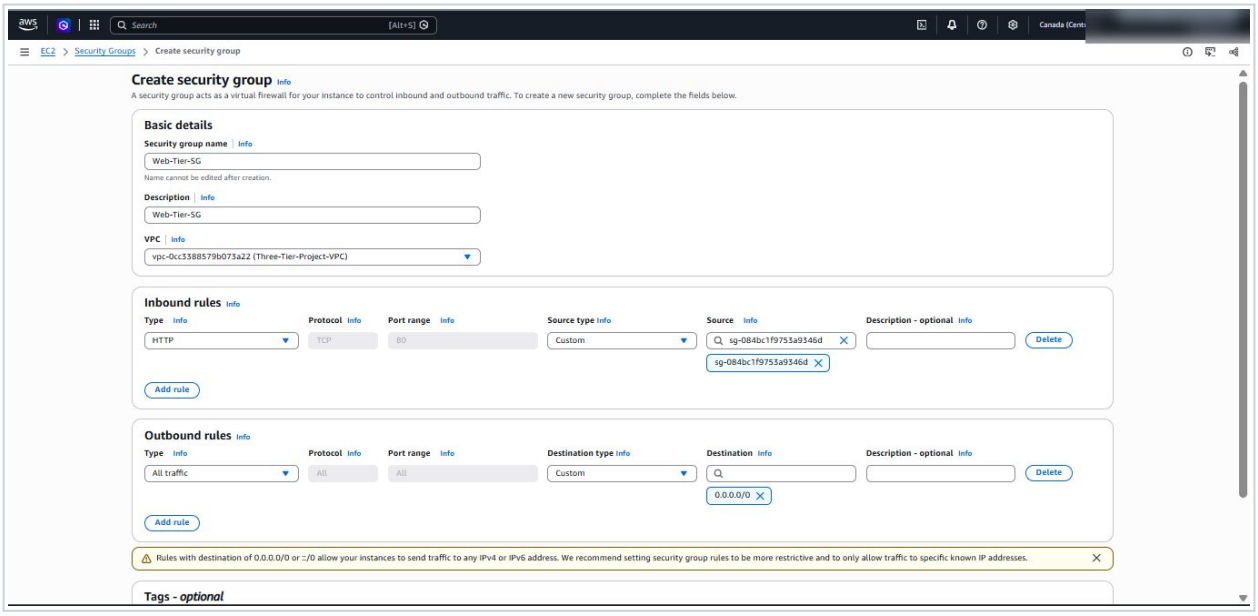


Figure 37. Web-Tier-SG: review the inbound rule and the default outbound rule before creating the group.

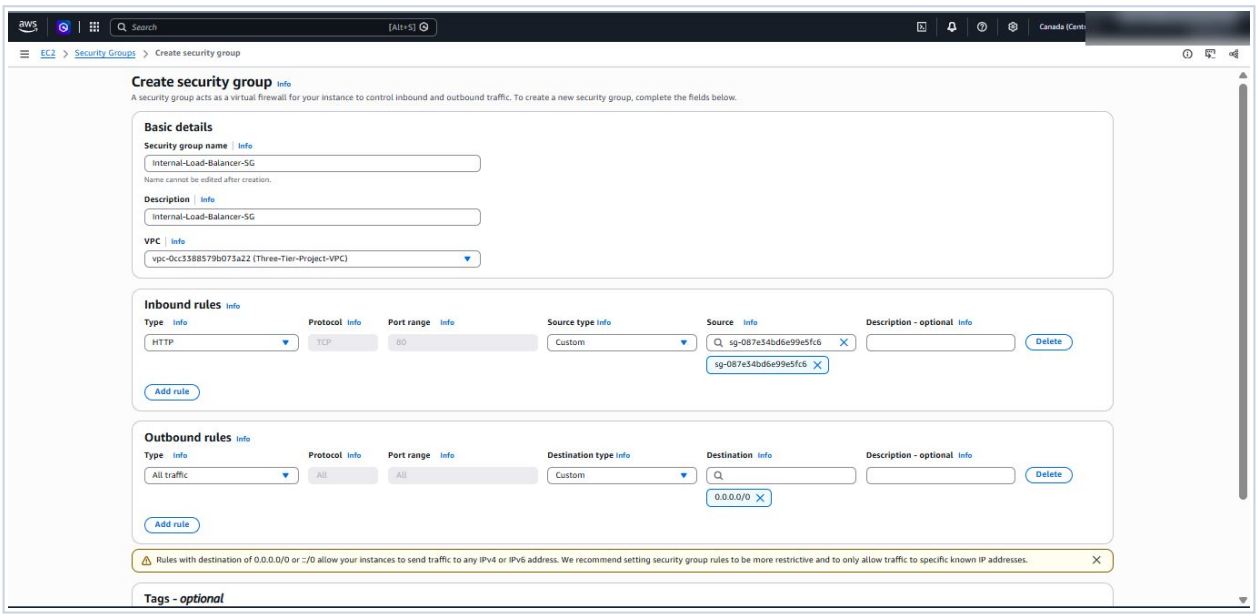


Figure 38. Internal-Load-Balancer-SG: inbound HTTP (80) from Web-Tier-SG.

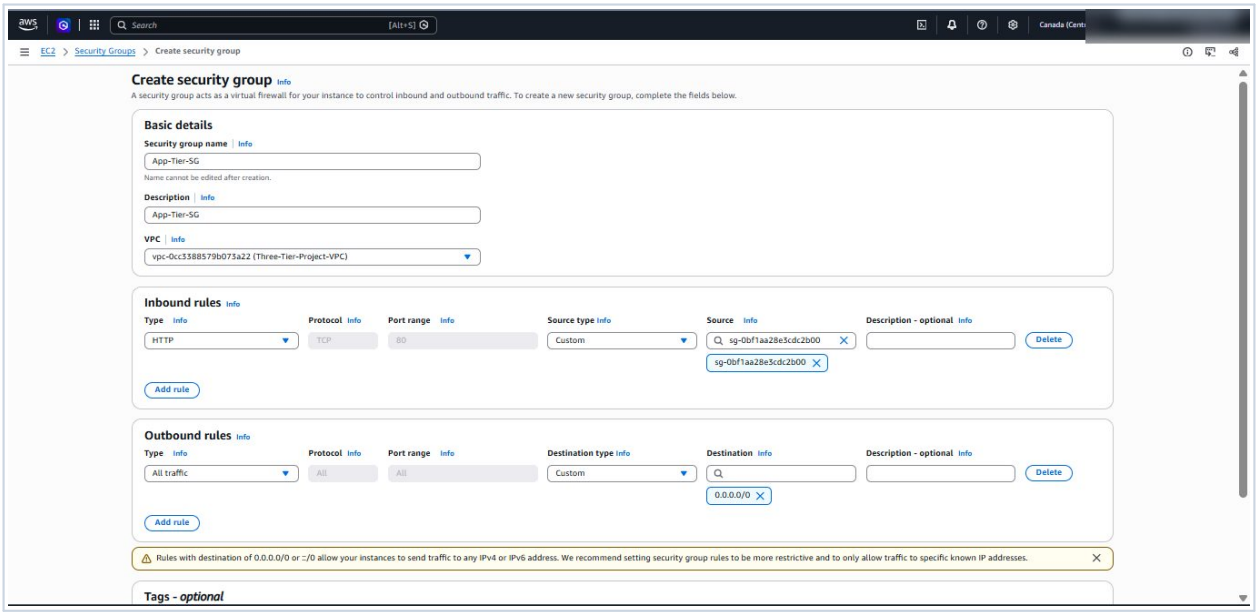


Figure 39. App-Tier-SG: inbound Custom TCP 4000 from Internal-Load-Balancer-SG.

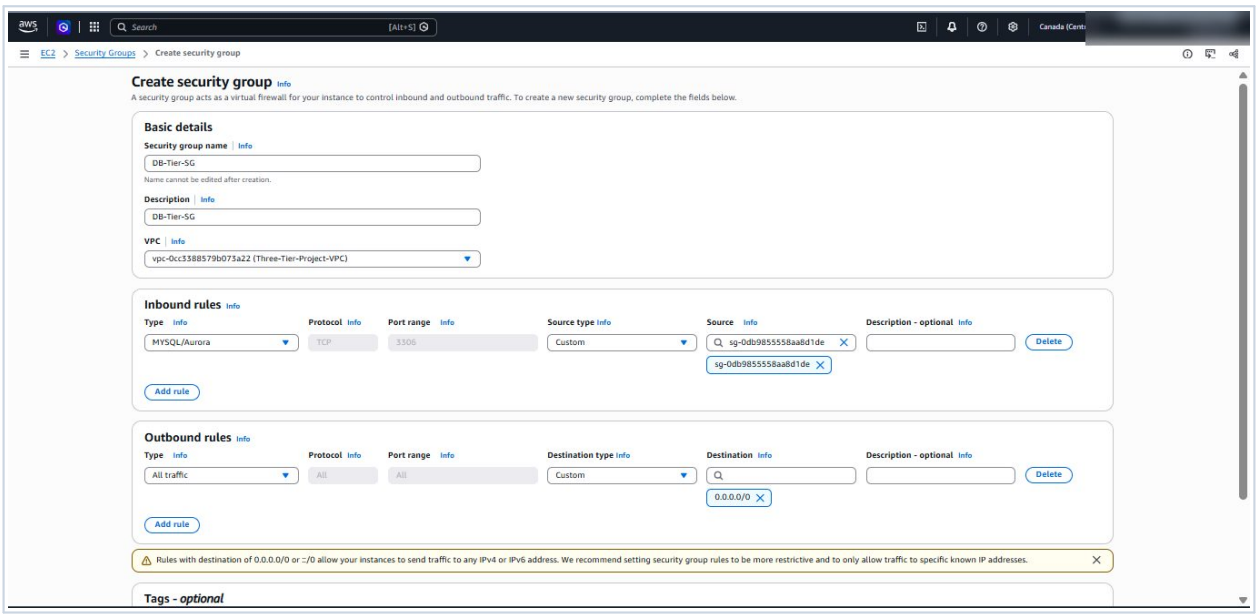


Figure 40. DB-Tier-SG: inbound MYSQL/Aurora 3306 from App-Tier-SG.

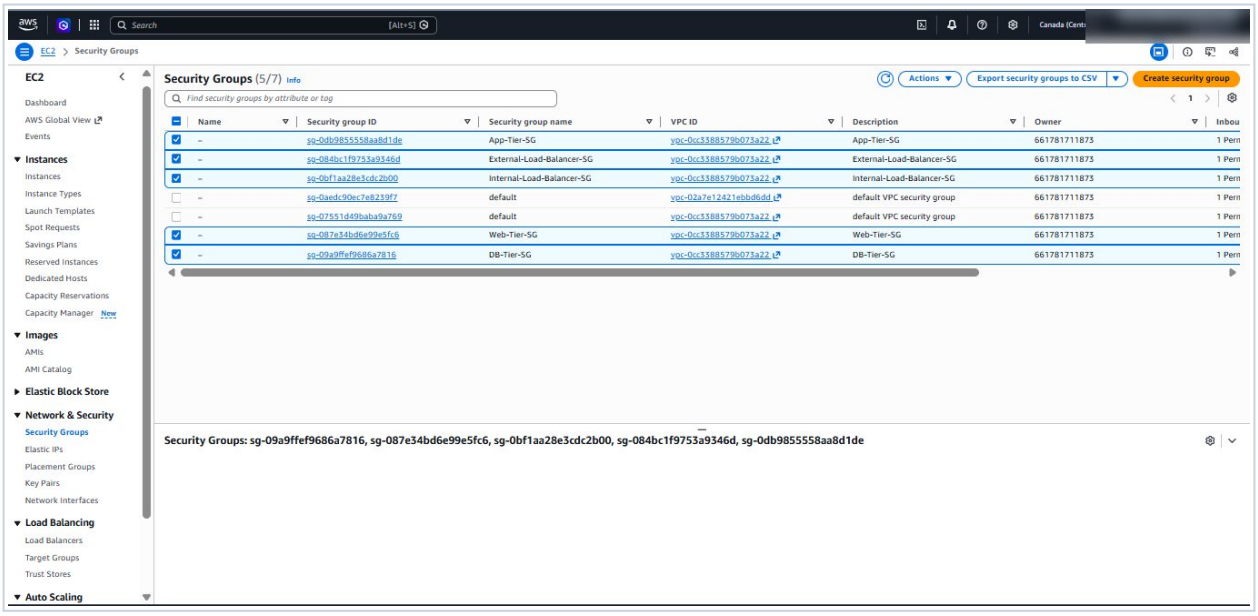


Figure 41. The Security groups list showing the five project groups next to the default groups.

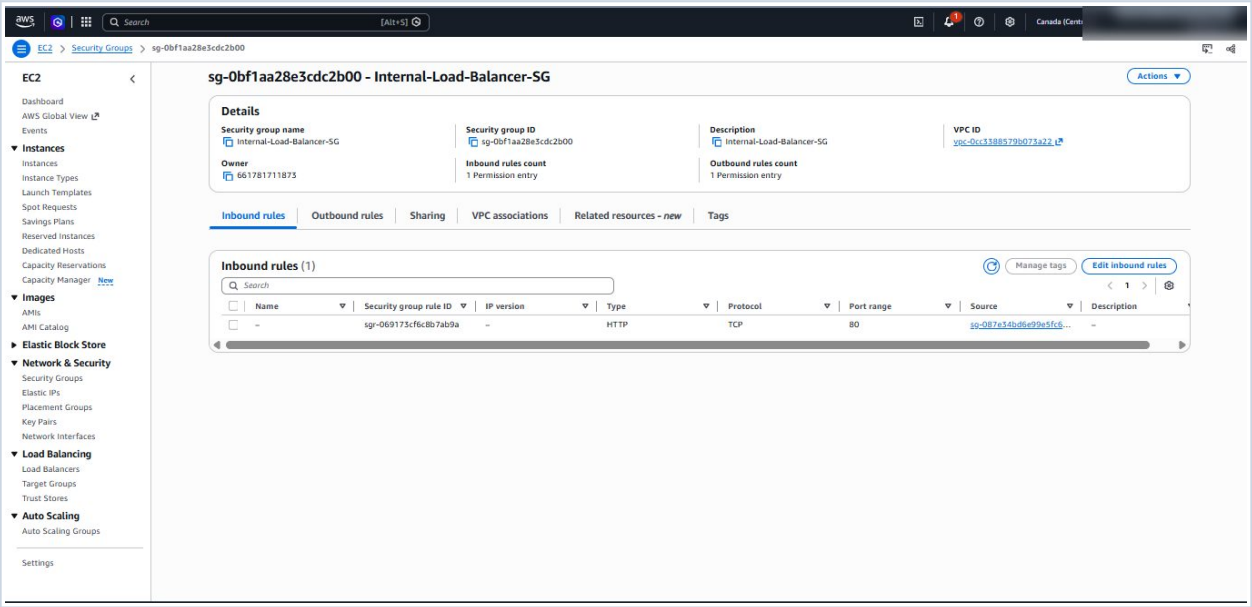


Figure 42. Internal-Load-Balancer-SG details: one inbound rule, TCP 80, with the Web tier group as the source.

CHECKPOINT

Five security groups exist in your VPC. Phase 1 is done: network, routes and firewalls are ready. Next come the data and access pieces.

STEP 04 OF 15 DATA AND ACCESS

Create the DB Subnet Group and the RDS Database

Give the data a protected home

WHY THIS STEP?

RDS needs to know which subnets it may use. A DB subnet group lists them, and AWS requires its subnets to be in different Availability Zones. That is what lets you add a standby later and keeps the database in private address space.

Create the DB subnet group

1. Search for **RDS** (Aurora and RDS), then open **Subnet groups > Create DB subnet group**.
2. Name: Three-Tier-Project-DB-Subnet-Group (same text as the description). VPC: **Three-Tier-Project-VPC**.
3. Availability Zones: **ca-central-1a** and **ca-central-1b**.
4. Subnets: the two database subnets, **10.0.3.0/24** and **10.0.6.0/24**. Click **Create**.

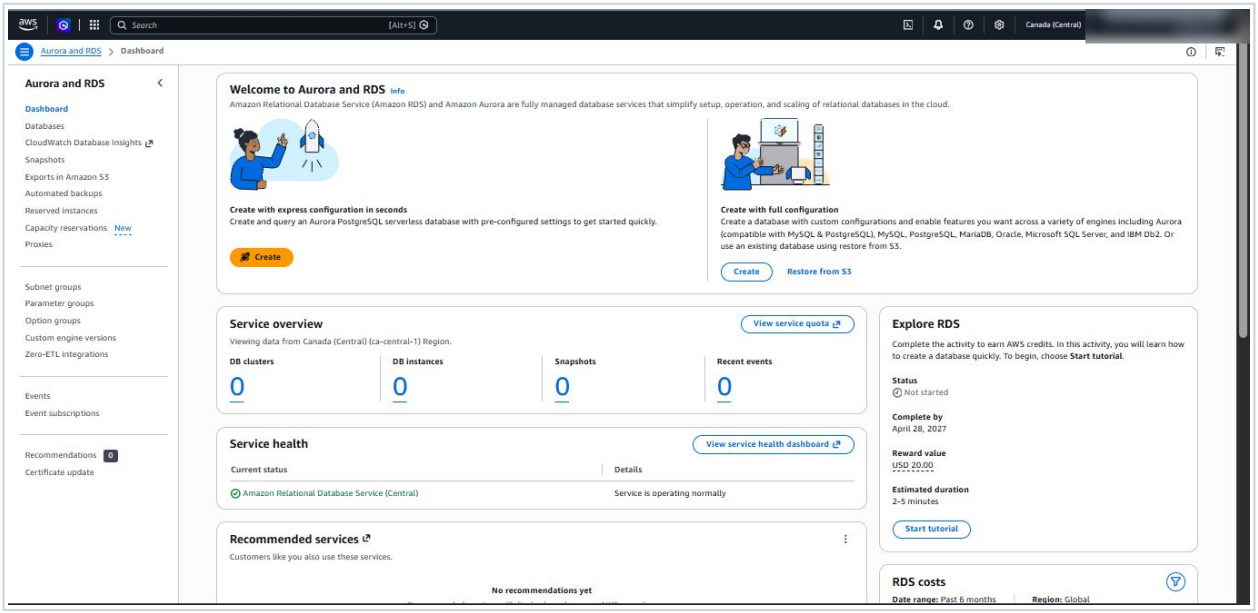


Figure 43. The Amazon Aurora and RDS dashboard.

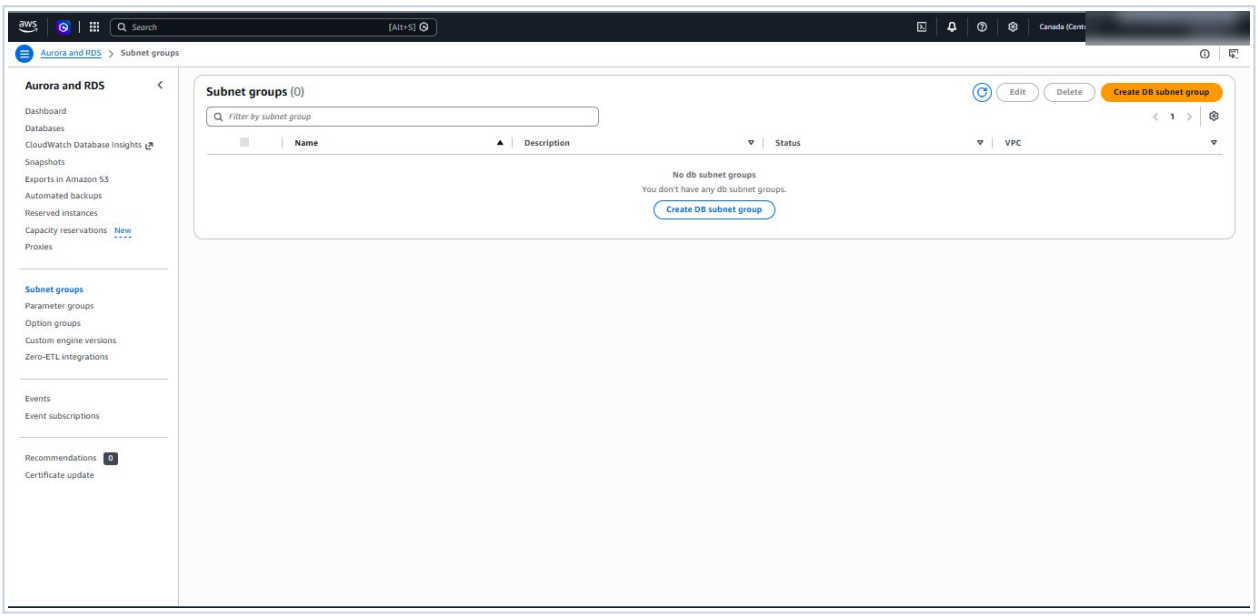


Figure 44. Subnet groups is empty: click Create DB subnet group.

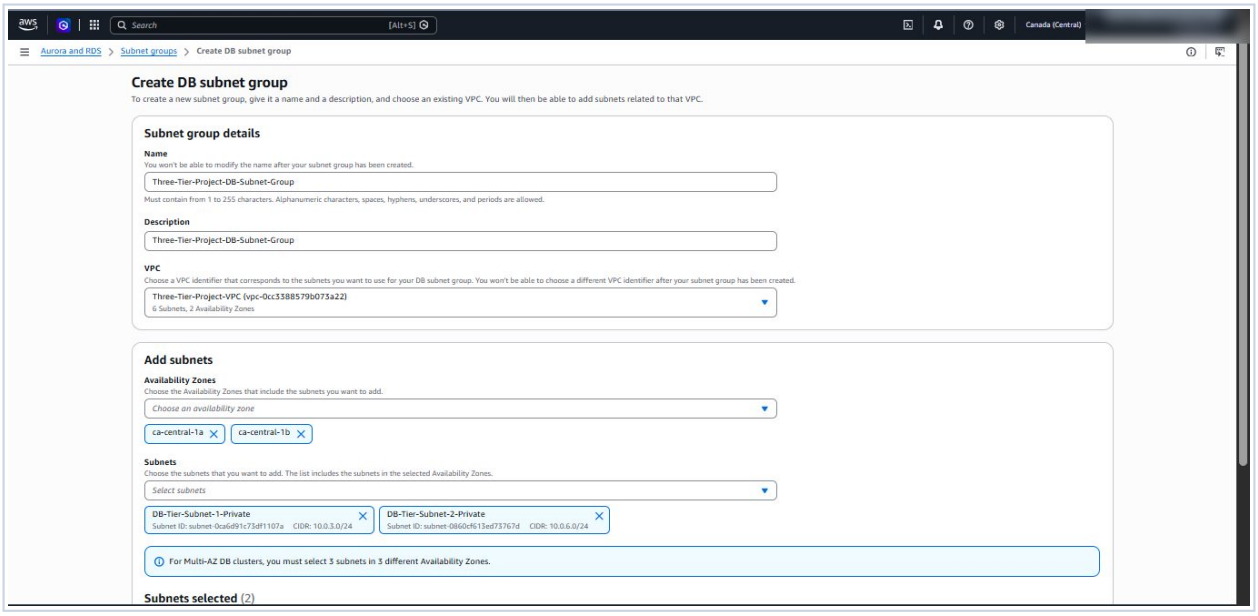


Figure 45. Three-Tier-Project-DB-Subnet-Group with DB-Tier-Subnet-1-Private and DB-Tier-Subnet-2-Private selected.

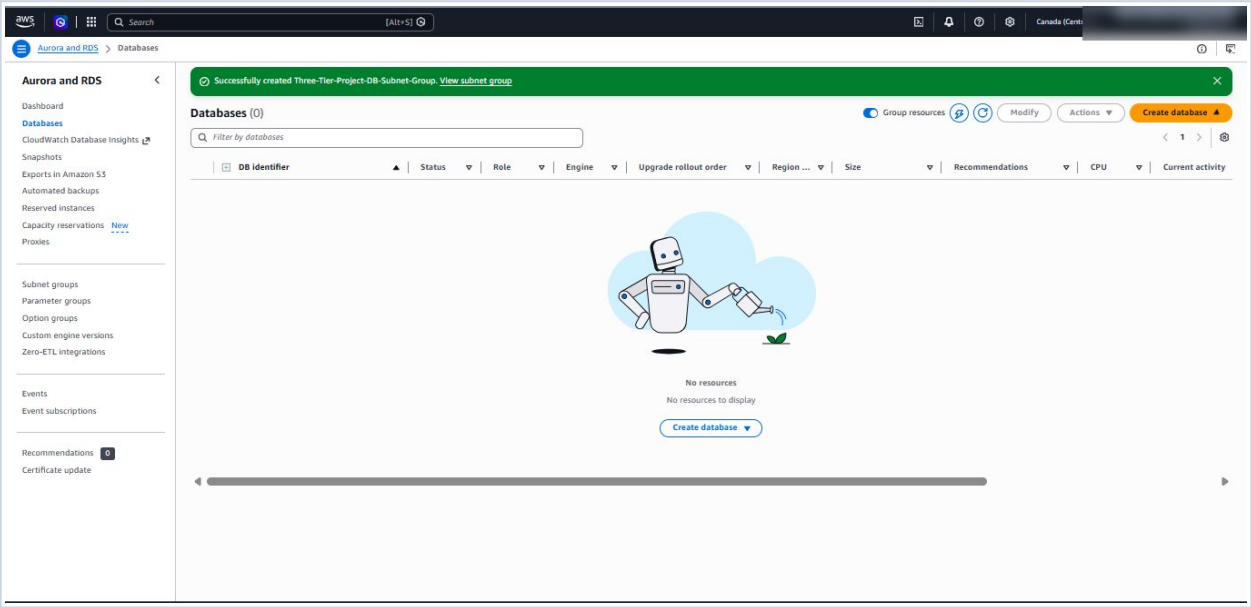


Figure 46. The subnet group is created. Next, open Databases and click Create database.

Create the database

Choose **Full configuration** and the **MySQL** engine, then fill the settings as below and keep everything else at its default.

Setting	Value
Engine / version	MySQL Community, version 8.4.11
Template / deployment	Free tier template, Single-AZ DB instance deployment (one instance)
DB instance identifier	webapptier
Master username	admin
Credentials management	Self managed (you type a master password)
Compute resource connection	Do not connect to an EC2 compute resource
VPC / DB subnet group	Three-Tier-Project-VPC / three-tier-project-db-subnet-group
Public access	No
VPC security group	Choose existing: DB-Tier-SG (remove the default group)
Monitoring	Database Insights - Standard

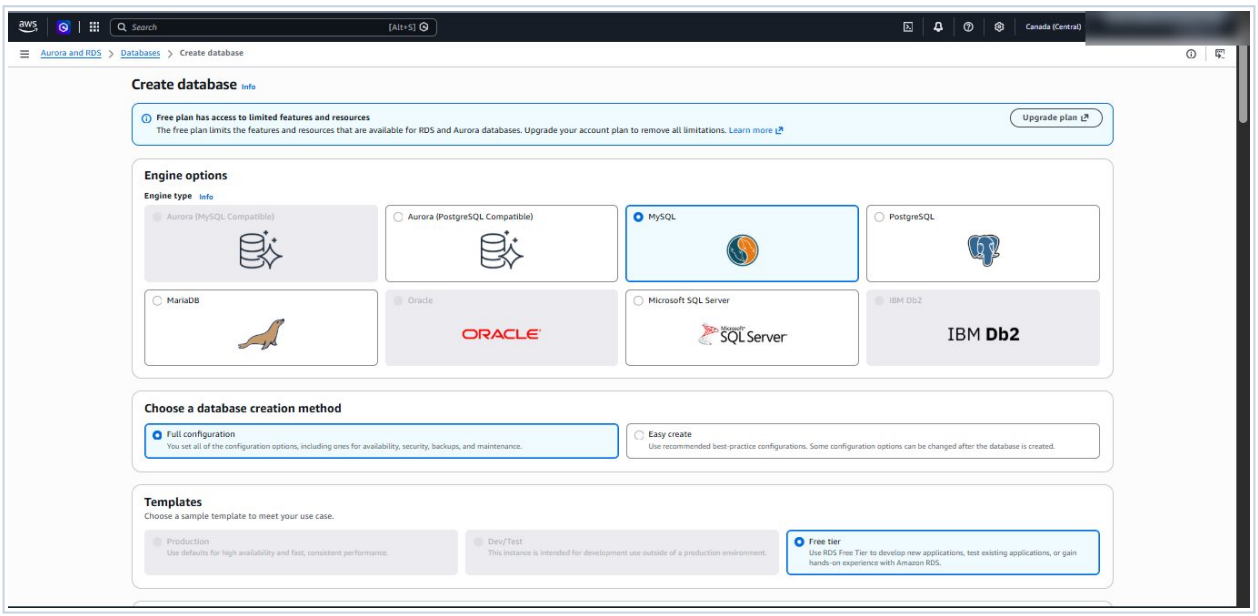


Figure 47. Create database: Full configuration, engine MySQL, template Free tier.

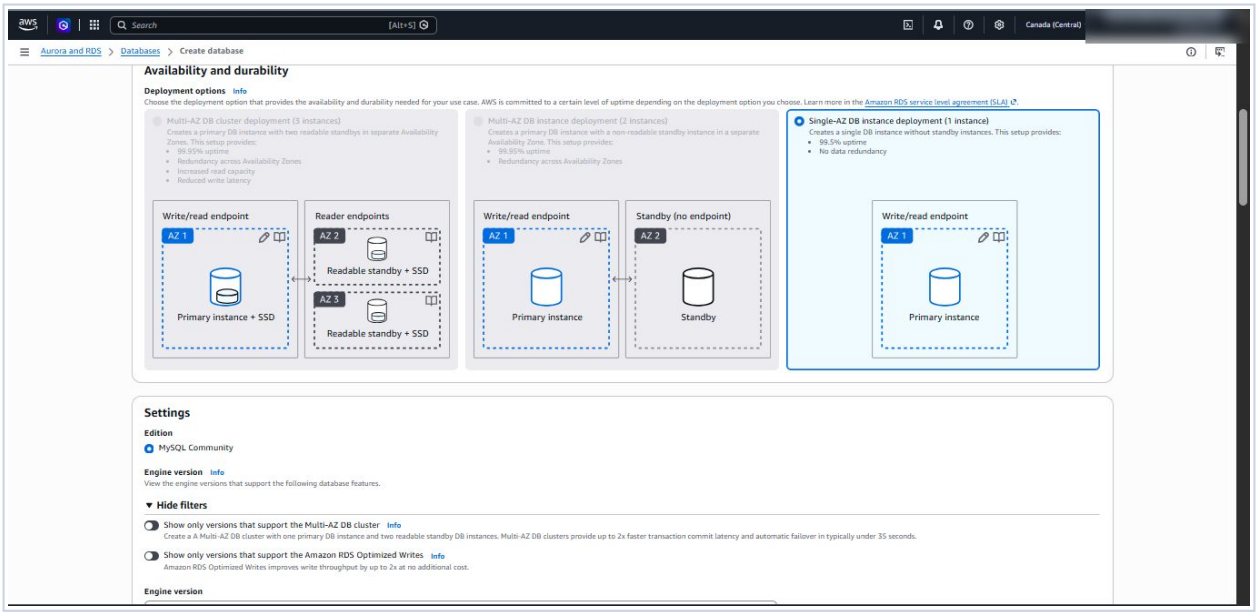


Figure 48. Availability and durability: Single-AZ DB instance deployment, with MySQL community engine 8.4.11.

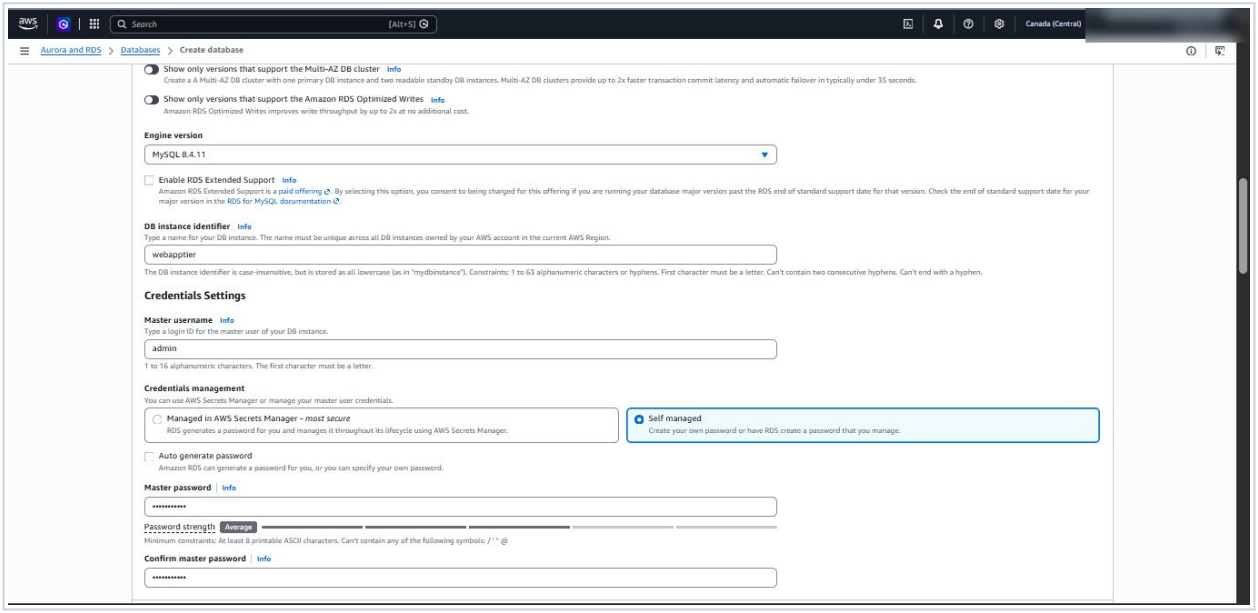


Figure 49. Engine version, DB instance identifier webapptier, master user admin and a self-managed password.

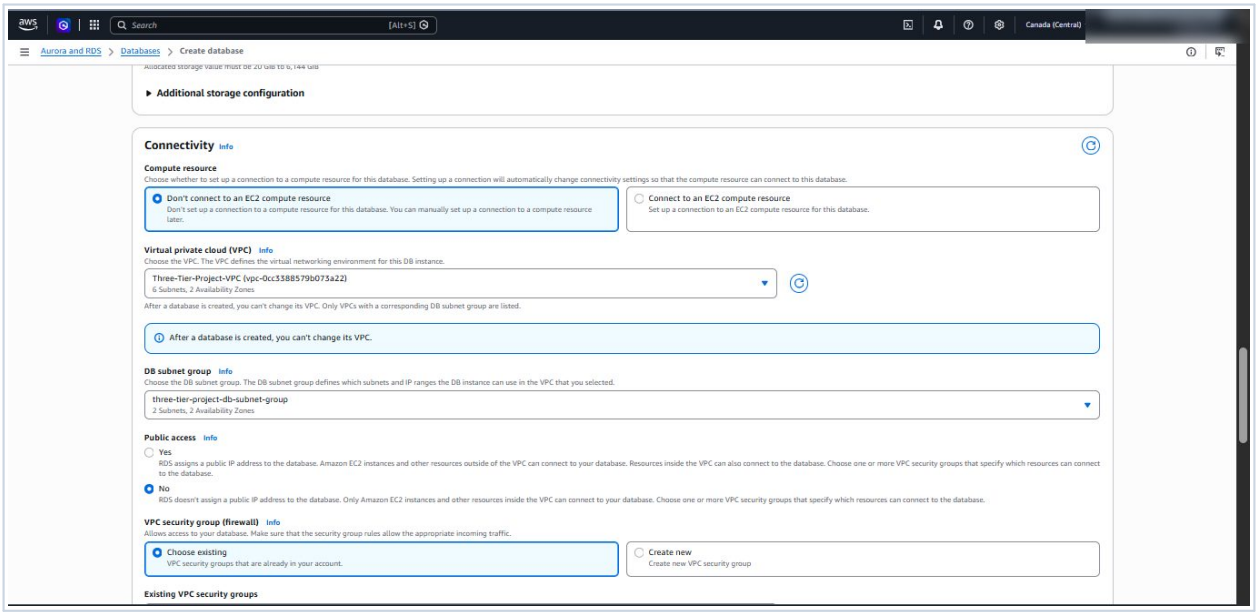


Figure 50. Connectivity: project VPC, project DB subnet group, public access No, existing security group DB-Tier-SG.

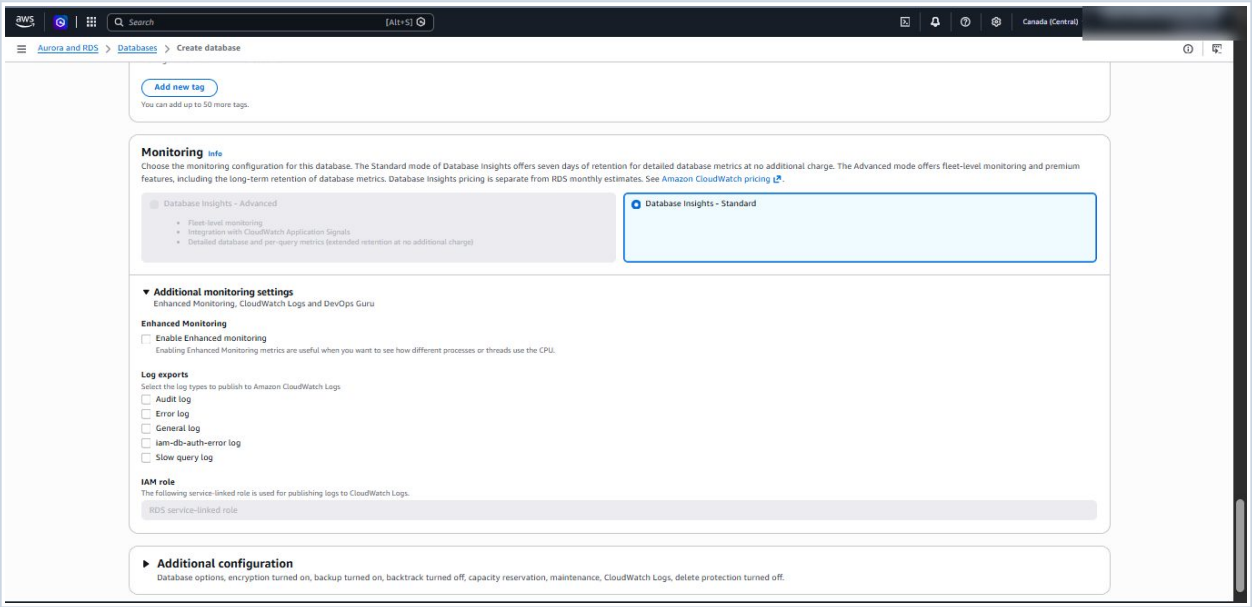


Figure 51. Monitoring and additional configuration. Review the settings, then click Create database.

WARNING

Never put a real database password in documentation, screenshots or Git. Use a long, unique password for each project and store it in a password manager.

NOTE

The free tier template creates a single database instance. For production use a Multi-AZ deployment; the DB subnet group already spans two zones, so that upgrade needs no network changes.

CHECKPOINT

The database status becomes **Available**. Open it and copy the **Endpoint** from Connectivity & security: you need it in Step 7.

STEP 05 OF 15

DATA AND ACCESS

Create the IAM Role

Hand the servers a key, not a password

WHY THIS STEP?

The servers must download the application code from S3. An IAM role gives them that permission through temporary credentials, so no access keys are ever stored on a machine.

1. Search for **IAM** and open **Roles > Create role**.
2. Trusted entity type **AWS service**, use case **EC2**.
3. Attach the managed policy **AmazonS3ReadOnlyAccess**.
4. Name the role IAM-ROLE-THREE-TIER-PROJECT and create it. The matching instance profile is created automatically.

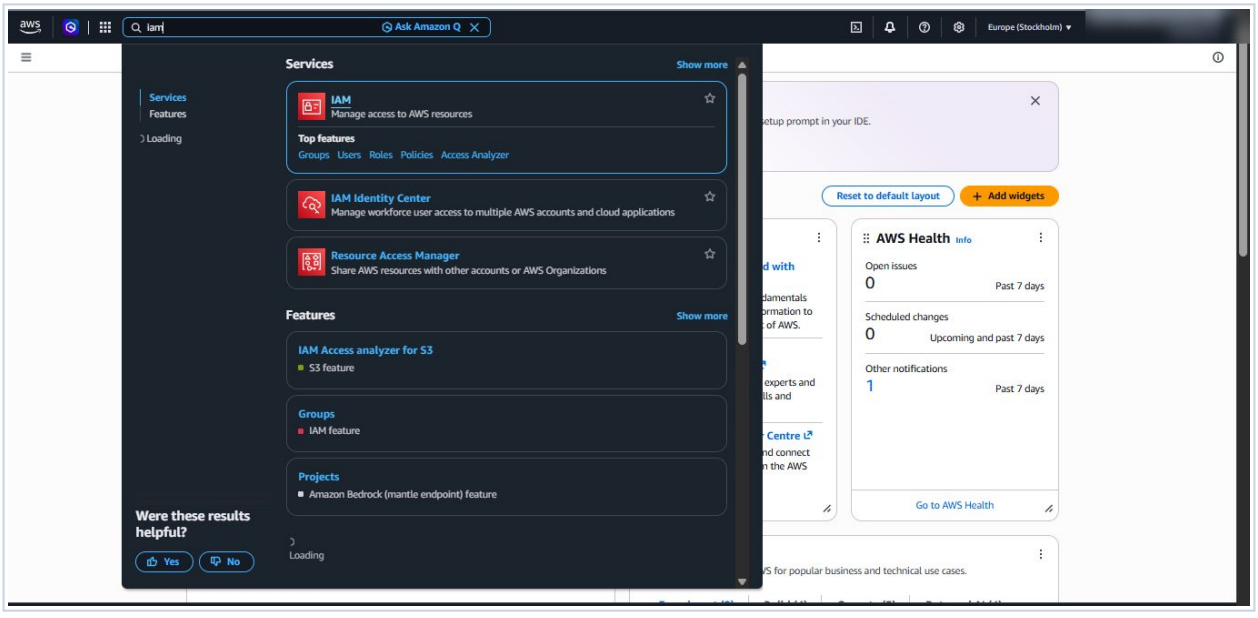


Figure 52. Search the console for IAM.

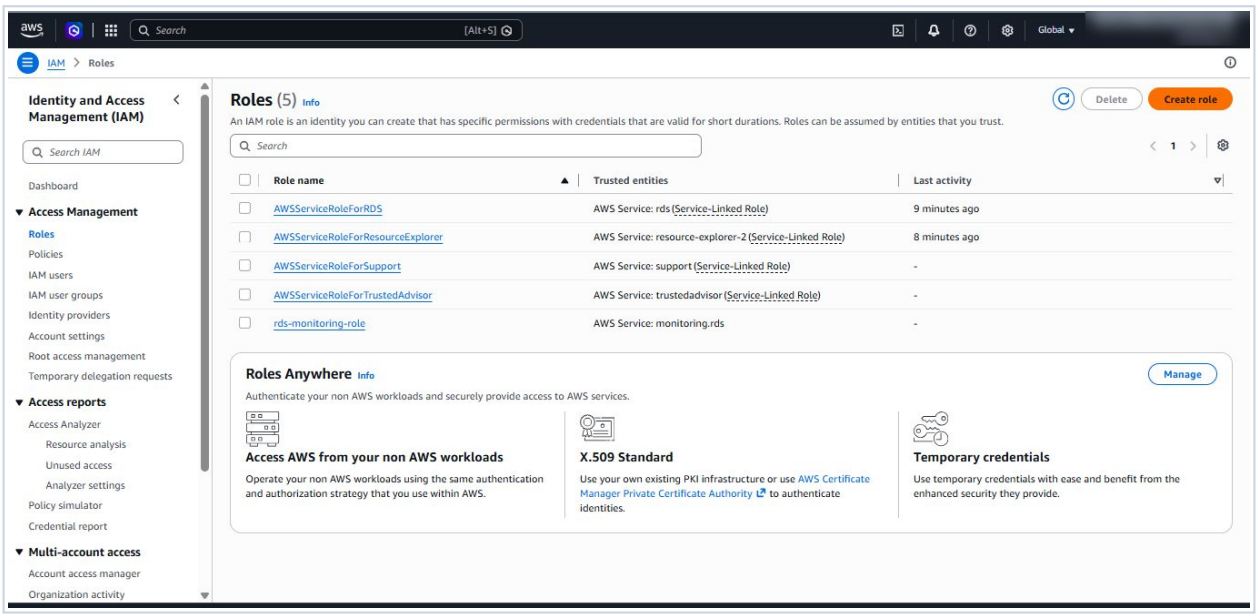


Figure 53. IAM > Roles: click Create role.

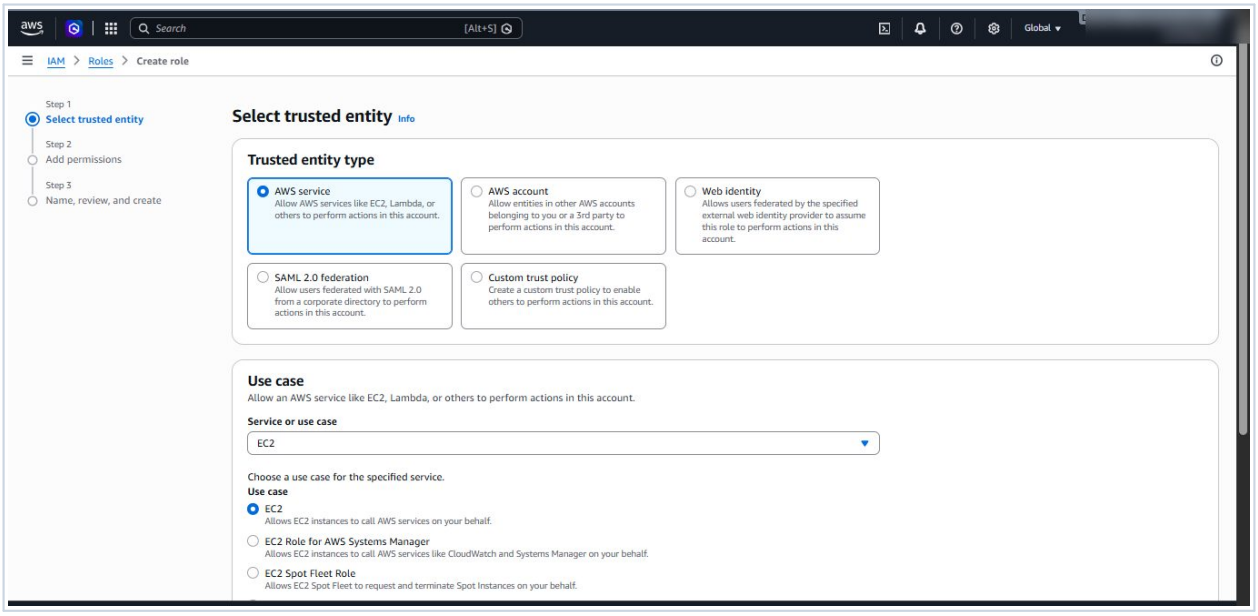


Figure 54. Select trusted entity: AWS service with the EC2 use case.

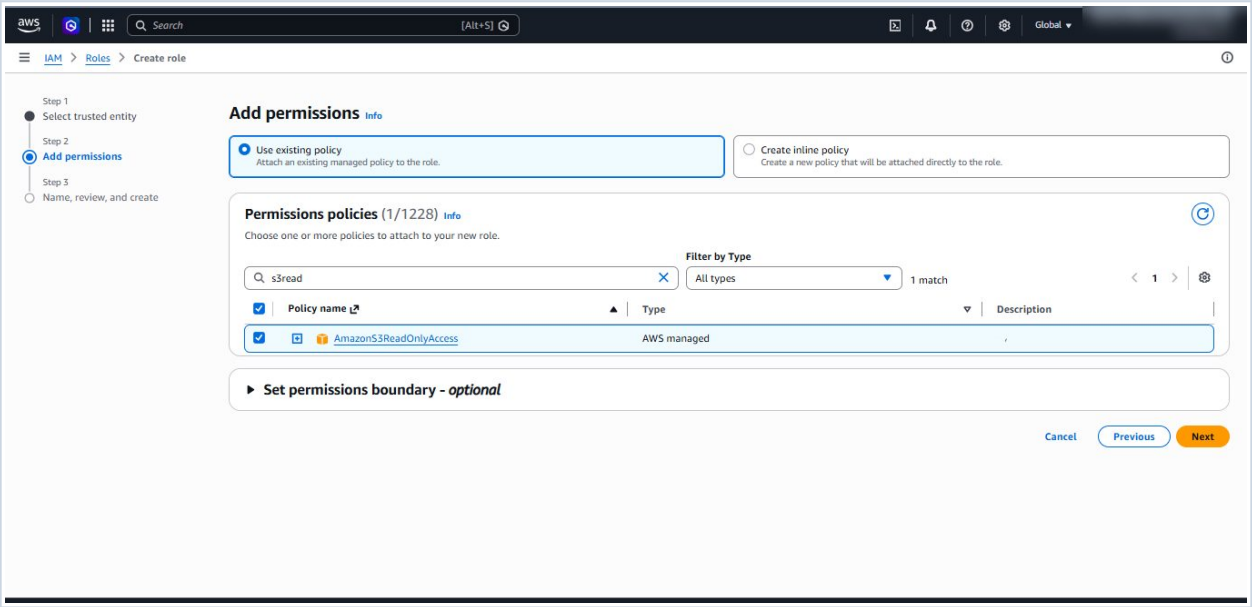


Figure 55. Add permissions: search for the S3 read-only policy and tick AmazonS3ReadOnlyAccess.

NOTE

Terminal access in this project comes from Systems Manager Session Manager (see Step 7), enabled once for the whole account and Region, so the role itself only needs the S3 read-only policy.

CHECKPOINT

The role IAM-ROLE-THREE-TIER-PROJECT exists and will appear in the **IAM instance profile** list when you launch servers.

STEP 06 OF 15 DATA AND ACCESS

Create an S3 Bucket and Upload the Source Code

One shelf for all the code

WHY THIS STEP?

Instead of copying files onto every server by hand, the code lives in one private bucket. Any server with the S3 read-only role can fetch it with one command.

Create the bucket

1. Search for **S3** and click **Create bucket**.
2. Bucket name: three-tier-project-28-26 (bucket names are global, so yours must be unique). Region: Canada (Central) ca-central-1.
3. Keep **ACLs disabled** and **Block all public access** ticked. Keep default encryption (SSE-S3).
4. Click **Create bucket**.

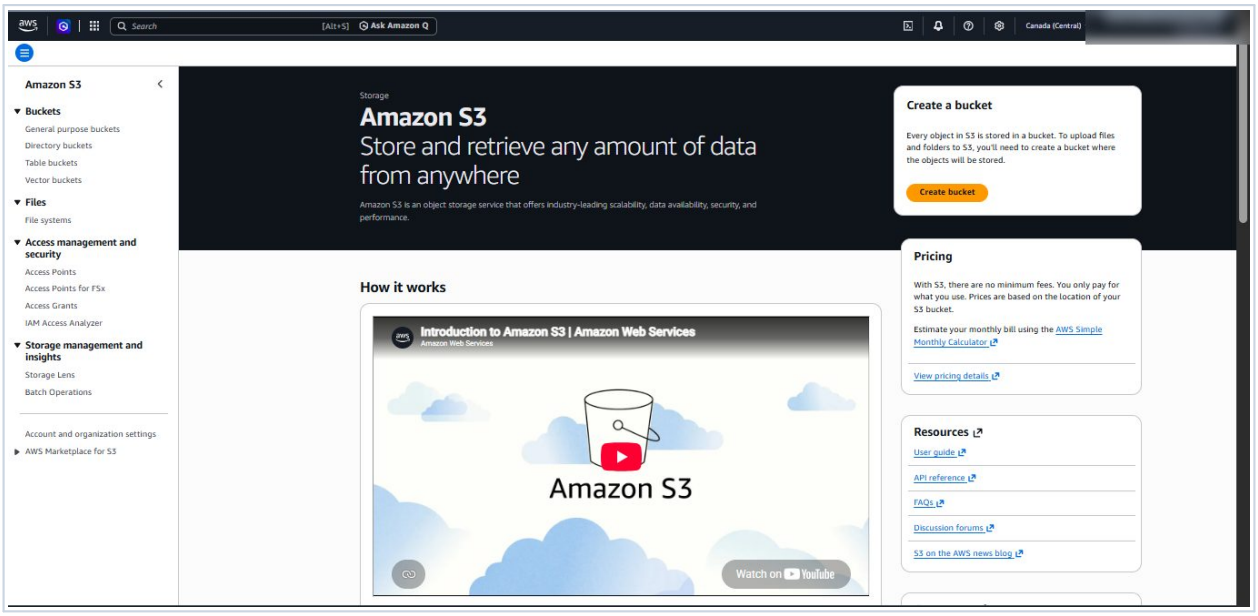


Figure 56. The Amazon S3 landing page: click Create a bucket.

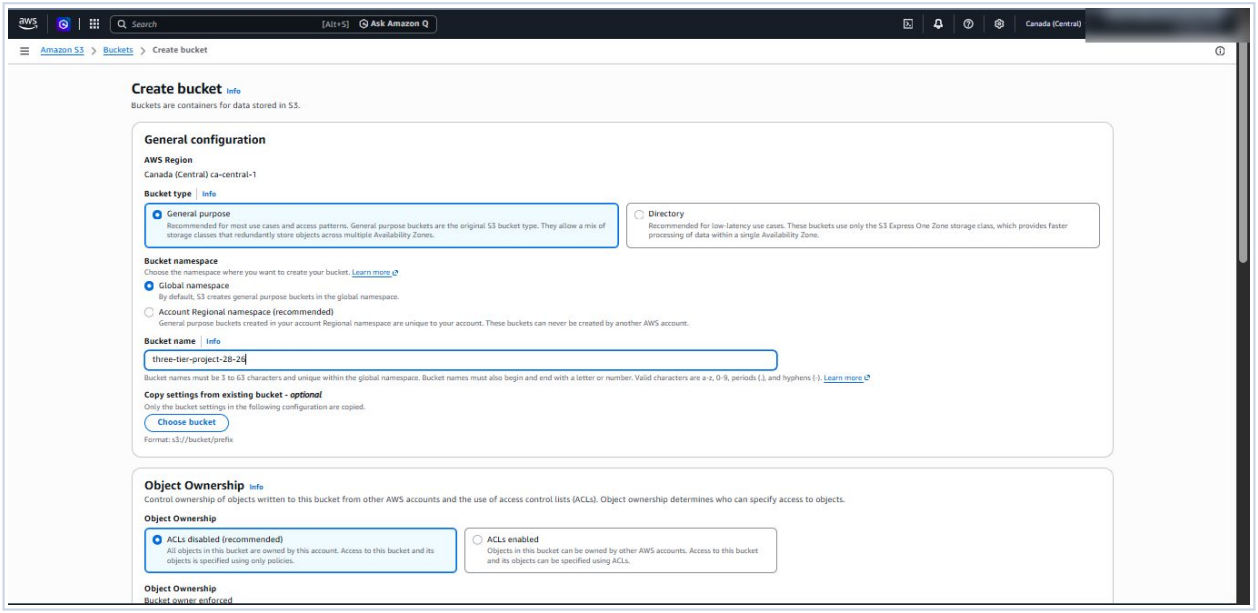


Figure 57. General configuration: bucket name three-tier-project-28-26 and object ownership.

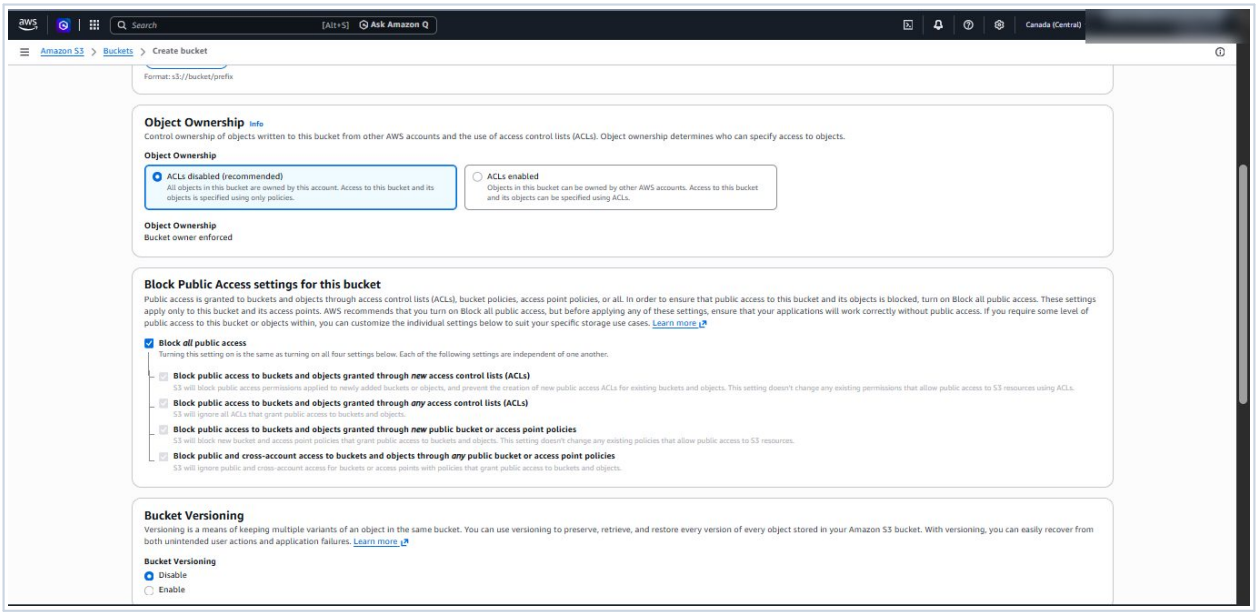


Figure 58. Object ownership with ACLs disabled, and Block all public access enabled.

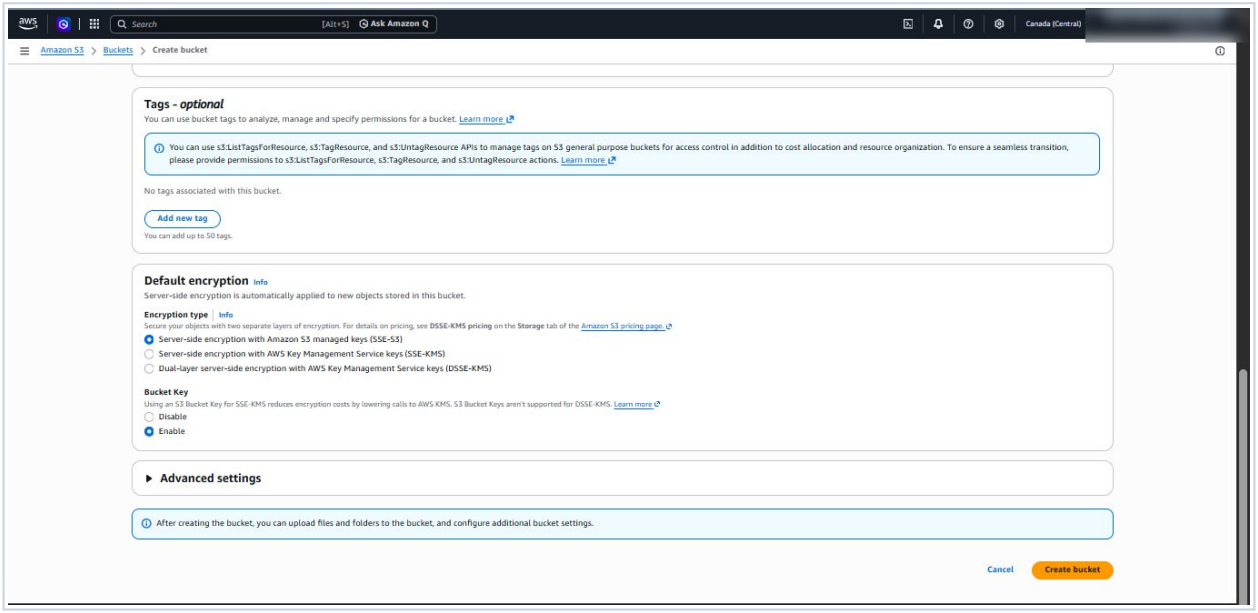


Figure 59. Tags and default encryption, then Create bucket.

Upload the application code

1. Open the new bucket and click **Upload**.
2. Click **Add folder** and select the **application-code** folder from the source code on your PC.
3. Click **Upload** and wait for the status page to show success.

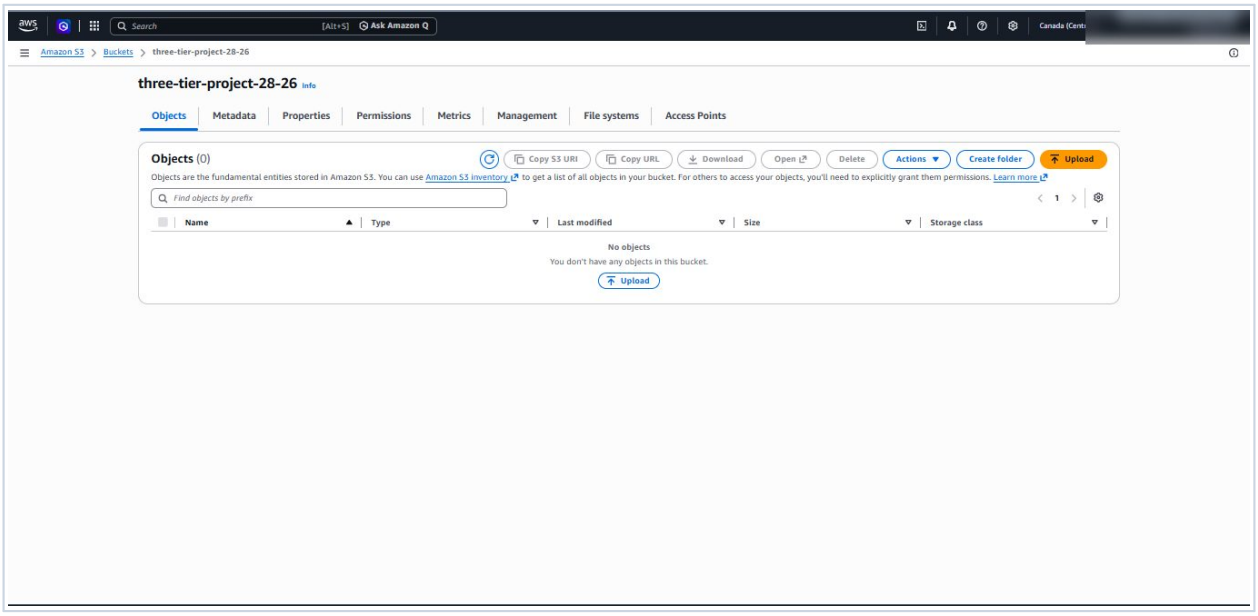


Figure 60. The empty bucket three-tier-project-28-26: click Upload.

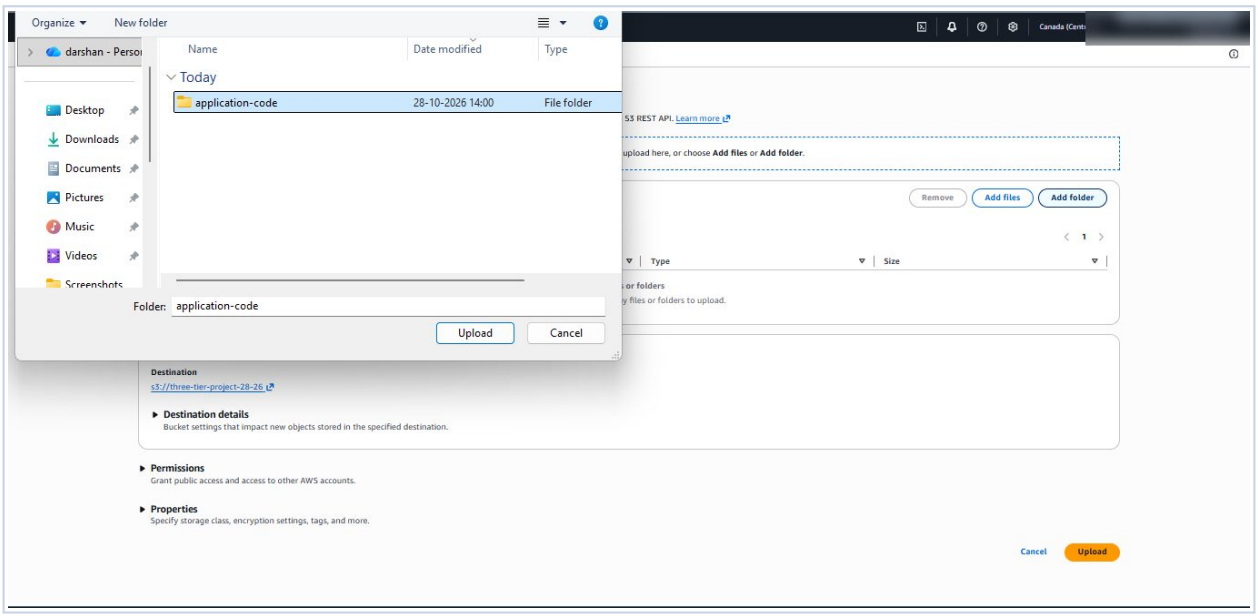


Figure 61. Add folder: choose the application-code folder from the PC.

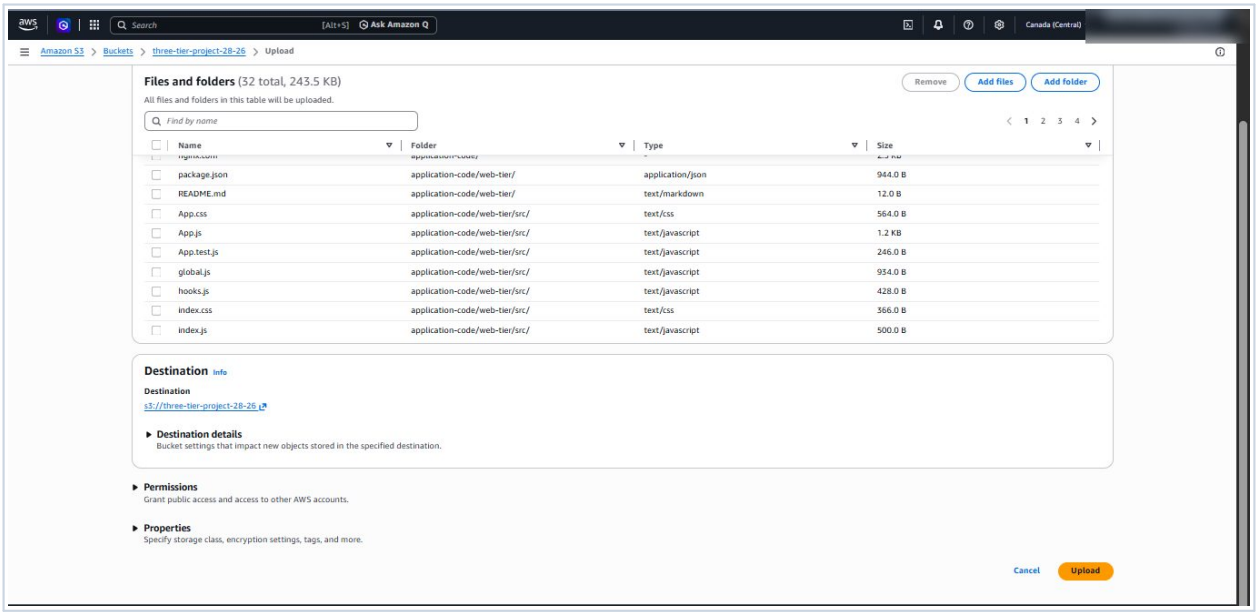


Figure 62. The file list: 52 files, 243.5 KB, ready to upload (destination is the new bucket).

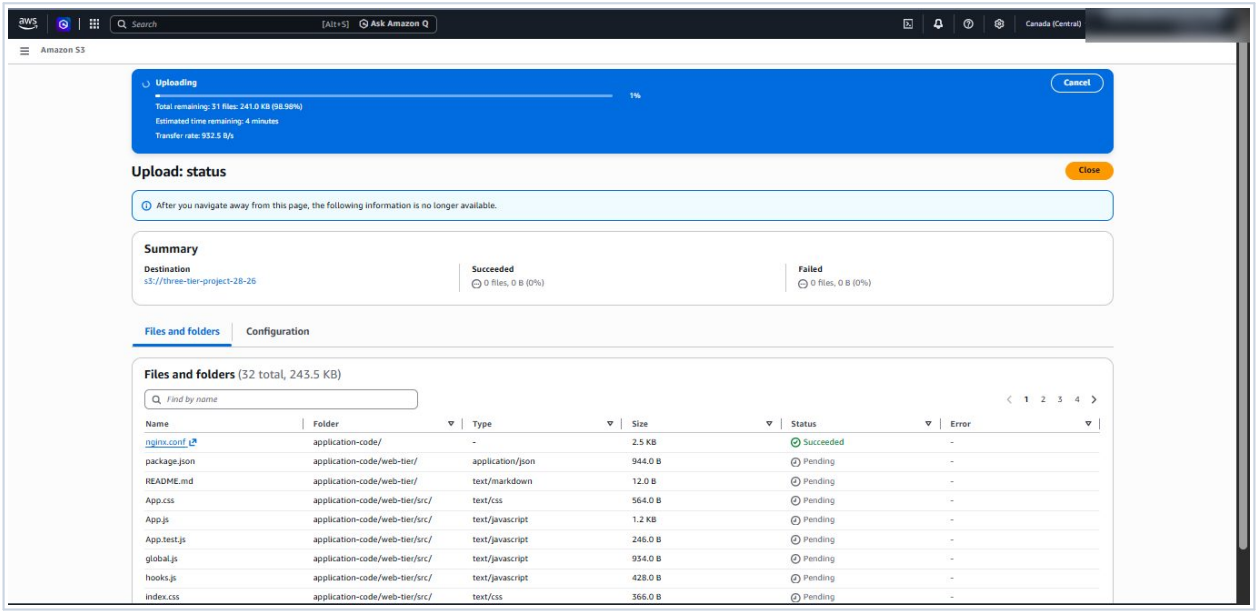


Figure 63. Upload in progress.

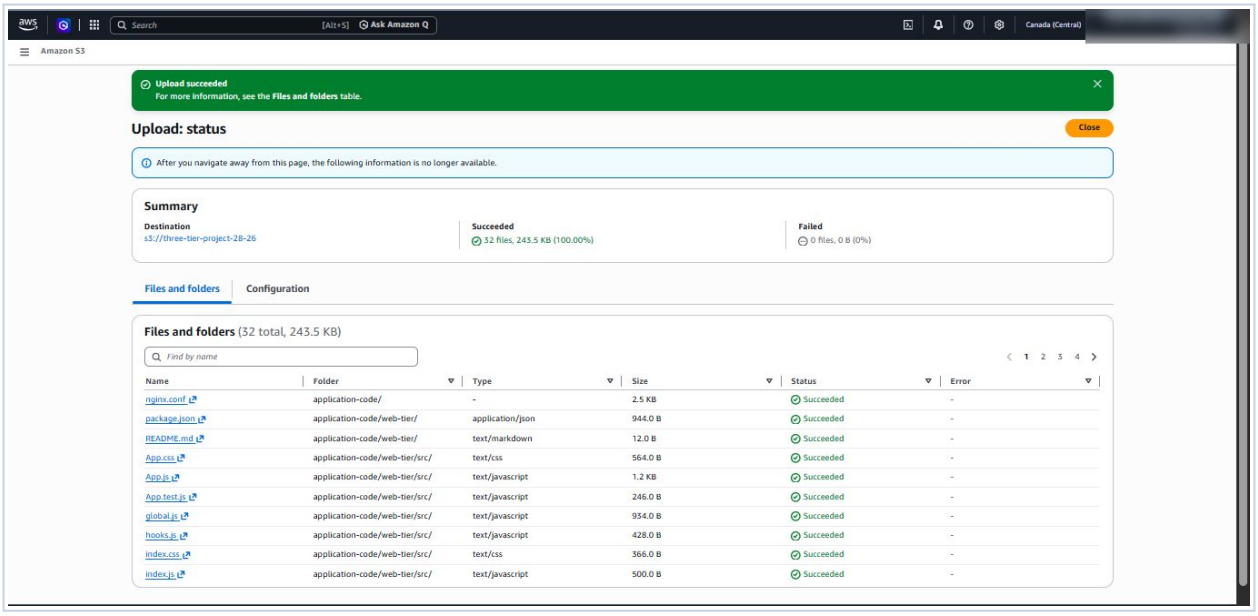


Figure 64. Upload succeeded: 52 files in application-code.

CHECKPOINT

The bucket holds **application-code** with the **web-tier** and **app-tier** folders plus nginx.conf. You will replace two of these files later, after you know the database endpoint and the internal load balancer address.

STEP 07 OF 15 SERVERS AND TRAFFIC

Create the Test App Server and Set It Up

Train the first app server

WHY THIS STEP?

Auto Scaling builds servers from launch templates, and a launch template needs an AMI. So one test server is configured by hand, proven to work, and saved as an AMI. The web tier repeats the same idea.

Launch the test app server

Search for **EC2**, open **Instances > Launch instances** and use these settings. Keep everything else at the default.

Setting	Value
Name	Test-App-Server
AMI	Amazon Linux 2023 AMI (64-bit x86)
Instance type	t3.micro (free tier eligible)
Key pair	Proceed without a key pair (access is via Session Manager)
VPC / Subnet	Three-Tier-Project-VPC / App-Tier-Subnet-1-Private (10.0.0.0/24)
Auto-assign public IP	Disable
Security group	Select existing: App-Tier-SG
IAM instance profile (Advanced details)	IAM-ROLE-THREE-TIER-PROJECT

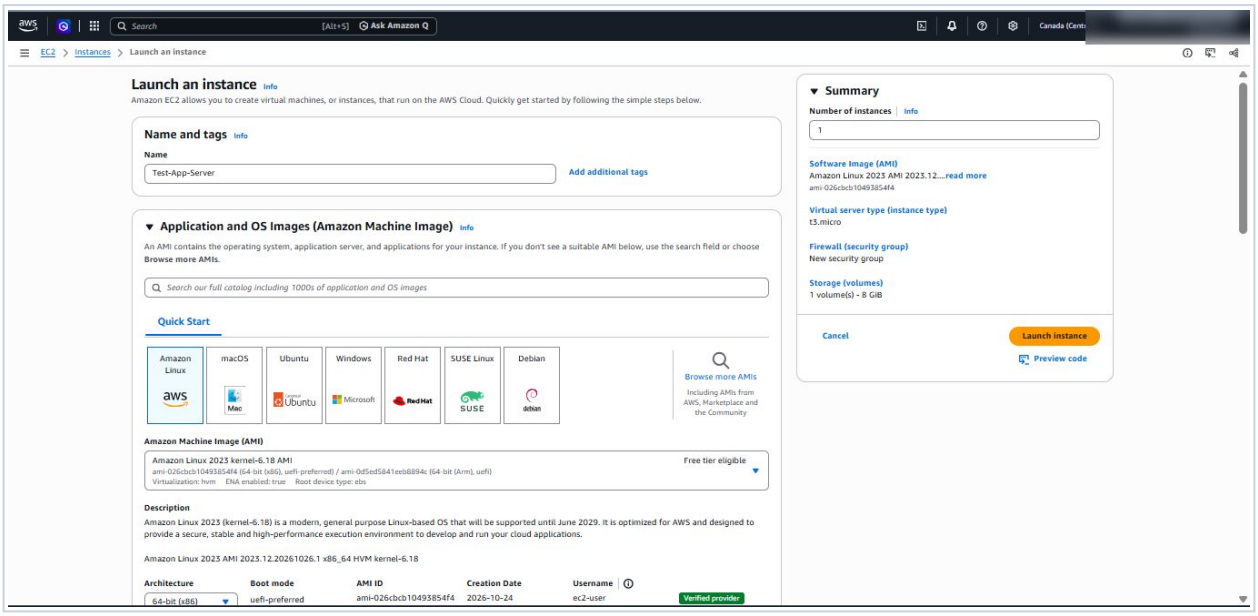


Figure 65. Launch an instance: name Test-App-Server and the Amazon Linux 2023 AMI.

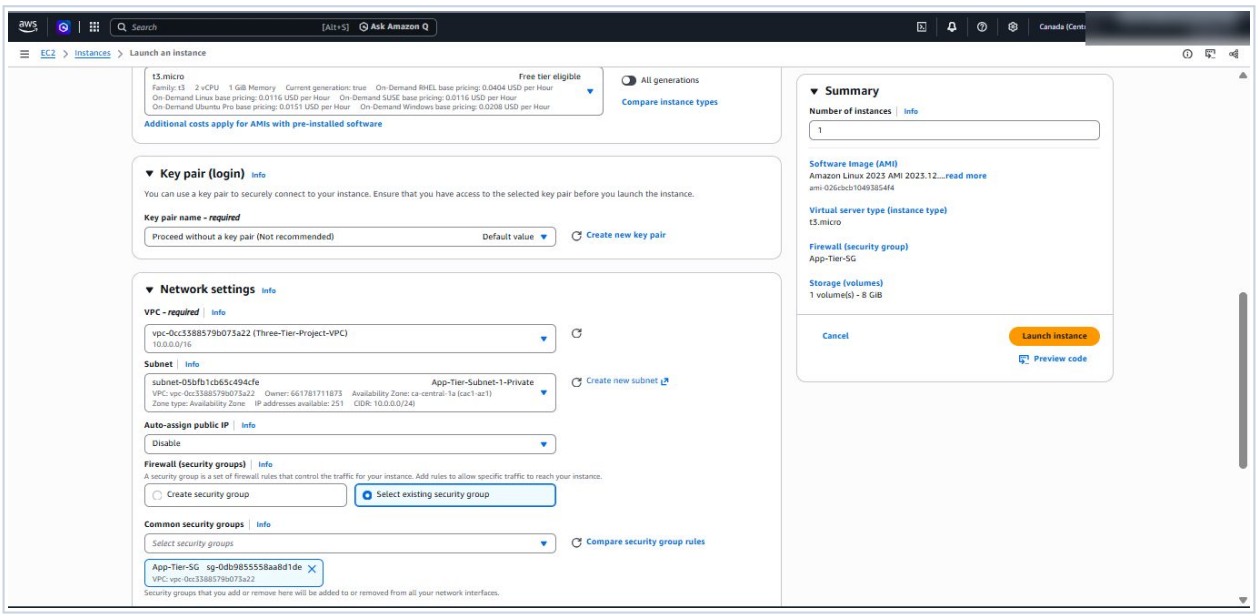


Figure 66. Network settings: project VPC, App-Tier-Subnet-1-Private, existing security group App-Tier-SG.

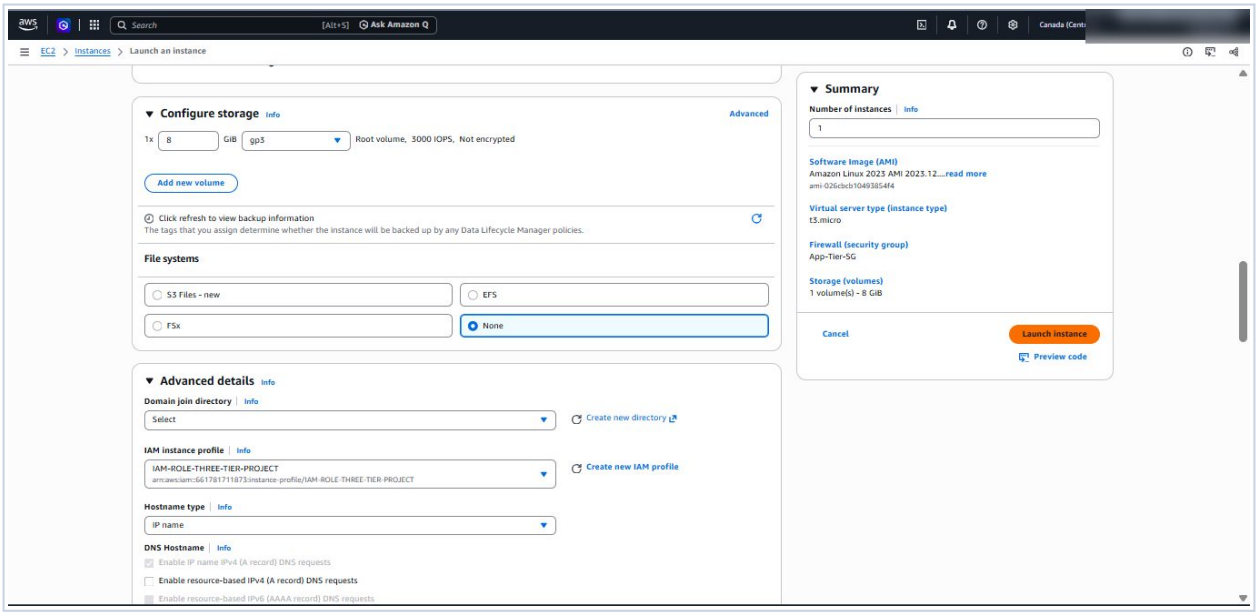


Figure 67. Storage and Advanced details: choose the IAM instance profile IAM-ROLE-THREE-TIER-PROJECT, then Launch instance.

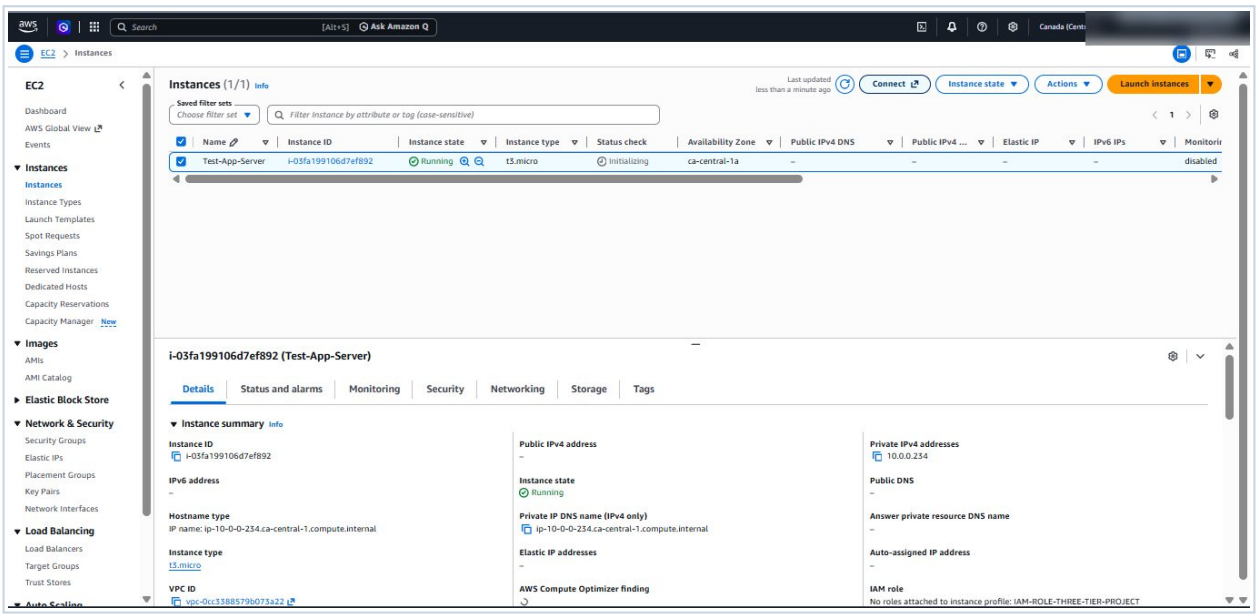


Figure 68. The instance is running in the EC2 Instances list.

Put your database details in the code

On your PC, open **application-code/app-tier/DbConfig.js** and change four values:

Setting	Value
DB_HOST	The RDS endpoint from Step 4 (Connectivity & security > Endpoint)
DB_USER	admin
DB_PWD	<YOUR_DB_PASSWORD> (the master password from Step 4)
DB_DATABASE	webappdb

Then open the S3 bucket, go to **application-code/app-tier**, delete the old DbConfig.js and upload the edited file.

```
module.exports = Object.freeze({
    DB_HOST : "",
    DB_USER : "",
    DB_PWD : "",
    DB_DATABASE : "-1",
});
```

DbConfig.js in application-code/app-tier.

Open a terminal with Session Manager

The test server is in a private subnet with no public IP and no key pair, so the terminal comes from **Systems Manager Session Manager**. Select the instance, click **Connect** and open the **Session Manager** tab. The instance must show up as a managed node. In my account it did not at first, so I did the following:

1. Open **Systems Manager** and choose **Enable Systems Manager** for the account and Region.
2. Under Settings, turn on **Default Host Management Configuration**, which lets every instance in the Region register without an instance profile for SSM.
3. Open **Fleet Manager** and wait until the instance is listed as a managed node. This also needs the instance to reach the Systems Manager service endpoints, which is why the NAT gateways from Step 2 matter.

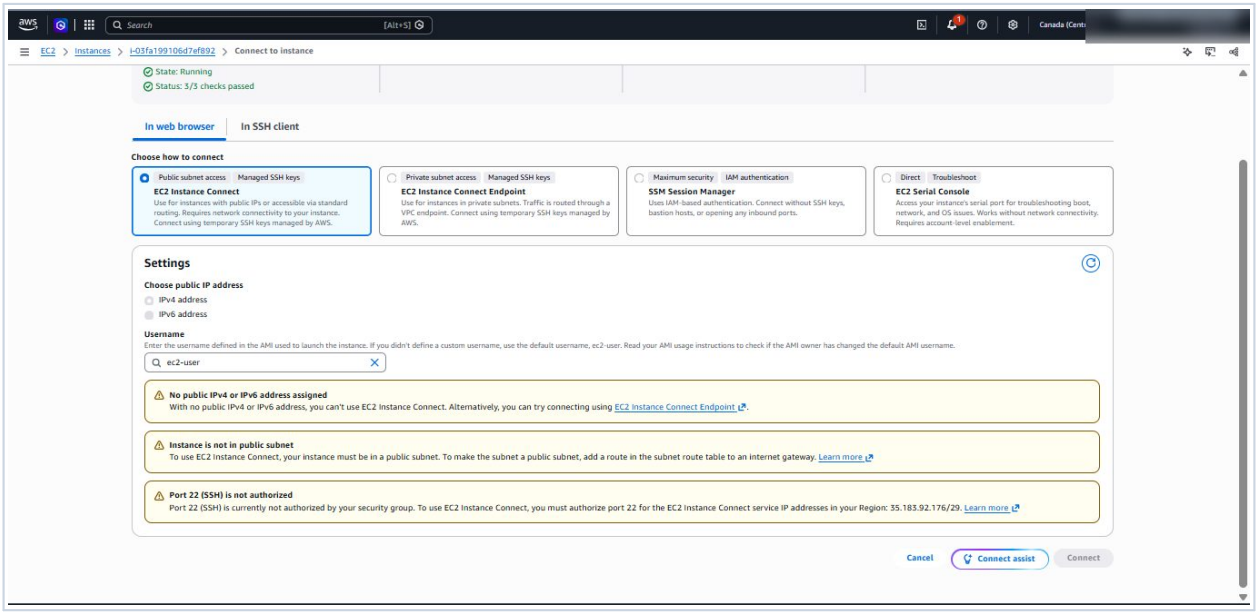


Figure 69. EC2 > Connect > Session Manager tab for the test app server.

PRO TIP

Paste commands one block at a time and wait for each to finish. If something fails, fix it before moving on; errors compound.

Configure the app tier

1. Switch user and install the MySQL client

BASH

```
sudo -su ec2-user
sudo wget https://dev.mysql.com/get/mysql80-community-release-el9-1.noarch.rpm
sudo dnf install mysql80-community-release-el9-1.noarch.rpm -y
sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2023
sudo dnf install mysql-community-client -y
mysql --version
```

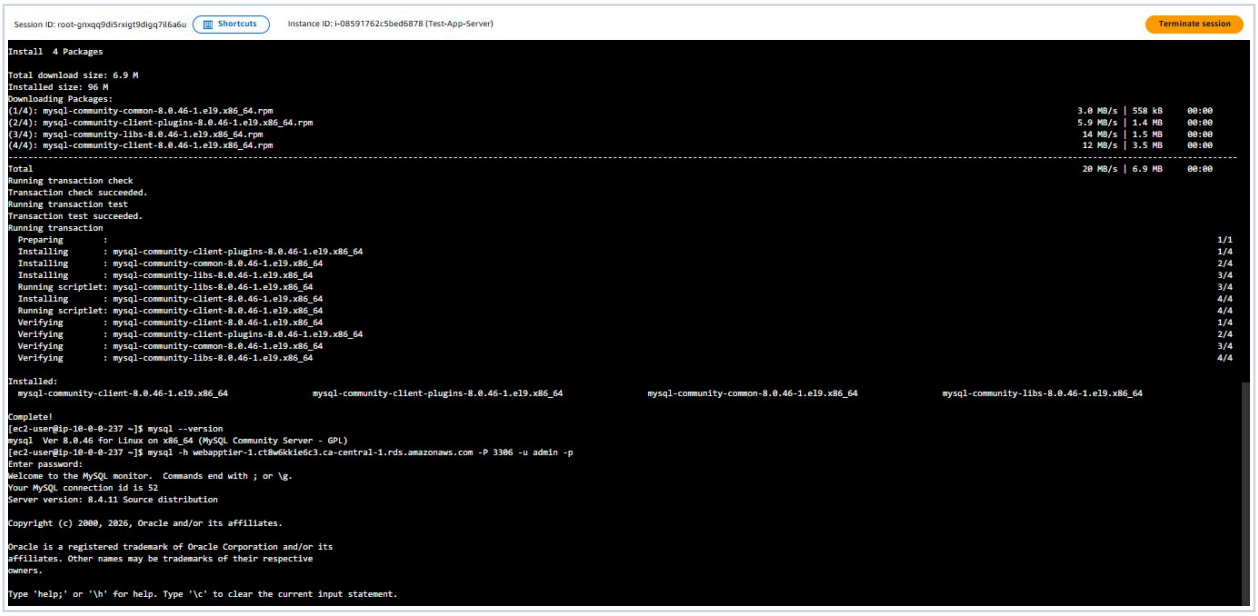


Figure 71. The MySQL client packages installed successfully; mysql --version reports the client.

2. Connect to RDS and create the schema

Use the RDS endpoint and the user **admin**. Type the password when prompted.

```bash
mysql -h <RDS-ENDPOINT> -P 3306 -u admin -p
```

```sql
CREATE DATABASE webappdb;
SHOW DATABASES;
USE webappdb;

CREATE TABLE IF NOT EXISTS transactions(
  id INT NOT NULL AUTO_INCREMENT,
  amount DECIMAL(10,2),
  description VARCHAR(100),
  PRIMARY KEY(id)
);
SHOW TABLES;
INSERT INTO transactions (amount, description) VALUES (400, 'groceries');
SELECT * FROM transactions;
exit
```

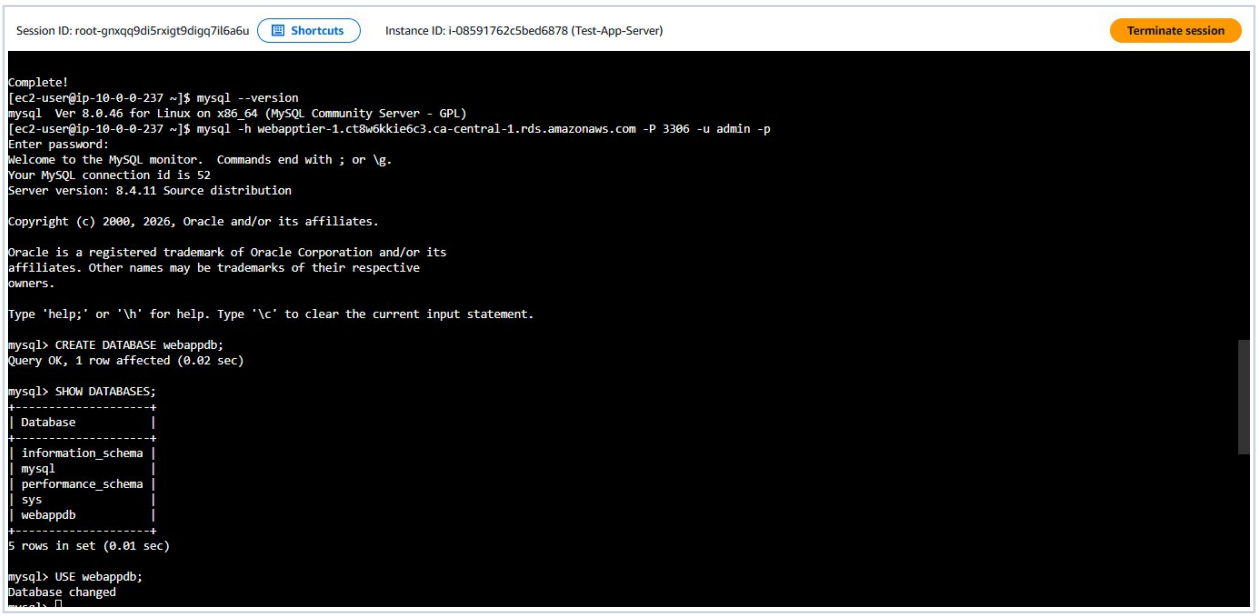


Figure 72. Connected to RDS (server 8.4.11): CREATE DATABASE webappdb and SHOW DATABASES.

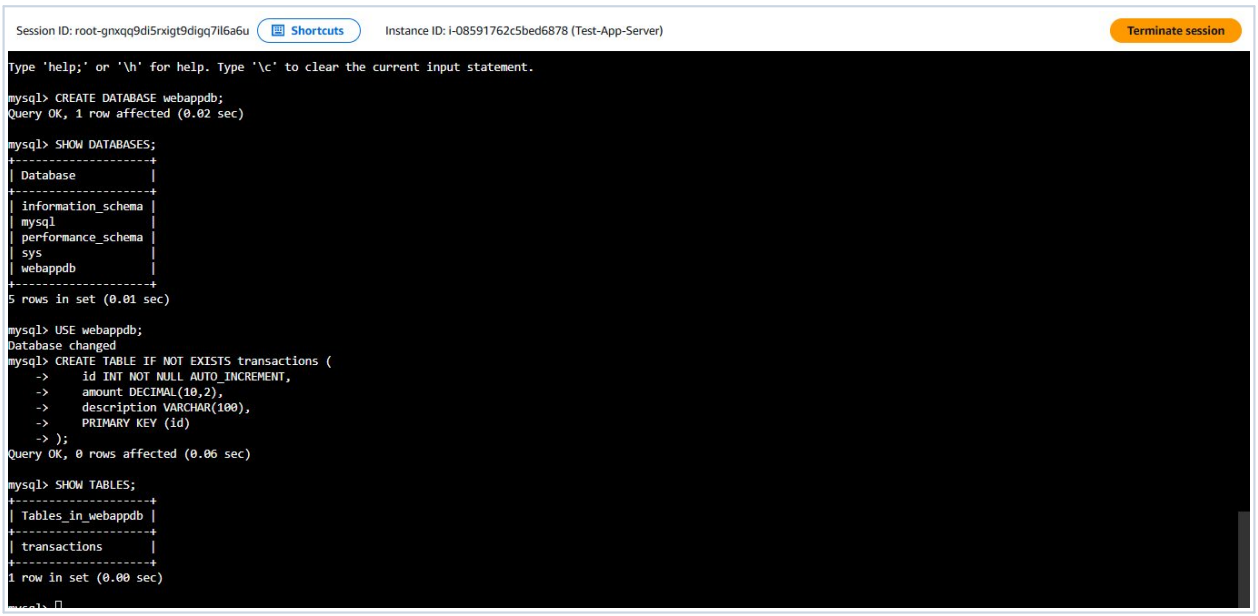


Figure 73. USE webappdb and the CREATE TABLE statement for transactions.

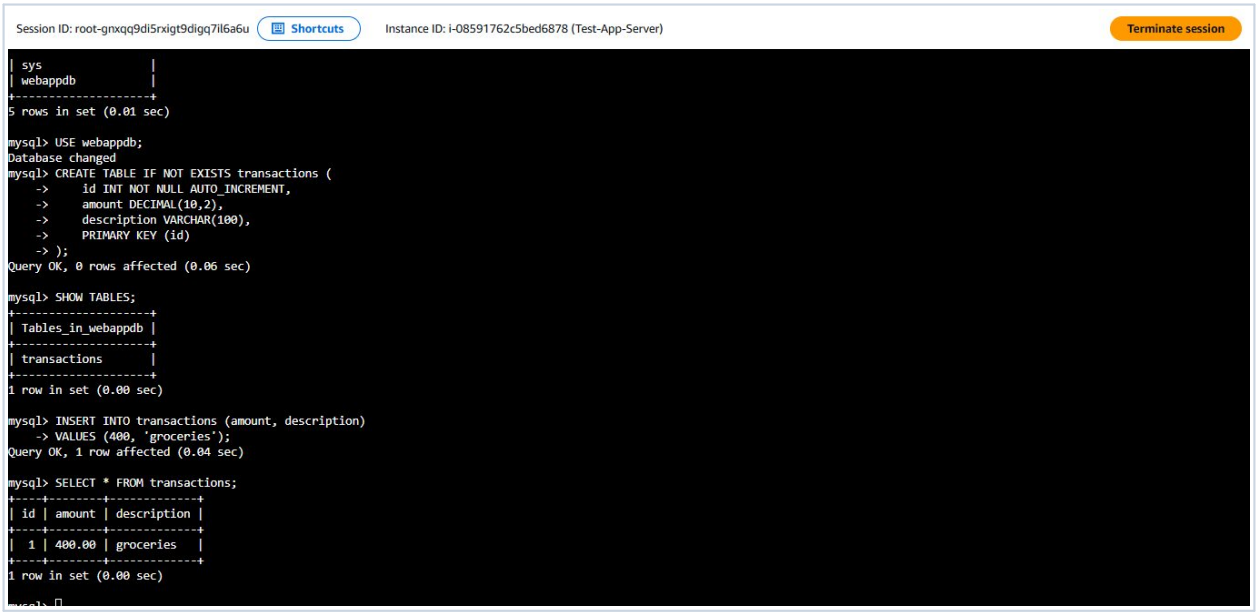


Figure 74. The transactions table is created; a test row is inserted and read back.

3. Download the app code from S3

```bash
cd /home/ec2-user
sudo aws s3 cp s3://three-tier-project-28-26/application-code/app-tier/ /home/ec2-user/app-tier
    --recursive
sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
sudo chmod -R 755 /home/ec2-user/app-tier
```

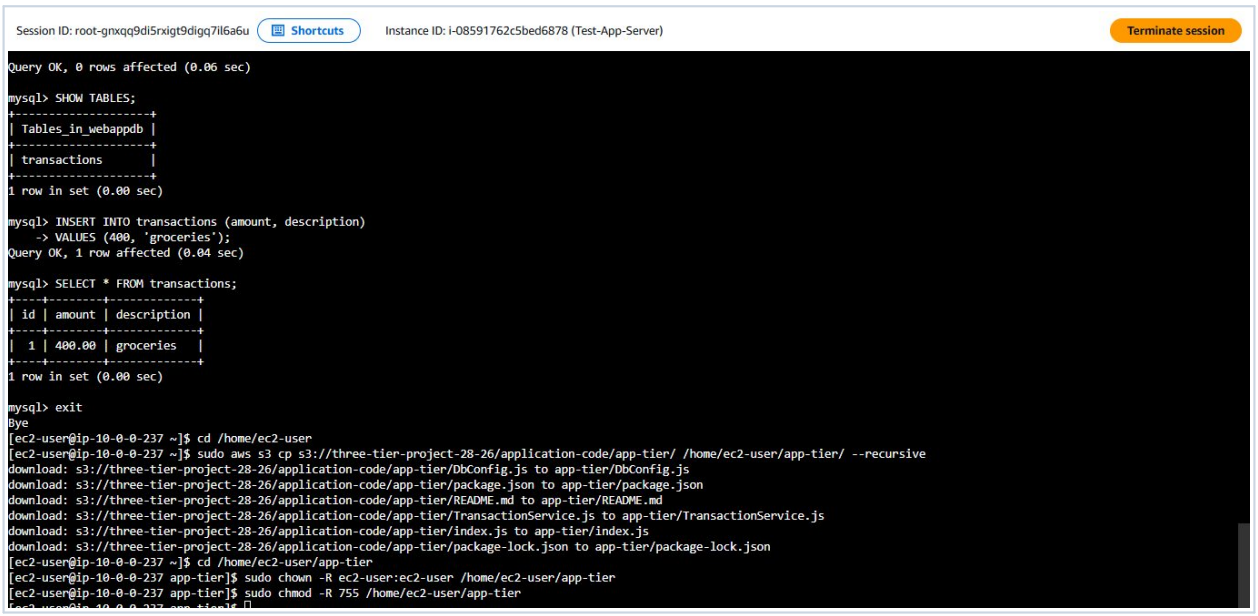


Figure 75. The SELECT shows the test row; after exit, the app-tier folder is copied from S3 and its permissions fixed.

4. Install Node.js and PM2, then start the app

```bash
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
nvm use 16
npm install -g pm2

cd /home/ec2-user/app-tier
npm install
pm2 start index.js
pm2 list
pm2 logs
```

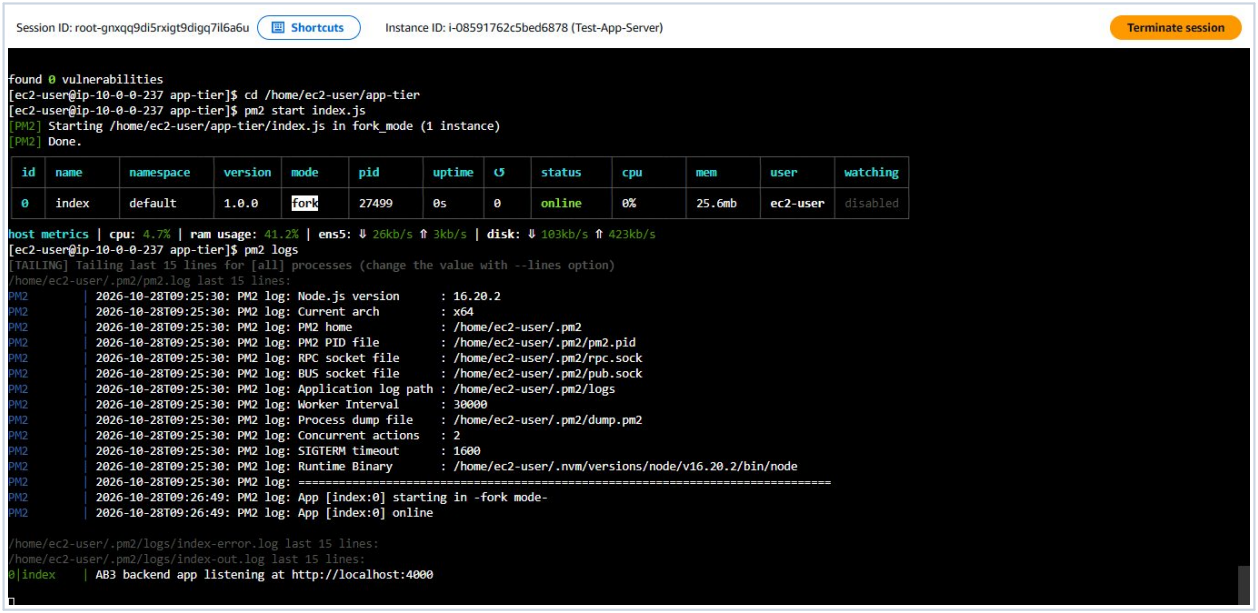


Figure 76. npm install finishes with 0 vulnerabilities; pm2 start index.js runs the app and pm2 logs follows it.

5. Keep it running after a reboot and test it

```bash
pm2 startup
# run the "sudo env PATH=..." command that pm2 prints
pm2 save
pm2 status
curl http://localhost:4000/health
```

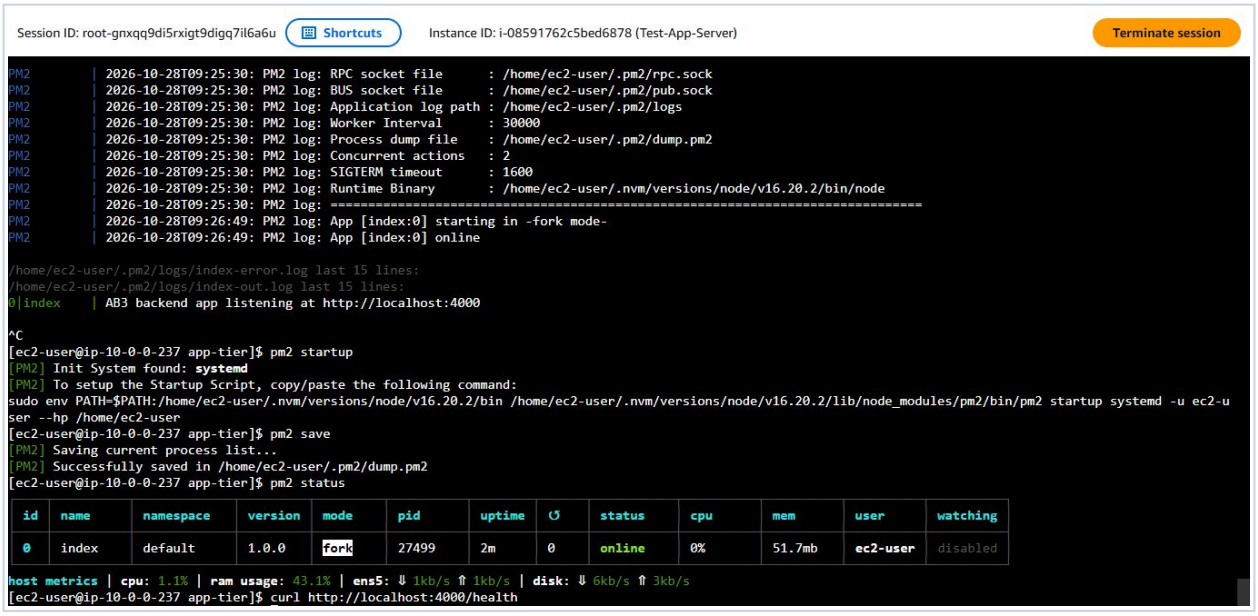


Figure 77. pm2 startup and pm2 save succeed, pm2 status shows the app online, then curl tests the /health route.

CHECKPOINT

The app answers on **http://localhost:4000/health**, the transactions table exists in webappdb, and PM2 shows the process online and saved. The app tier works.

STEP 08 OF 15 SERVERS AND TRAFFIC

Create an AMI of the App Server

Photograph the finished server

WHY THIS STEP?

The AMI freezes the configured app server, with its code, Node.js and PM2 startup, so Auto Scaling can stamp out identical copies on demand.

1. In EC2, select **Test-App-Server**.
2. Choose **Actions > Image and templates > Create image**.
3. Name the image App-Server-Image and click **Create image**.
4. Open **Images > AMIs** and wait until the status changes from **Pending** to **Available**.

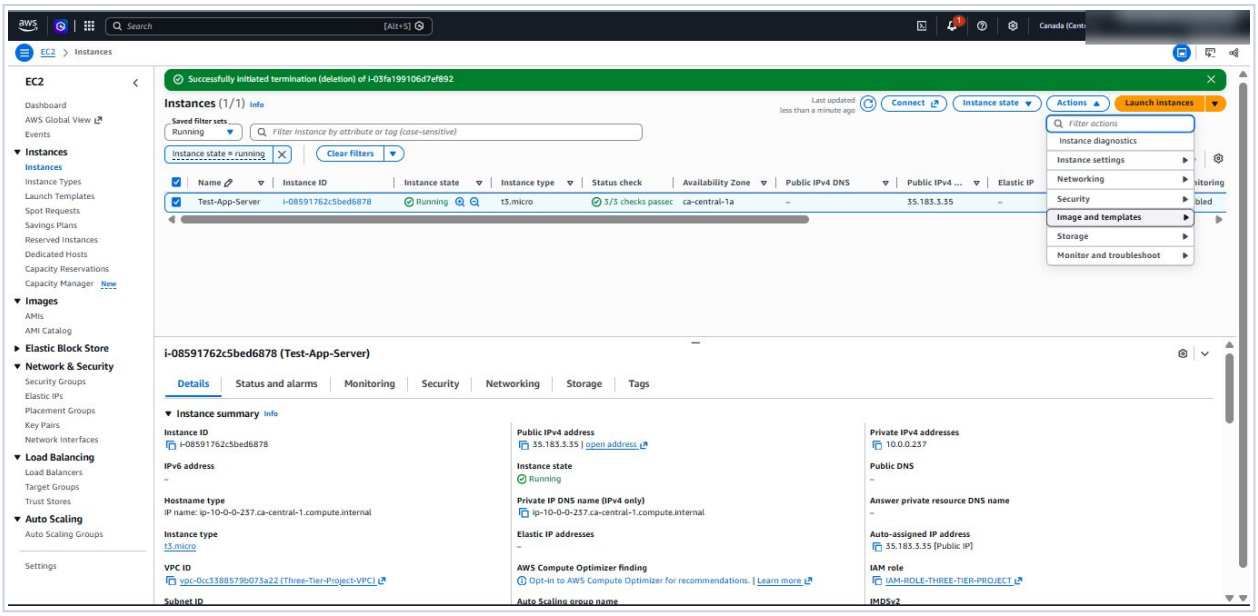


Figure 78. Select Test-App-Server, then Actions > Image and templates > Create image.

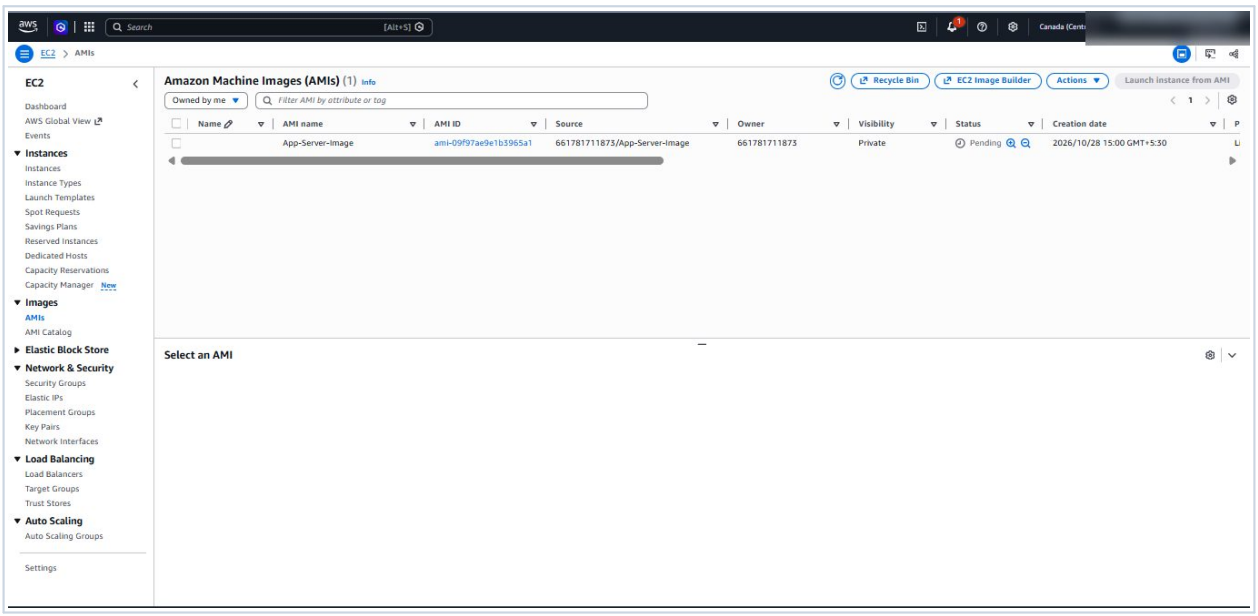


Figure 79. Images > AMIs: App-Server-Image is created and still pending.

CHECKPOINT

App-Server-Image shows **Available**. You can now keep or terminate Test-App-Server; the image is all Auto Scaling needs.

STEP 09 OF 15 SERVERS AND TRAFFIC

Target Group, Launch Template and Internal Load Balancer (App Tier)

Hire a traffic controller for the app tier

A. Create the target group

Go to **EC2 > Target Groups > Create target group** and change only these settings:

Setting	Value
Target type	Instances
Target group name	App-Server-TG
Protocol : Port	HTTP : 4000
VPC	Three-Tier-Project-VPC
Health check path	/health

Click **Next** and then **Create target group**. No targets are added by hand: the Auto Scaling group registers its own instances later.

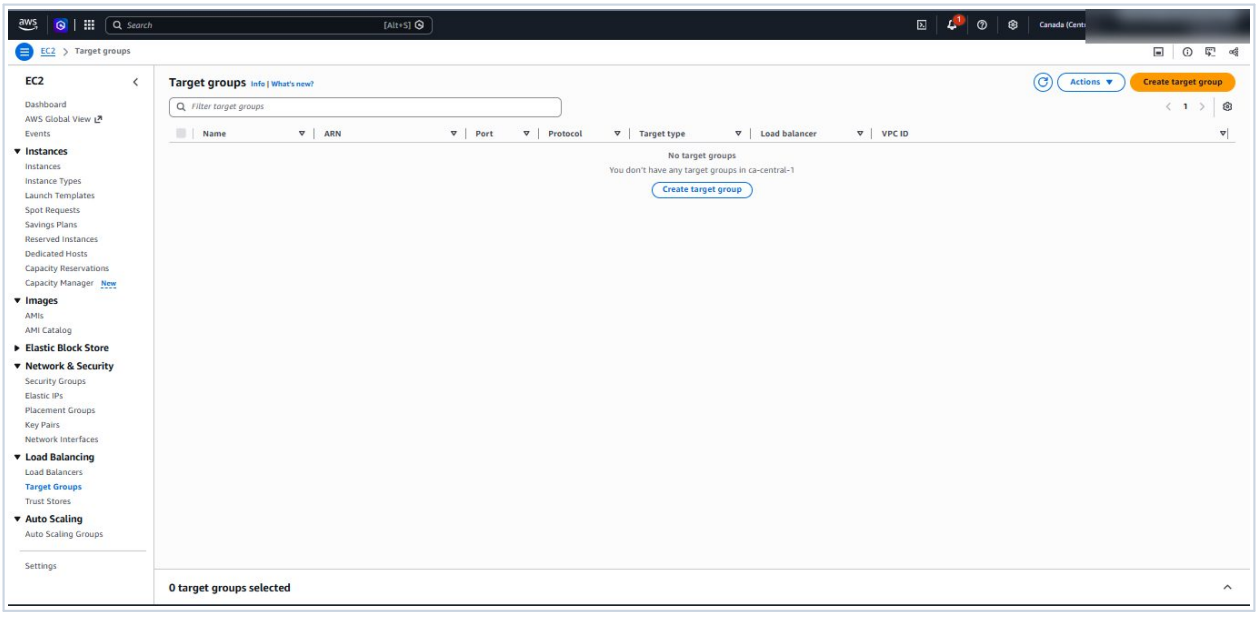


Figure 80. Target groups starts empty: click Create target group.

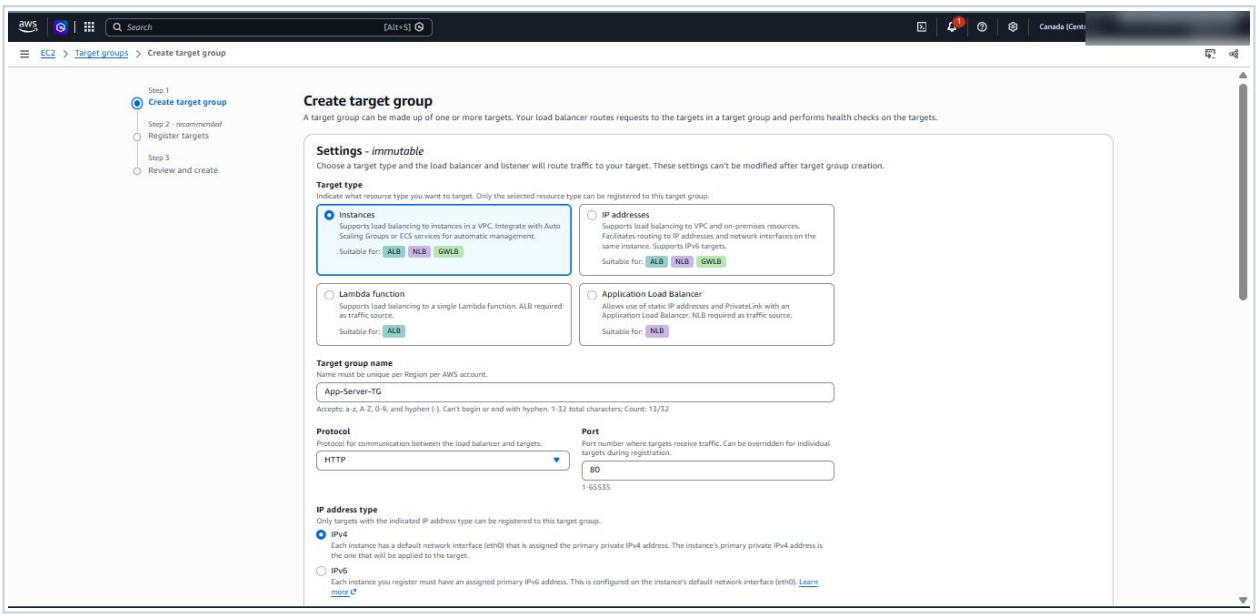


Figure 81. Choose Instances as the target type and name the group App-Server-TG (HTTP, port 4000).

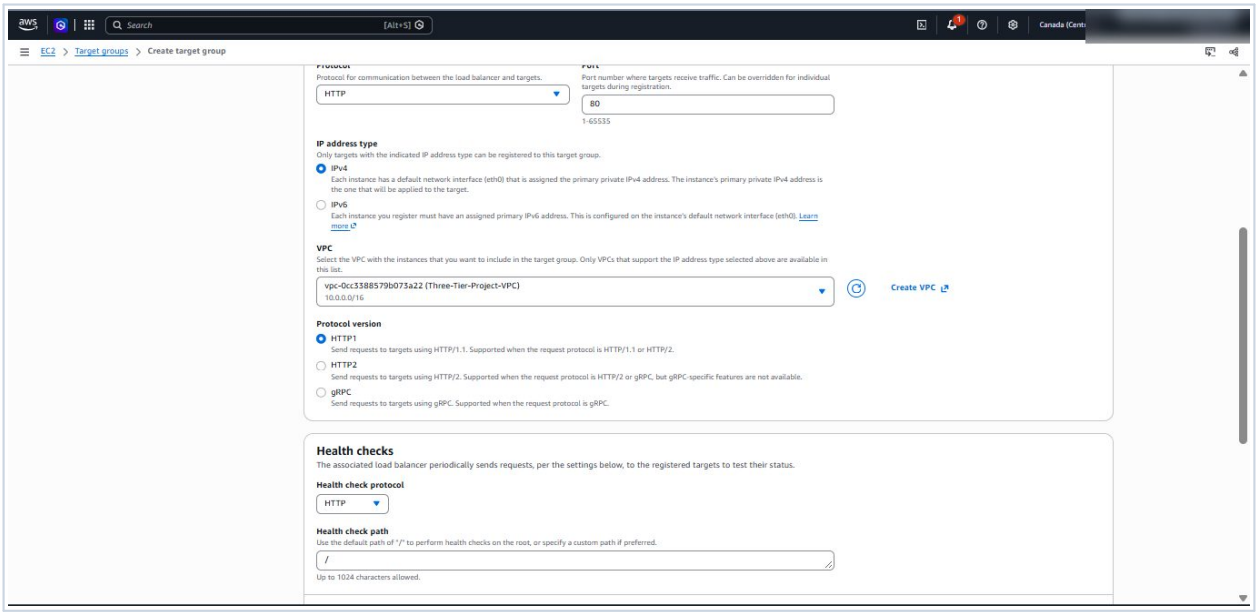


Figure 82. Choose Three-Tier-Project-VPC and set the health check path to /health.

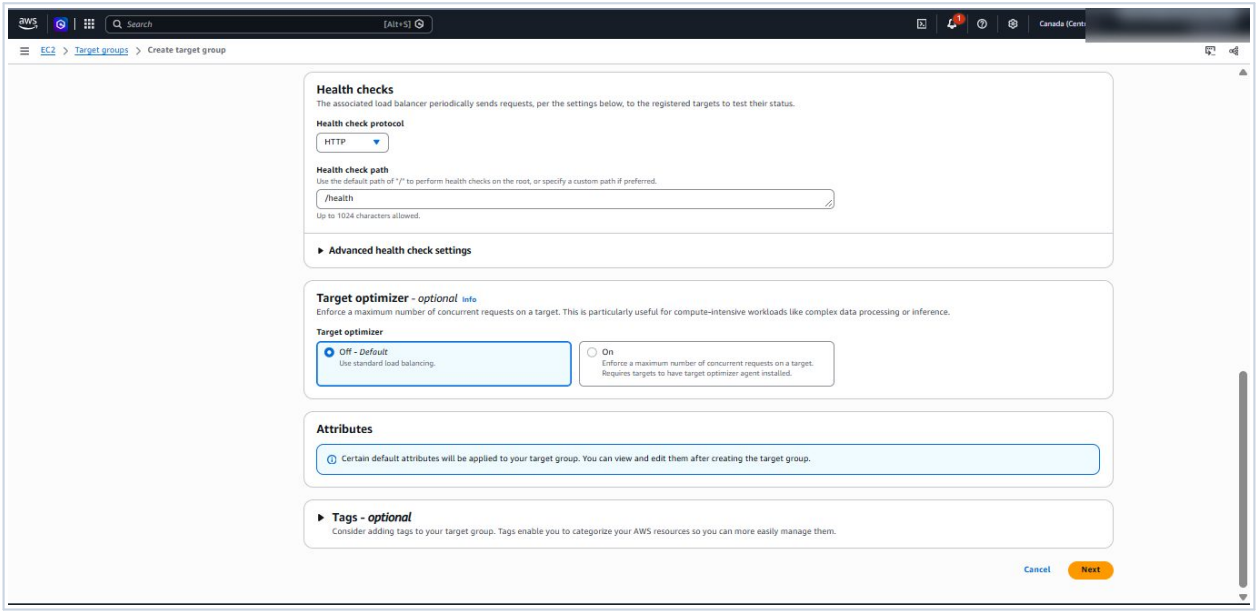


Figure 83. Health checks and the target optimizer section (defaults).

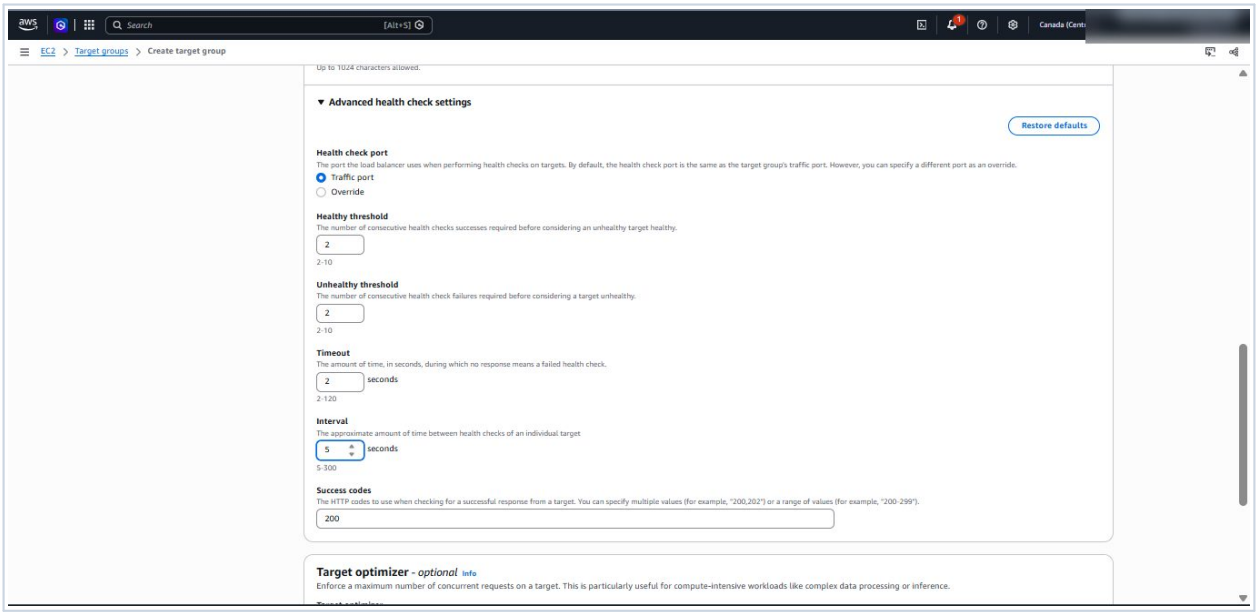


Figure 84. Advanced health check settings: thresholds, timeout and interval.

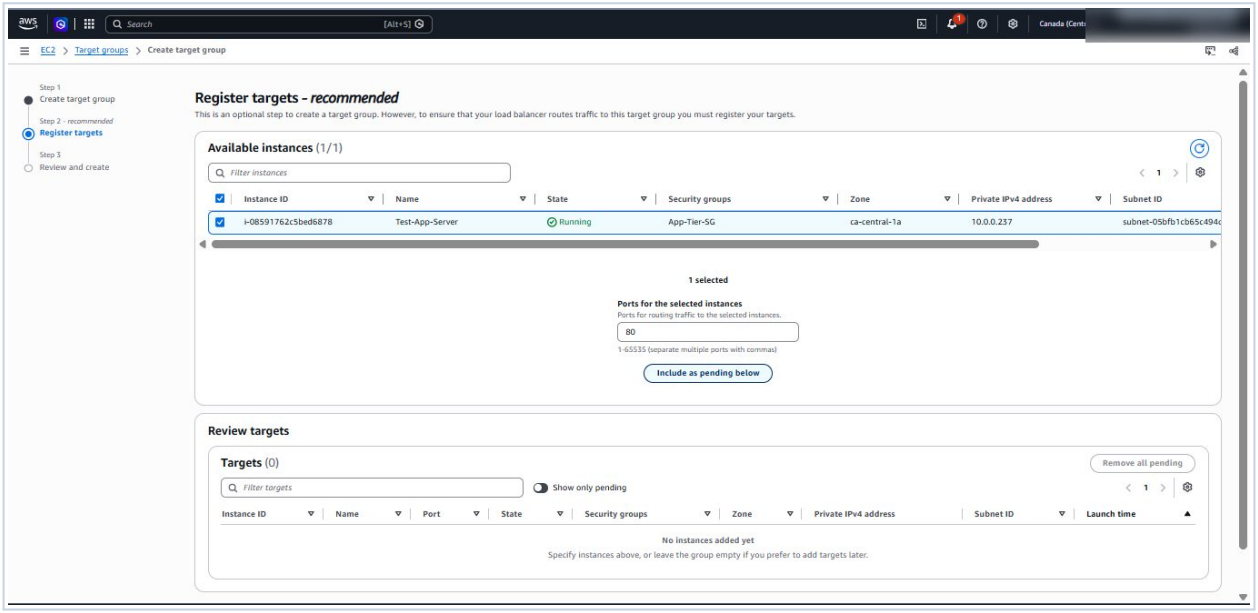


Figure 85. Register targets lists the test server. The port for the selected instances is 4000.

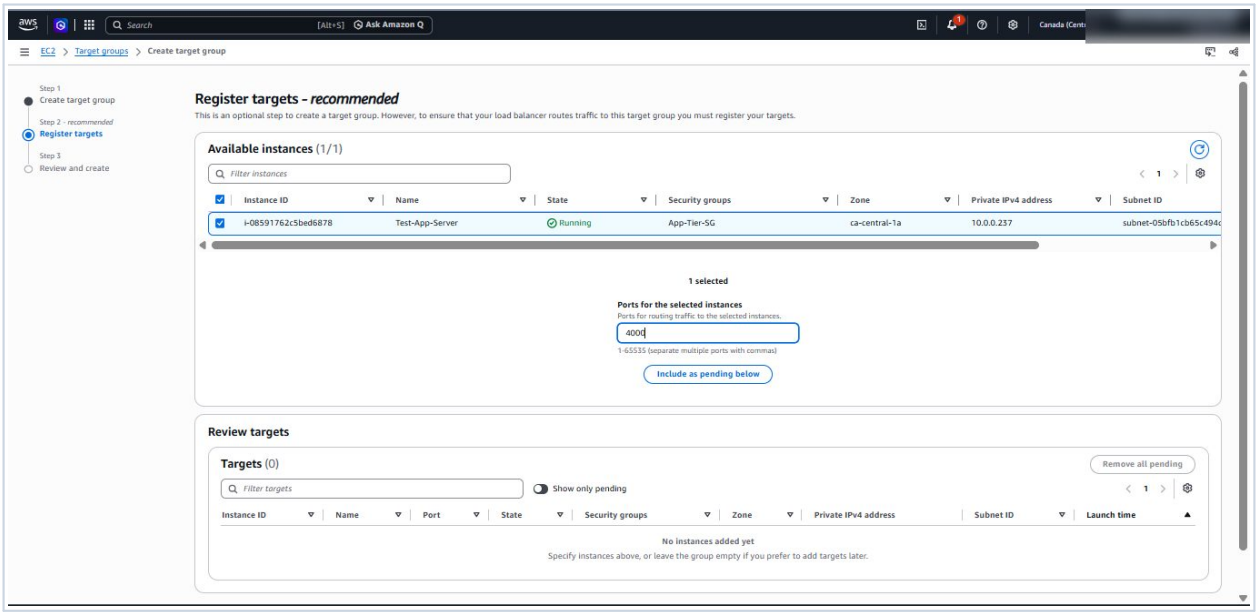


Figure 86. Editing the port field. No instance is added to the group.

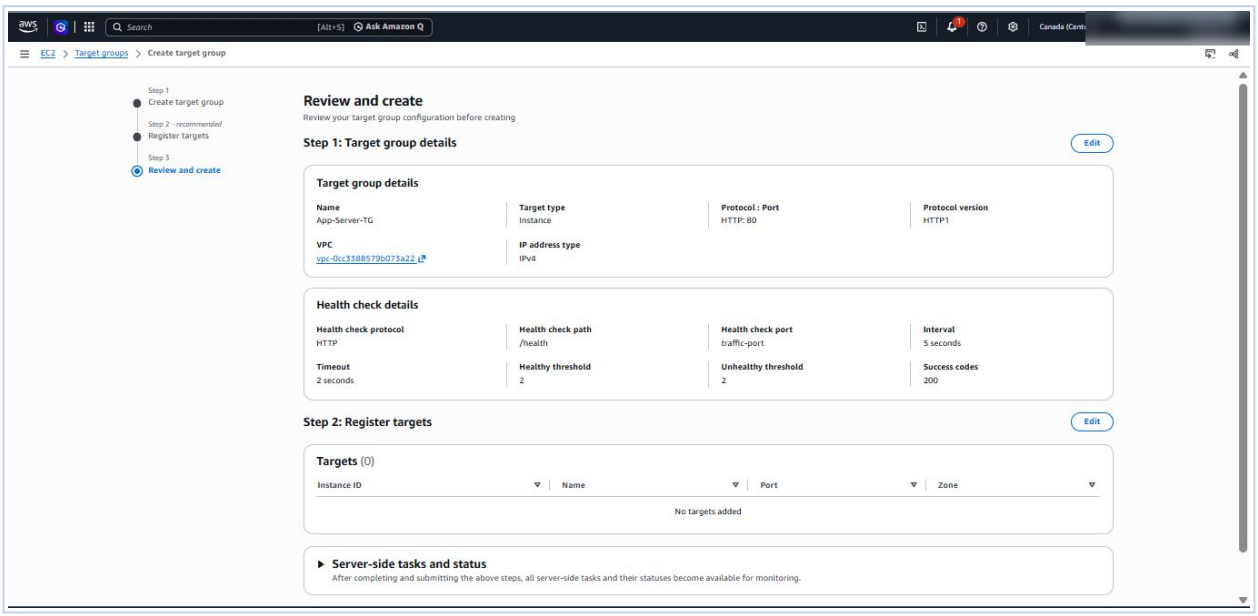


Figure 87. Review and create: the group settings are correct and Targets (0) is expected.

B. Create the launch template

Go to **EC2 > Launch templates > Create launch template**.

Setting	Value
Template name	App-Server-Launch-Template
AMI	My AMIs > Owned by me > App-Server-Image
Instance type	t3.micro
Key pair	Do not include in launch template
Security group	App-Tier-SG (no subnet in the template)
IAM instance profile (Advanced details)	IAM-ROLE-THREE-TIER-PROJECT

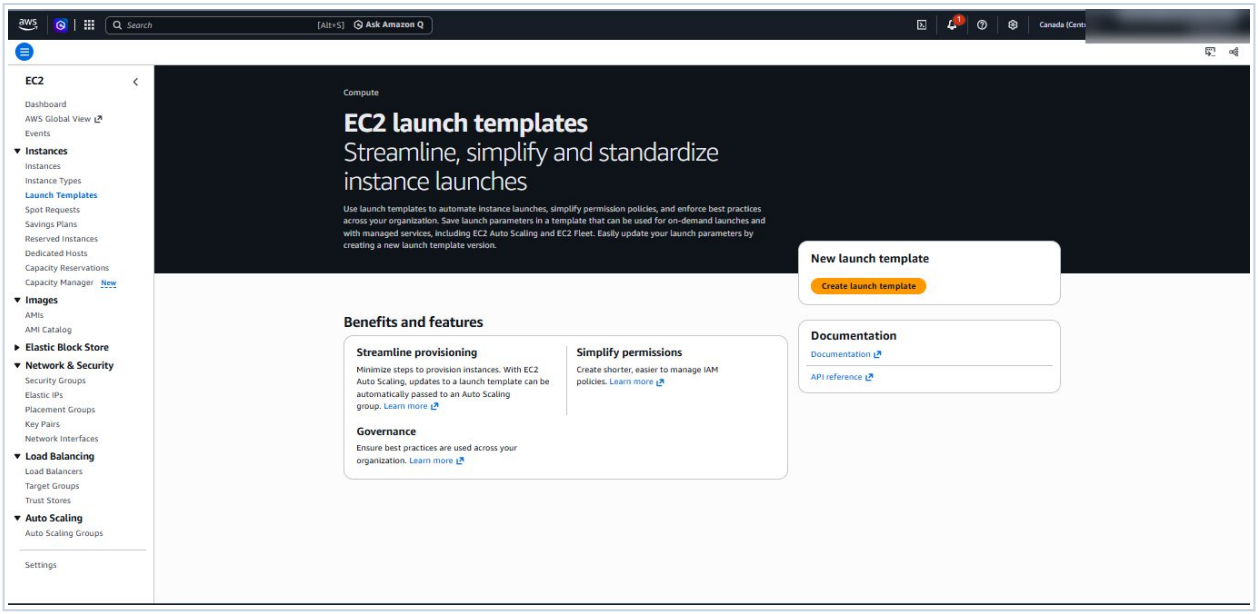


Figure 88. EC2 launch templates landing page: click Create launch template.

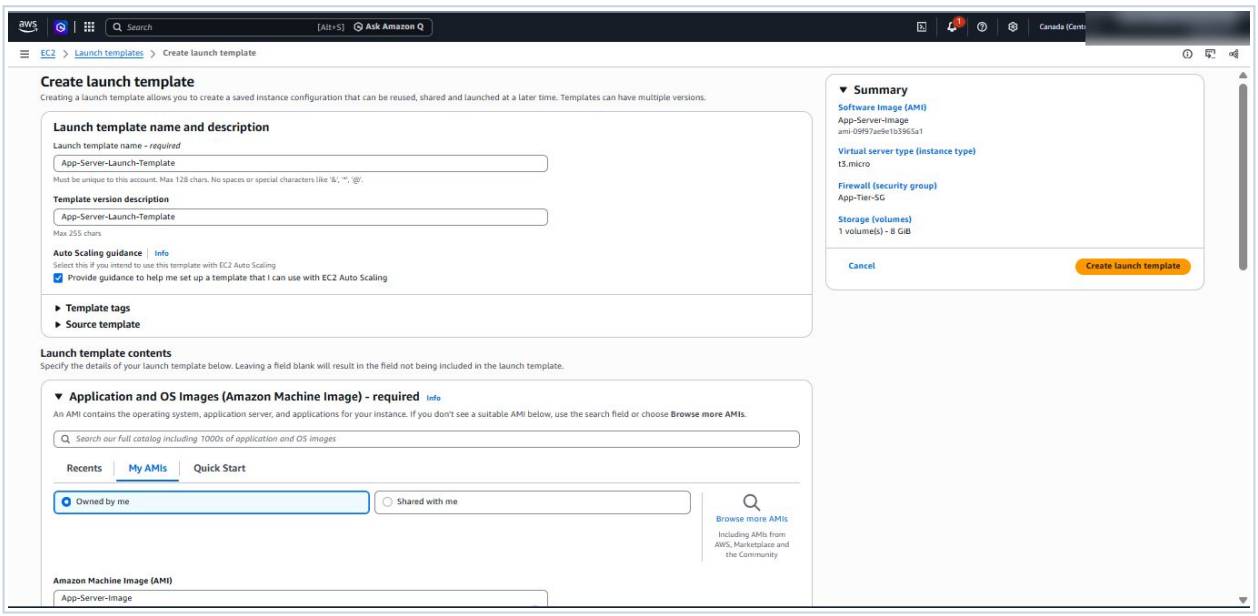


Figure 89. Template name App-Server-Launch-Template and the AMI App-Server-Image from My AMIs.

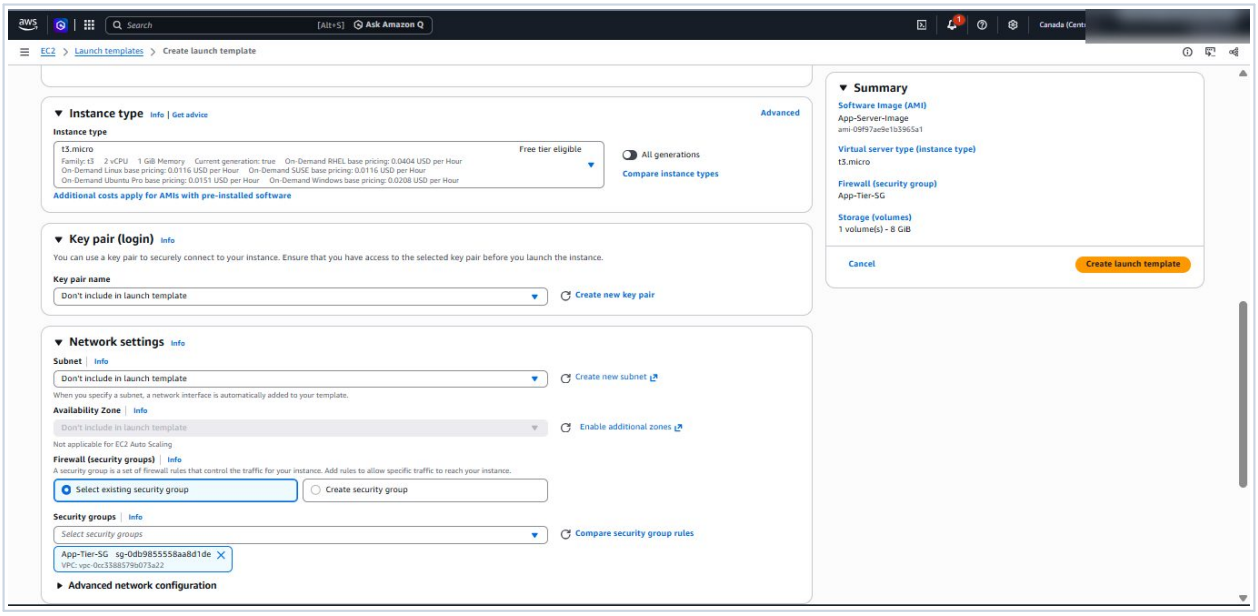


Figure 90. Instance type, key pair and network settings with the existing security group App-Tier-SG.

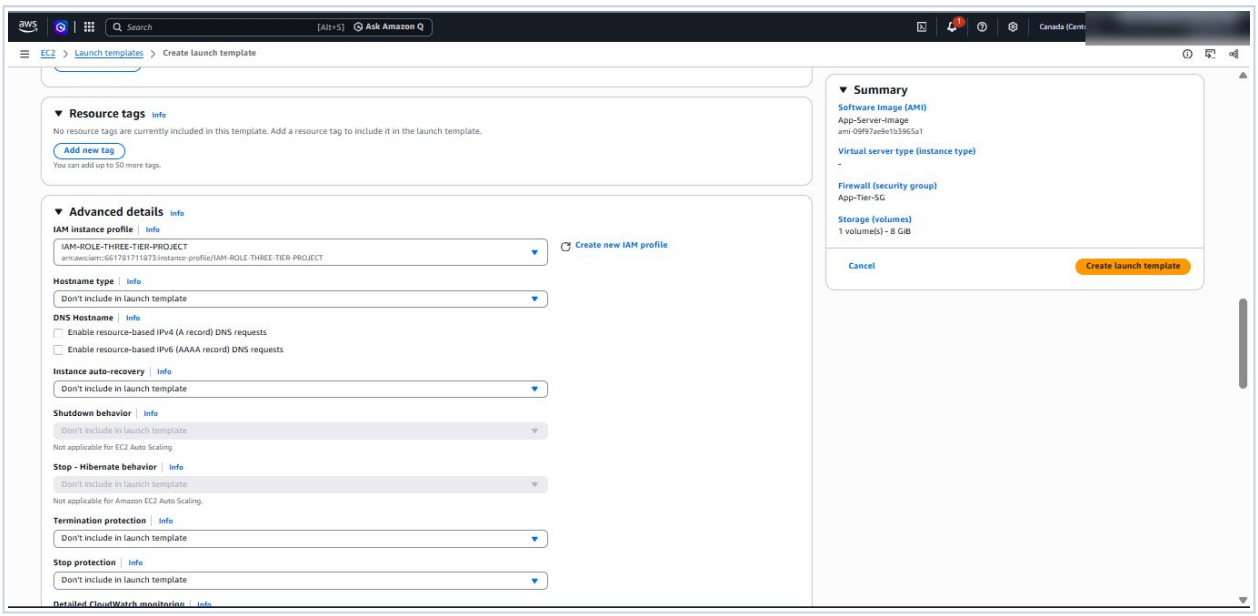


Figure 91. Resource tags and Advanced details: set the IAM instance profile to the project role.

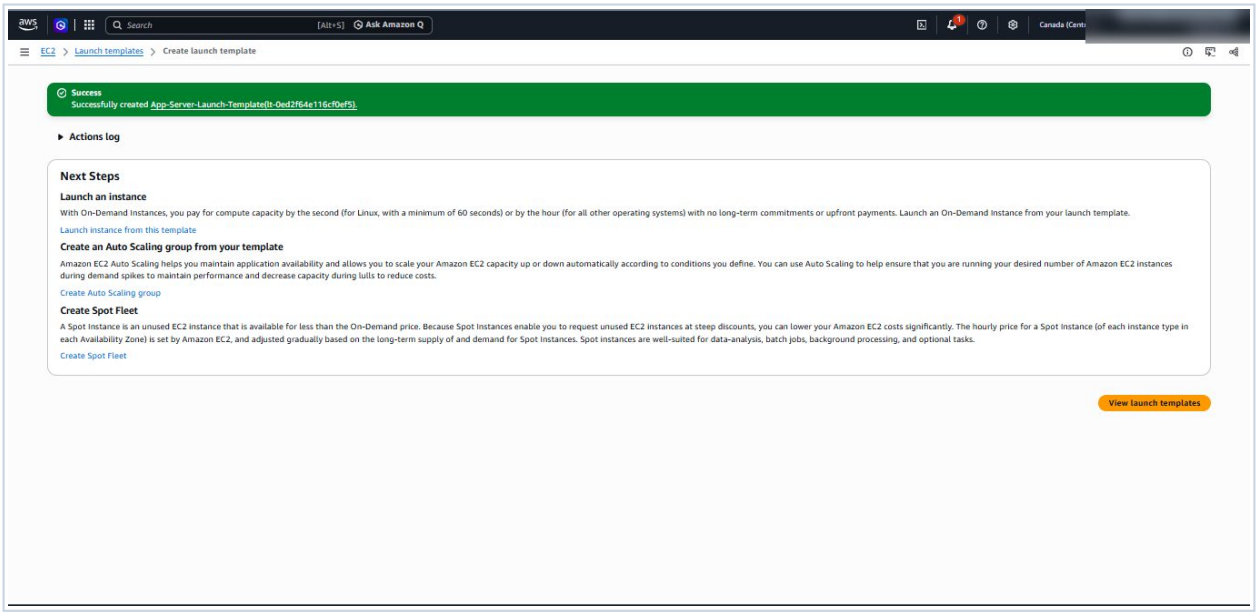


Figure 92. The launch template is created successfully.

C. Create the internal Application Load Balancer

Go to **EC2 > Load Balancers > Create load balancer**, choose **Application Load Balancer** and create:

Setting	Value
Name	A clear name of your choice, for example App-Tier-Internal-ALB
Scheme	Internal
VPC	Three-Tier-Project-VPC (not the default VPC)
Subnets	App-Tier-Subnet-1-Private (1a) and App-Tier-Subnet-2-Private (1b)
Security group	Internal-Load-Balancer-SG
Listener	HTTP : 80, forward to App-Server-TG

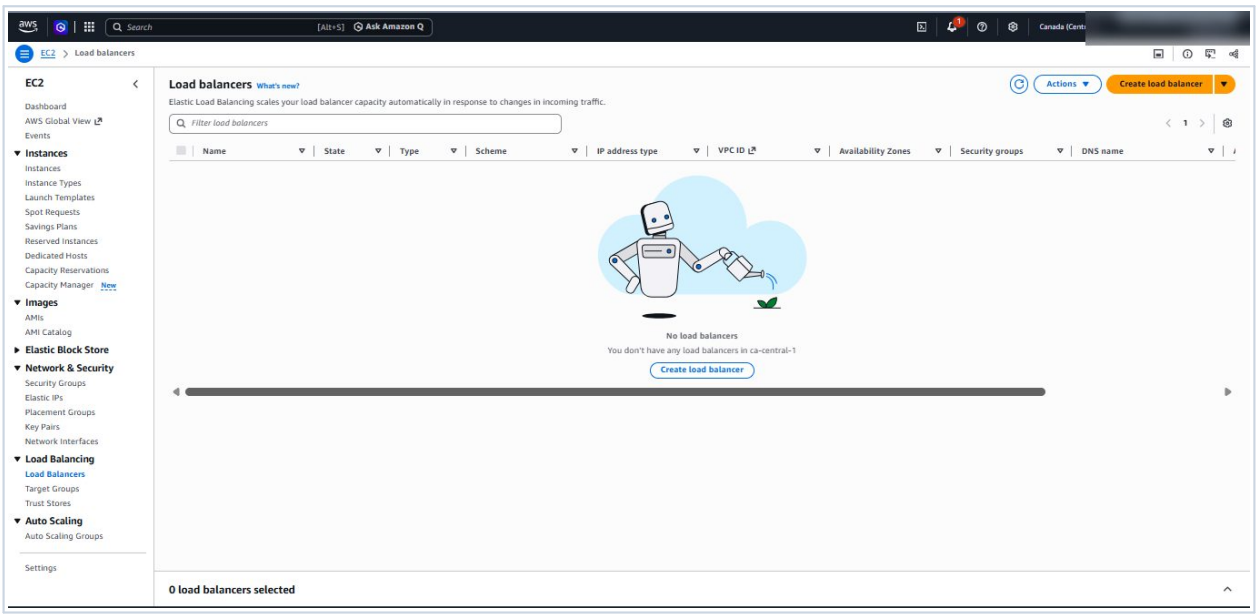


Figure 93. The Load balancers list is still empty: click Create load balancer.

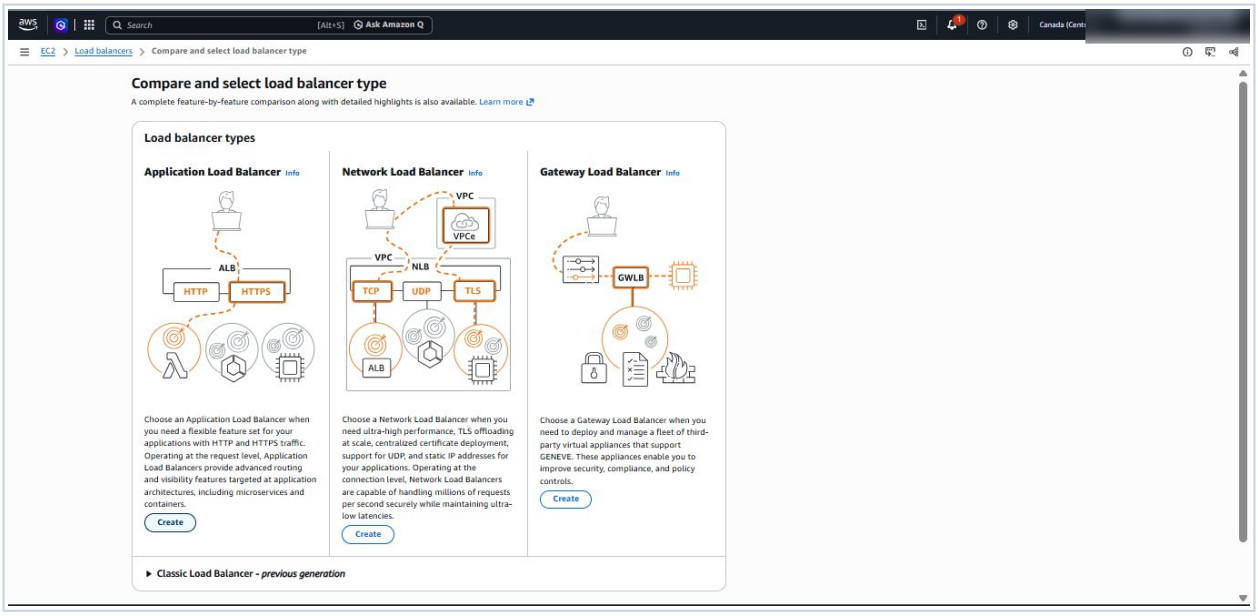


Figure 94. Compare and select load balancer type: choose Application Load Balancer.

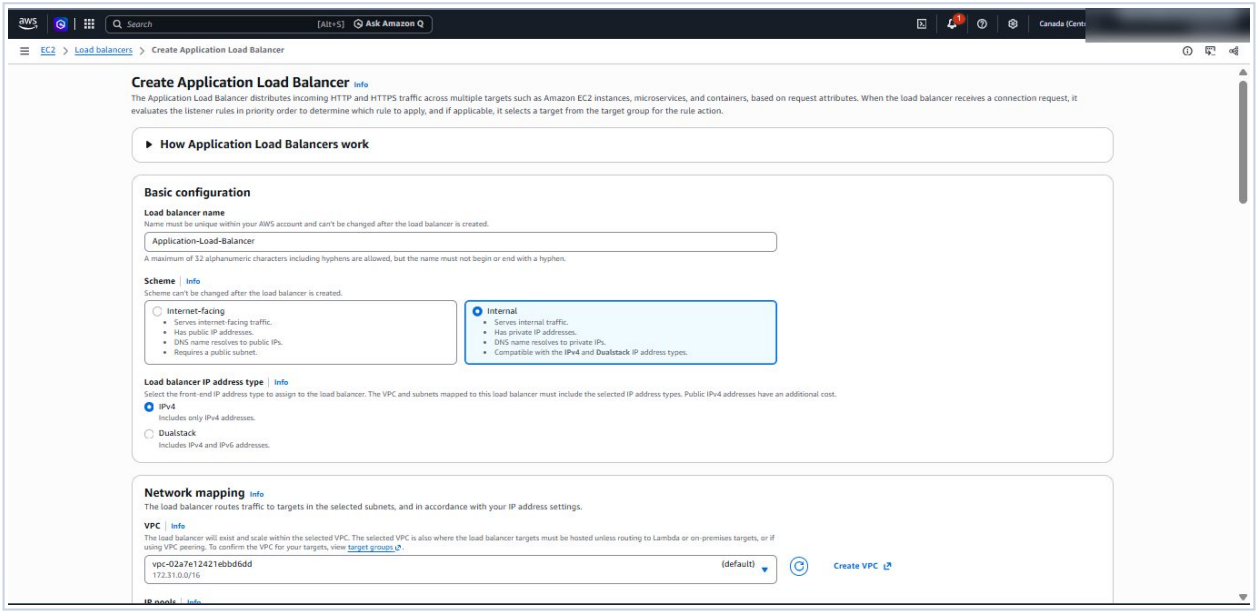


Figure 95. Basic configuration: scheme Internal. The VPC selector starts on the default VPC and must be changed.

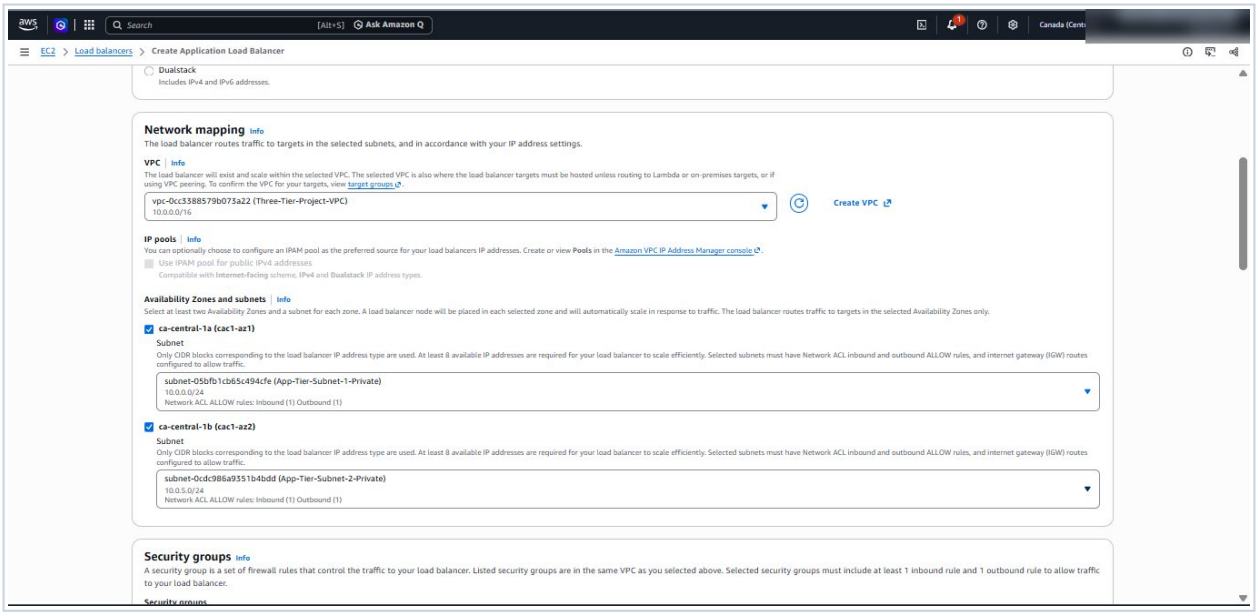


Figure 96. Network mapping: the project VPC and the two private app subnets.

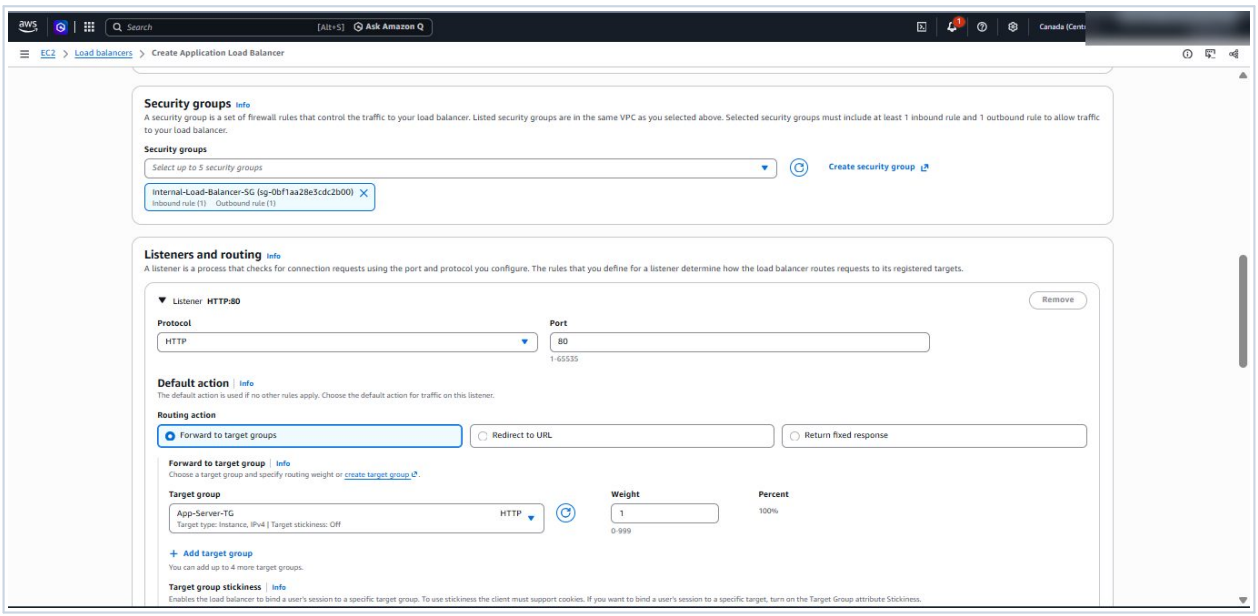


Figure 97. Security group Internal-Load-Balancer-SG and the HTTP:80 listener forwarding to App-Server-TG.

IMPORTANT

The VPC drop-down defaults to the default VPC. If you leave it, the target group will not appear and the subnets will be wrong. Always switch it to Three-Tier-Project-VPC first.

CHECKPOINT

App-Server-TG, App-Server-Launch-Template and the internal load balancer all exist. The app tier has a front door.

STEP 10 OF 15

SERVERS AND TRAFFIC

Create the Test Web Server and Set It Up

Meet the web server

The process mirrors the app server. Only the EC2 settings and the software differ.

Point nginx.conf at the internal load balancer

1. In **EC2 > Load Balancers**, select the *internal* load balancer and copy its DNS name.
2. On your PC open **application-code/nginx.conf**.
3. Paste the DNS name into the **proxy_pass** line of the **/api/** location block.
4. In the S3 bucket, delete the old nginx.conf and upload the edited one.

NGINX.CONF

```
#proxy for internal lb
location /api/{
        proxy_pass http://<INTERNAL-LOAD-BALANCER-DNS-NAME>:80/;
}
```

SCREENSHOT TO ADD

nginx.conf with the internal load balancer DNS name pasted in, and the internal load balancer details page.

IMPORTANT

Use the DNS name of the **internal** load balancer, not the external one. A wrong name here is the most common cause of 502 and 504 errors later.

Launch the test web server

Setting	Value
Name	Test-Web-Server
AMI / Type	Amazon Linux 2023 / t3.micro
VPC / Subnet	Three-Tier-Project-VPC / Web-Tier-Subnet-1-Public (10.0.1.0/24), auto-assign public IP enabled
Security group	Web-Tier-SG
IAM instance profile (Advanced details)	IAM-ROLE-THREE-TIER-PROJECT

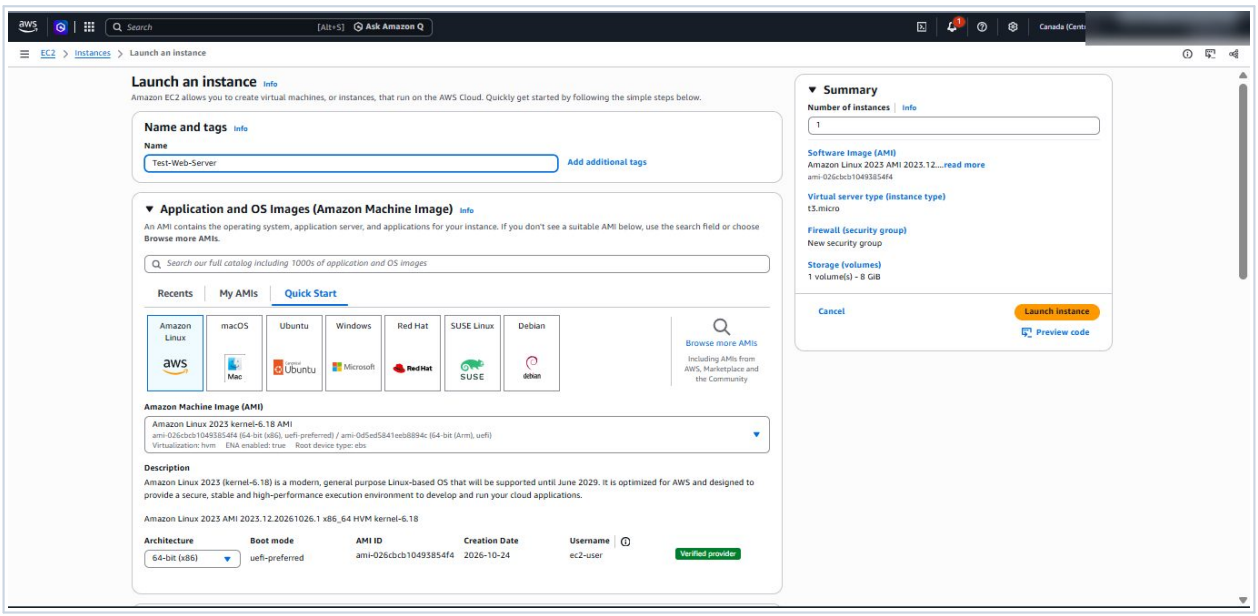


Figure 98. Launch an instance: name Test-Web-Server and the Amazon Linux 2023 AMI.

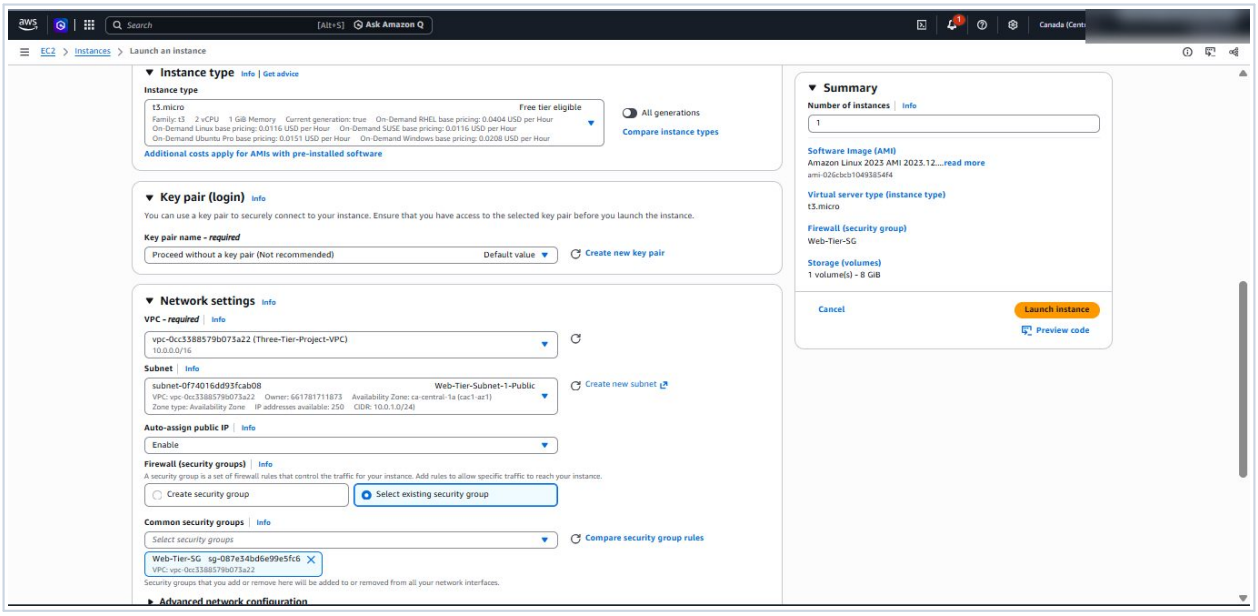


Figure 99. Instance type, key pair and network settings for Web-Tier-Subnet-1-Public with the existing group Web-Tier-SG.

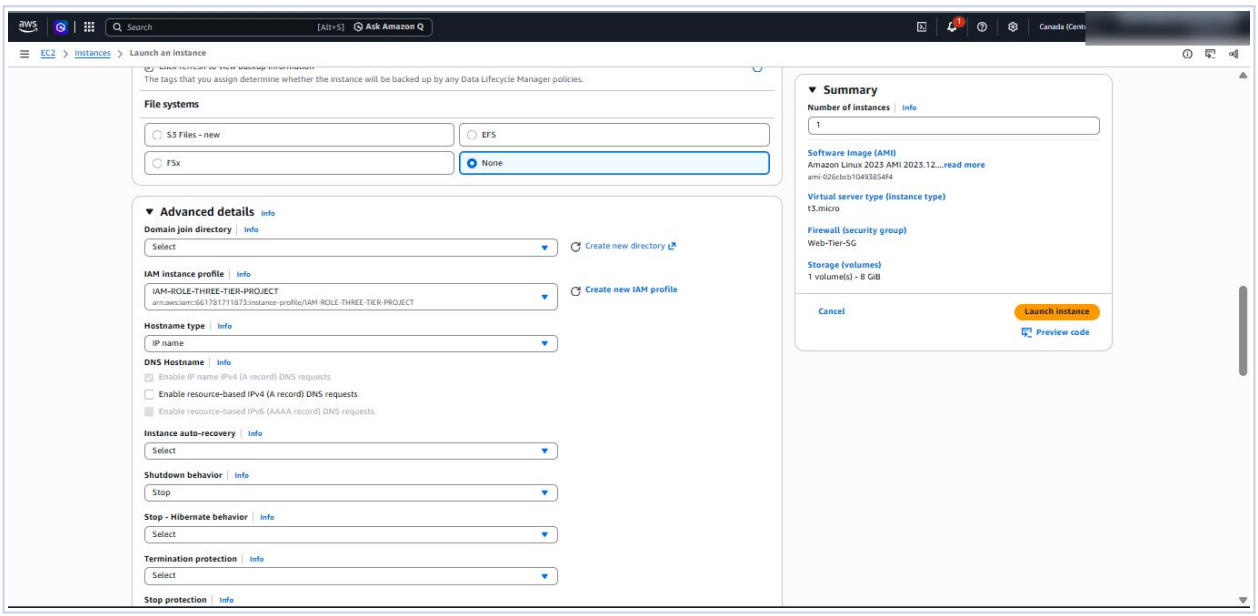


Figure 100. Advanced details: the IAM instance profile IAM-ROLE-THREE-TIER-PROJECT, then Launch instance.

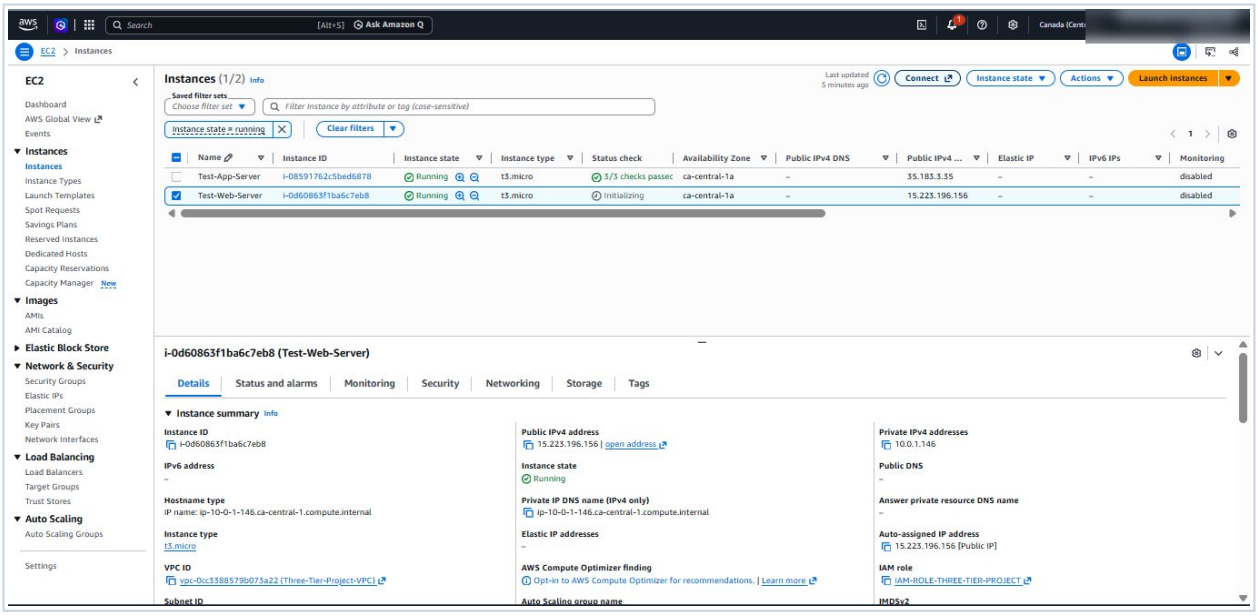


Figure 101. The Instances list: Test-App-Server and Test-Web-Server are both running.

Configure the web server

Connect to Test-Web-Server with Session Manager, as in Step 7, then follow the commands below.

1. Switch user and copy the web-tier code from S3

```bash
sudo su - ec2-user
cd /home/ec2-user
sudo aws s3 cp s3://three-tier-project-28-26/application-code/web-tier/ /home/ec2-user/web-tier/
    --recursive
sudo chown -R ec2-user:ec2-user /home/ec2-user/web-tier
sudo chmod -R 755 /home/ec2-user/web-tier
```

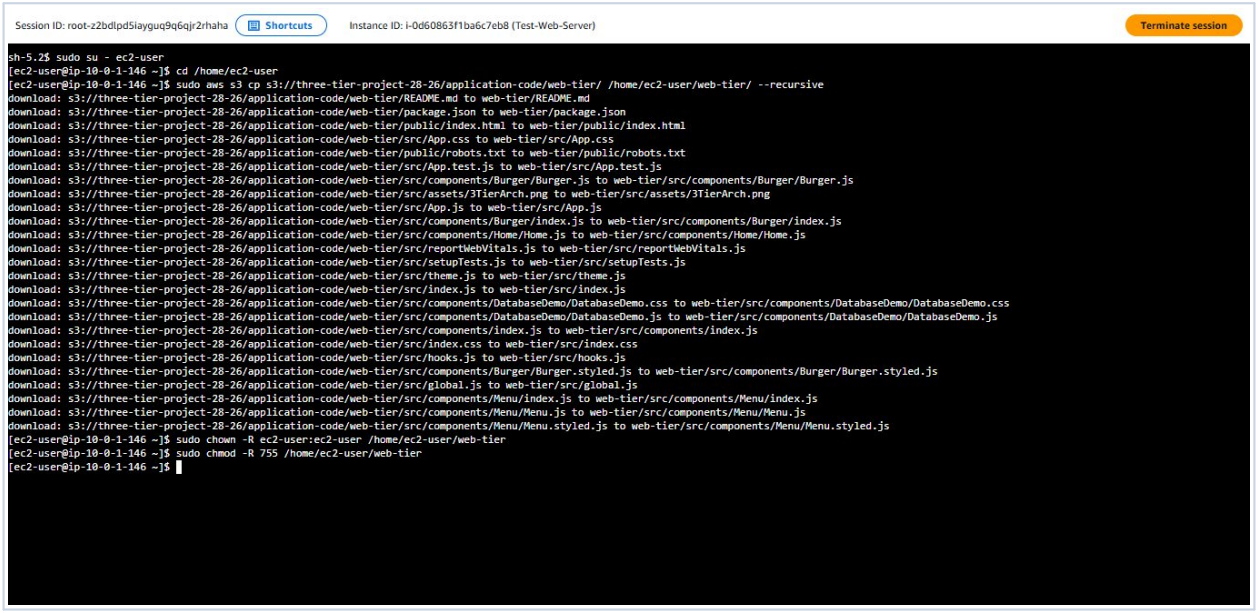


Figure 102. Session Manager terminal on Test-Web-Server: the web-tier files download from S3 and ownership and permissions are set.

2. Install Node.js and build the React app

BASH

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
nvm use 16

cd /home/ec2-user/web-tier
npm install
npm run build
```

3. Install nginx and load the project configuration

BASH

```
sudo dnf install nginx -y

cd /etc/nginx
sudo rm nginx.conf
sudo aws s3 cp s3://three-tier-project-28-26/application-code/nginx.conf .

sudo chmod -R 755 /home/ec2-user
sudo systemctl enable nginx
sudo systemctl restart nginx
```

PRO TIP

If nginx fails to start, run **sudo nginx -t**. It names the exact line of nginx.conf that is wrong.

CHECKPOINT

Opening the Test-Web-Server public IP in a browser shows the application, and pages that use the API load without a 502 or 504 error.

STEP 11 OF 15

SERVERS AND TRAFFIC

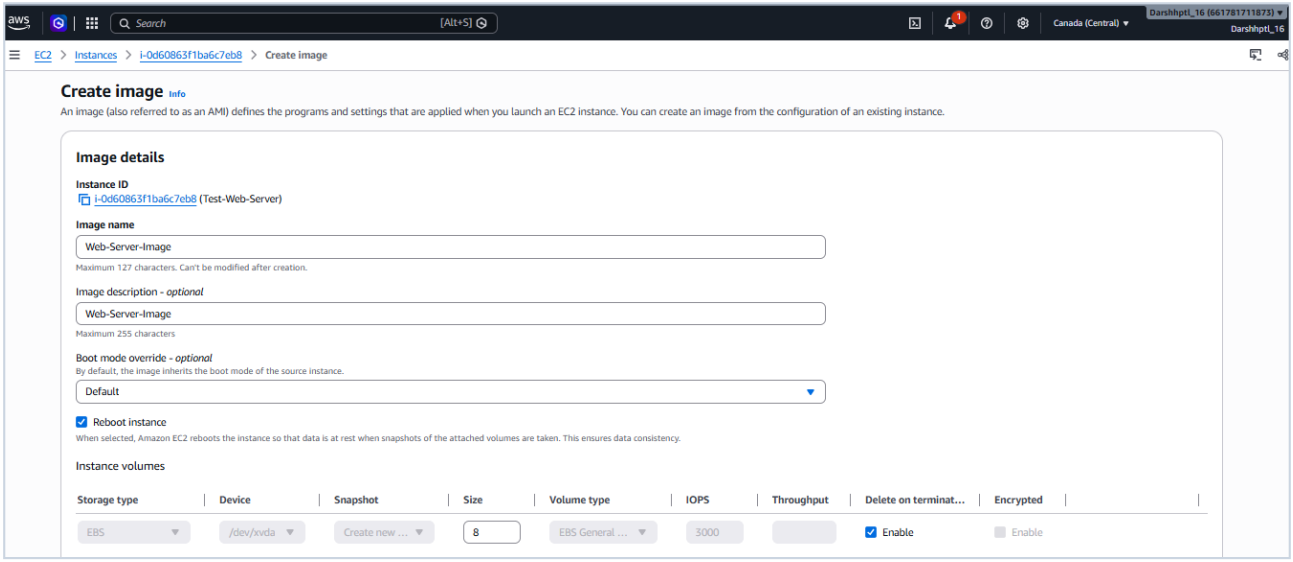
Create an AMI of the Web Server

Photograph the second server

WHY THIS STEP?

As with the app tier, the web AMI captures the working web server so Auto Scaling can launch identical copies.

1. In EC2, select **Test-Web-Server**.
2. Choose **Actions > Image and templates > Create image**.
3. Name the image Web-Server-Image and click **Create image**.
4. Wait until **Images > AMIs** shows **Available** for both images.



Create image for Test-Web-Server: image name Web-Server-Image, with Reboot instance selected.

CHECKPOINT

Two AMIs exist, one per tier. Phase 3 now needs only the web tier load balancing.

STEP 12 OF 15 SERVERS AND TRAFFIC

Target Group, Launch Template and External Load Balancer (Web Tier)

Open the front door

A. Create the target group (web tier)

Setting	Value
Target type	Instances
Target group name	Web-Server-TG
Protocol : Port	HTTP : 80
VPC	Three-Tier-Project-VPC
Health check path	/health
Targets	None to register by hand

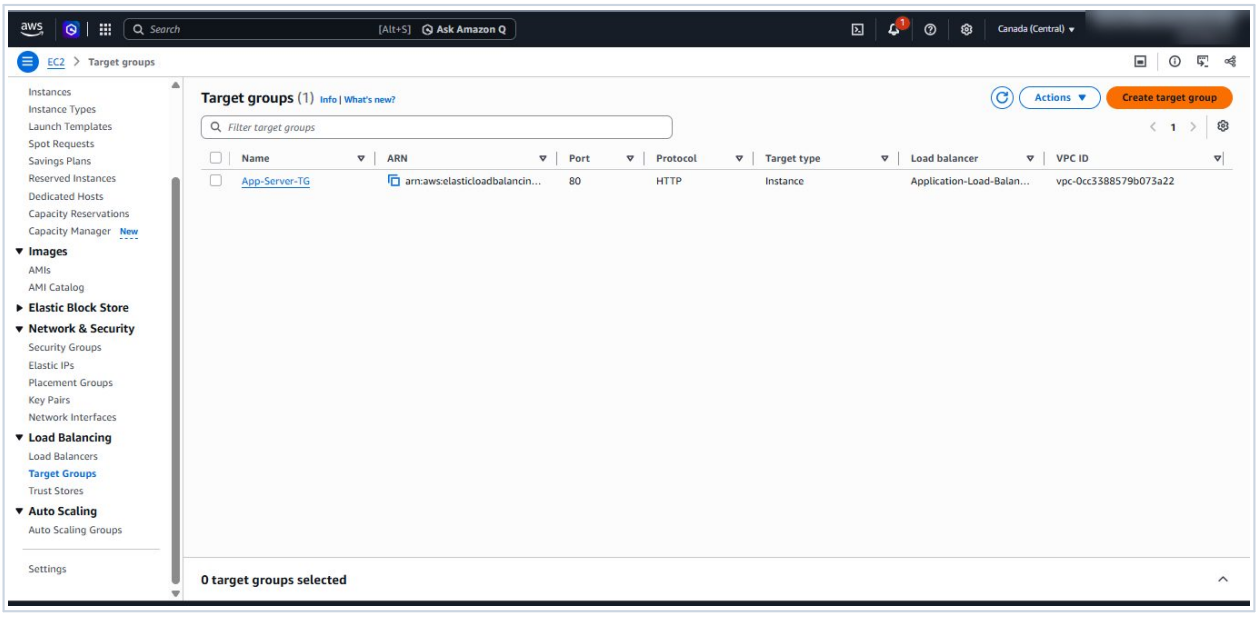


Figure 103. Target groups list: App-Server-TG already exists. Click Create target group for the web tier.

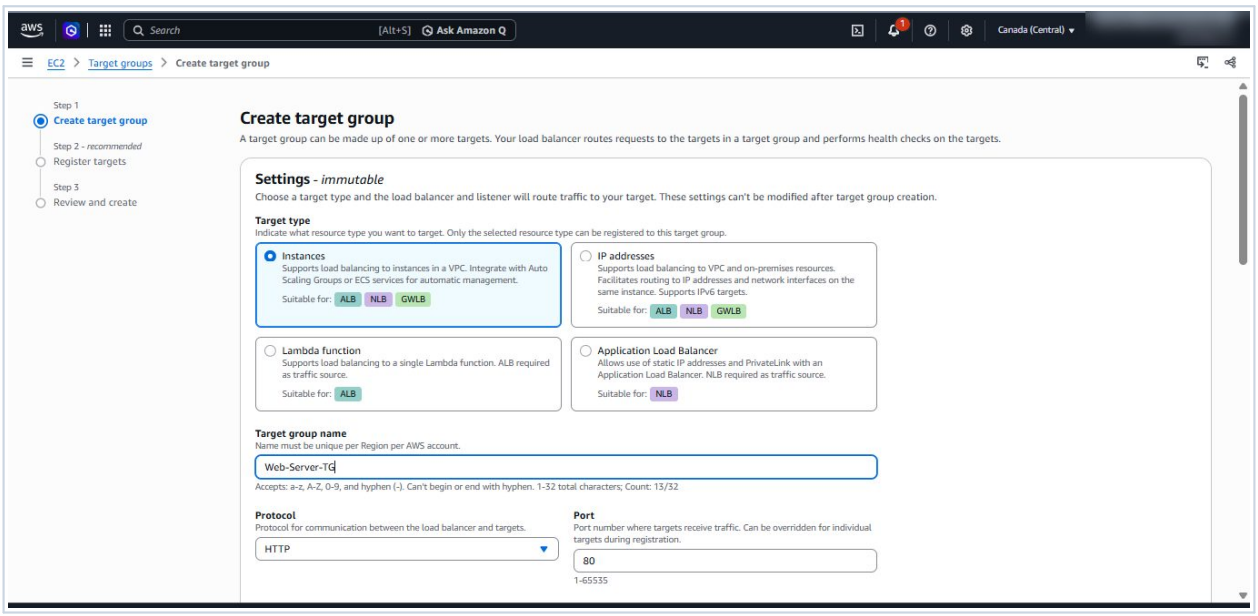


Figure 104. Target type Instances and the name Web-Server-TG (HTTP, port 80).

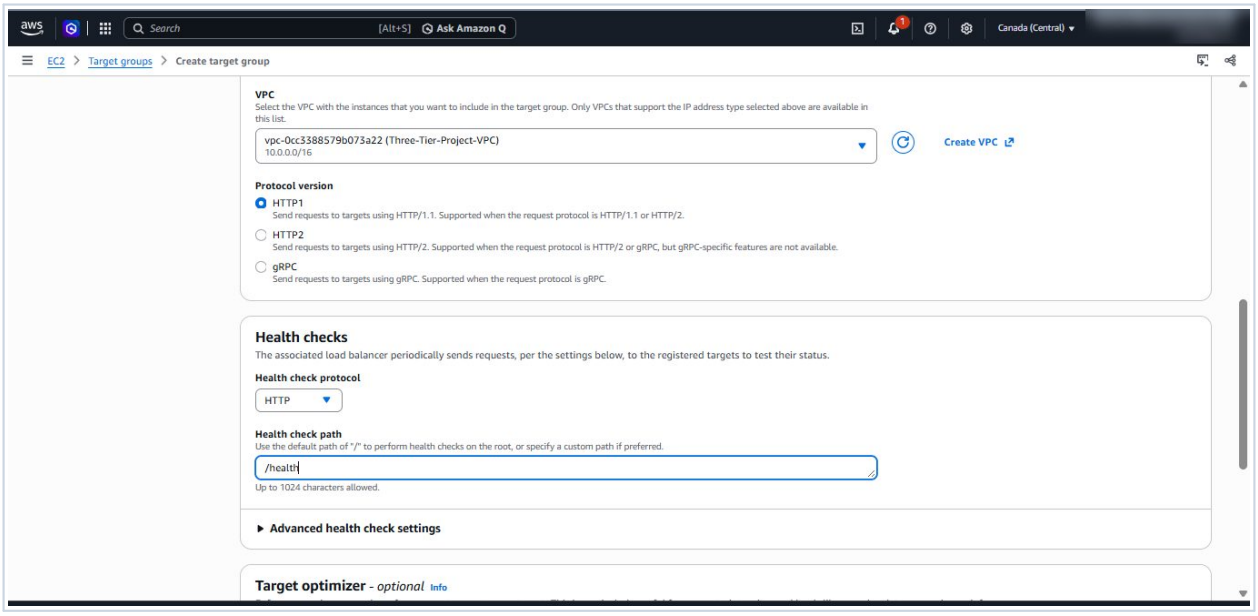


Figure 105. Choose the project VPC and set the health check path to /health.

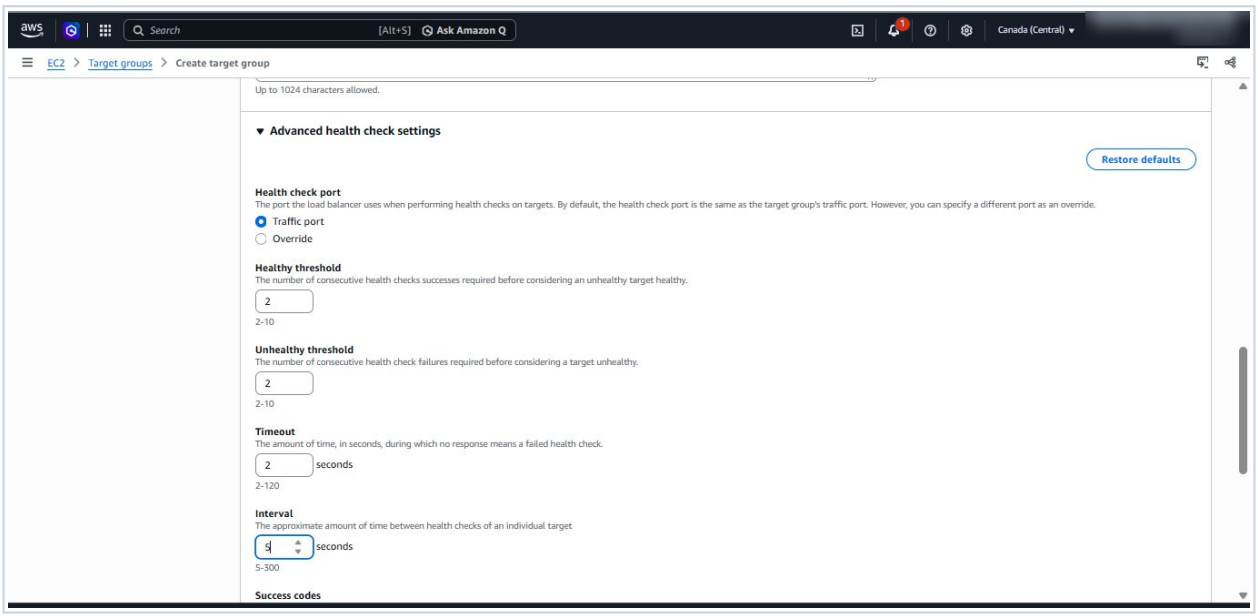


Figure 106. Advanced health check settings: thresholds, timeout and interval for faster failure detection.

B. Create the launch template (web tier)

Setting	Value
Template name	Web-Server-Launch-Template
AMI	My AMIs > Owned by me > Web-Server-Image
Instance type	t3.micro
Key pair	Do not include in launch template
Security group	Web-Tier-SG
IAM instance profile (Advanced details)	IAM-ROLE-THREE-TIER-PROJECT

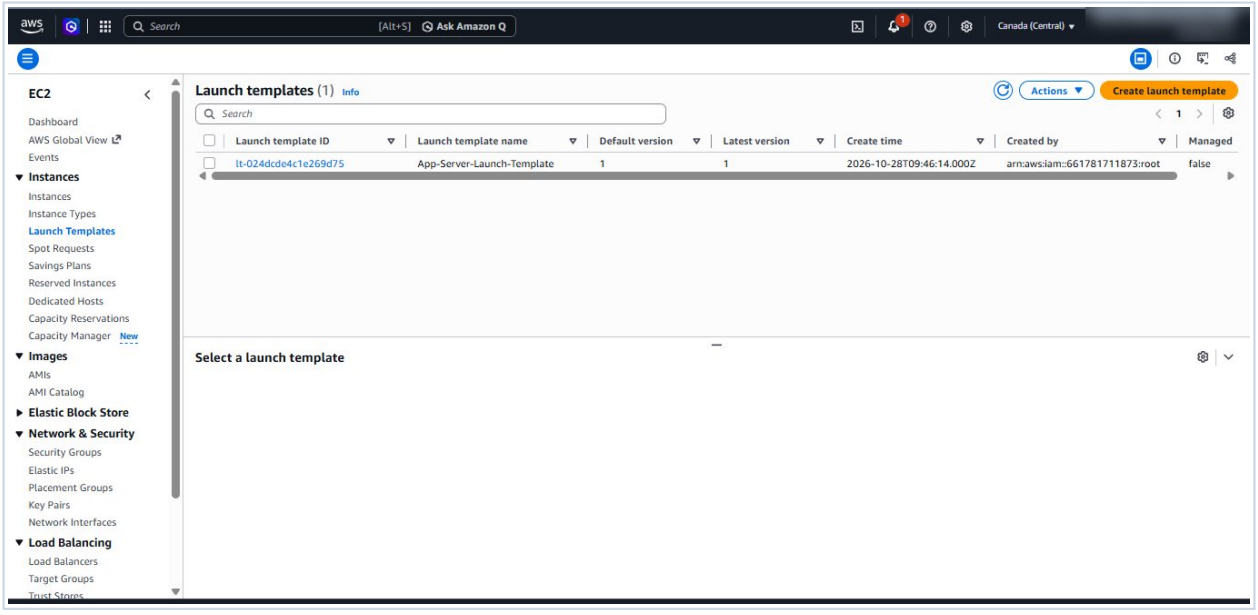


Figure 107. The Launch templates list shows App-Server-Launch-Template. Click Create launch template again.

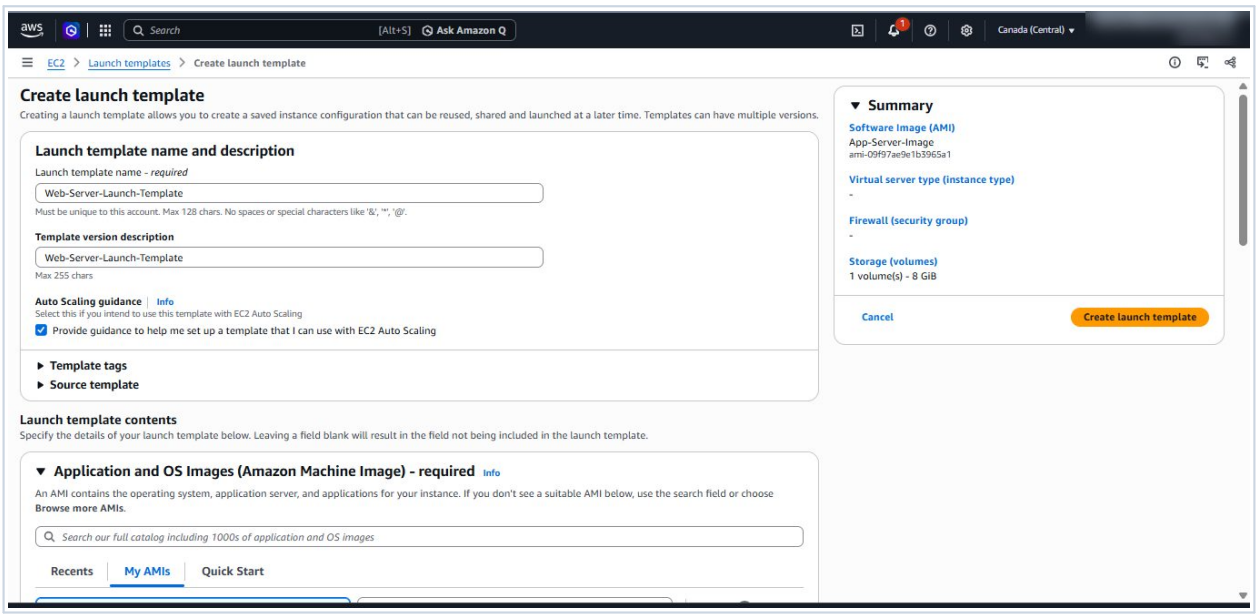


Figure 108. Template name Web-Server-Launch-Template and the AMI selection.

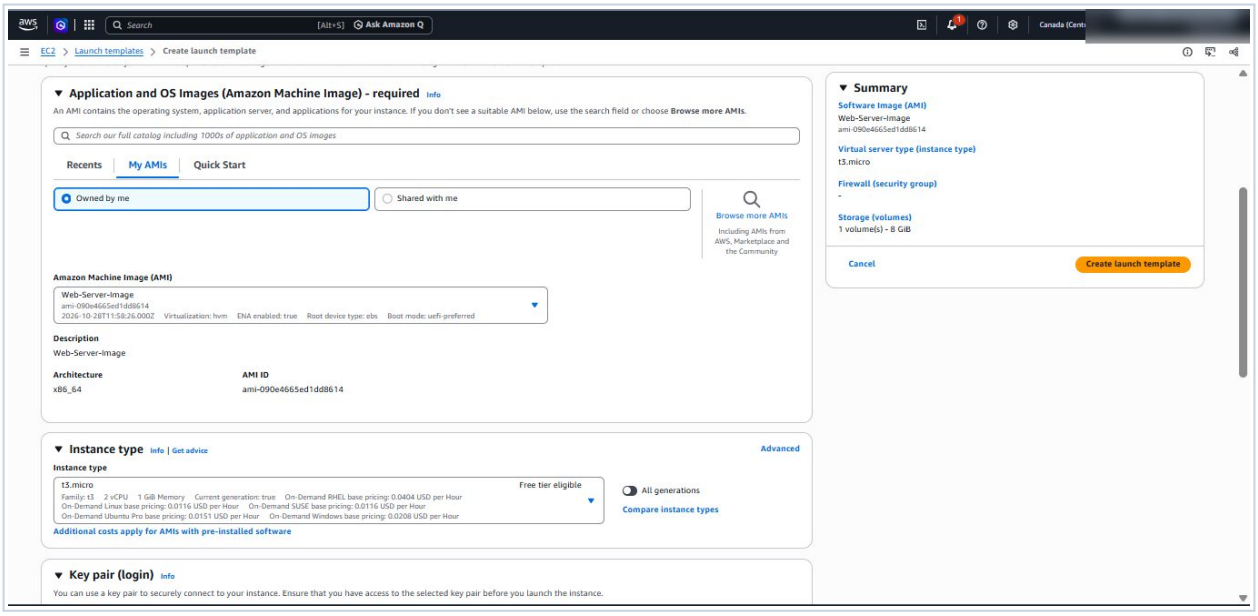


Figure 109. Choose My AMIs, the web server image, and the instance type.

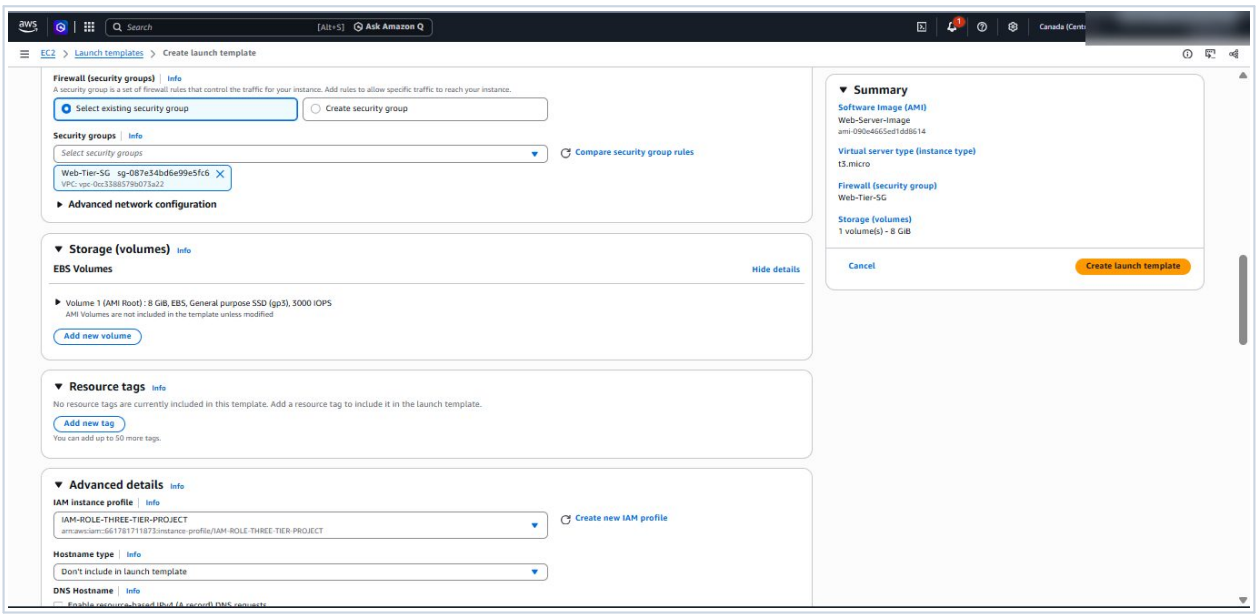


Figure 110. Network settings (security group Web-Tier-SG), storage and Advanced details with the IAM instance profile.

C. Create the external (internet-facing) load balancer

Setting	Value
Name	A clear name of your choice, for example Web-Tier-External-ALB
Scheme	Internet-facing
VPC	Three-Tier-Project-VPC
Subnets	Web-Tier-Subnet-1-Public (1a) and Web-Tier-Subnet-2-Public (1b)
Security group	External-Load-Balancer-SG
Listener	HTTP : 80, forward to Web-Server-TG

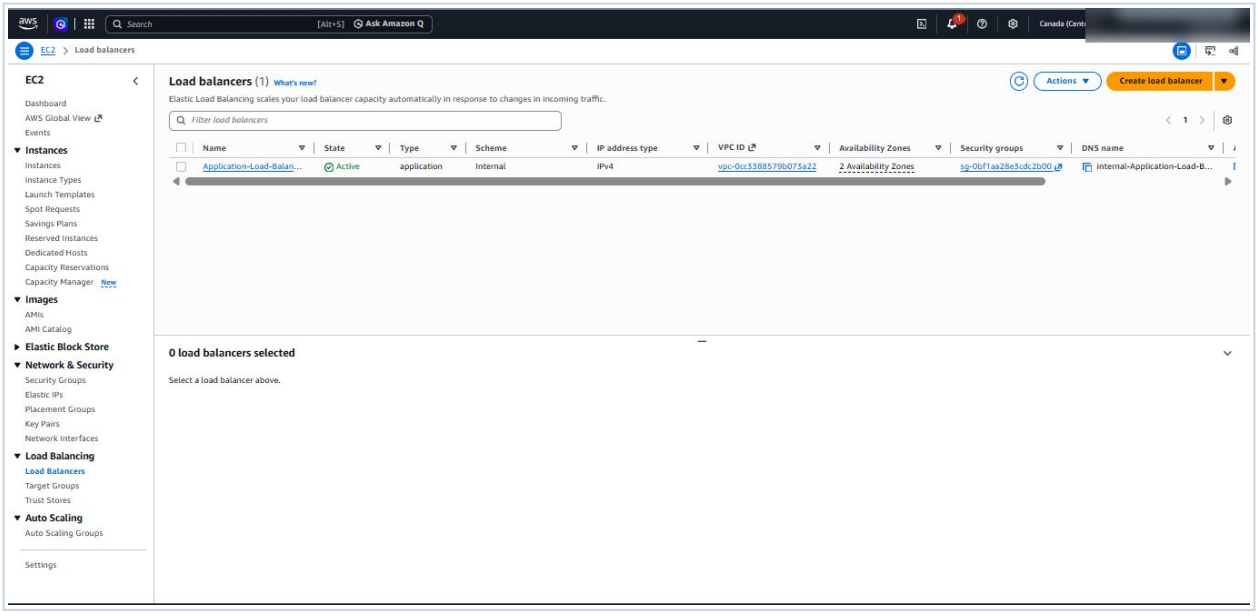


Figure 111. The Load balancers list now shows the internal load balancer as Active.

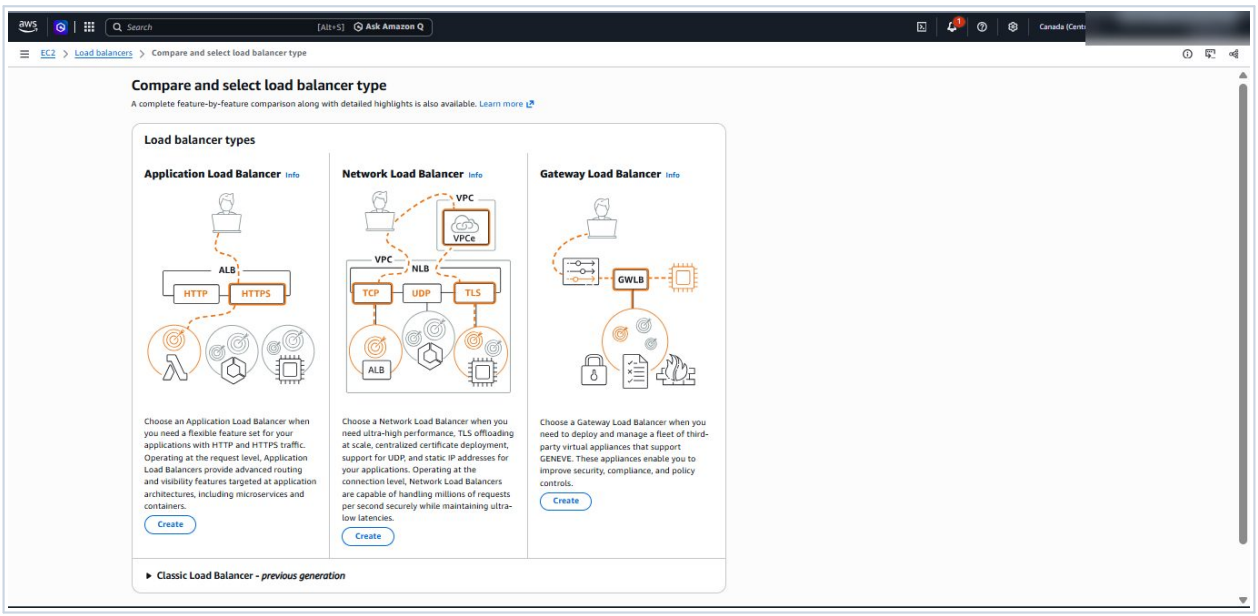


Figure 112. Select Application Load Balancer as the type.

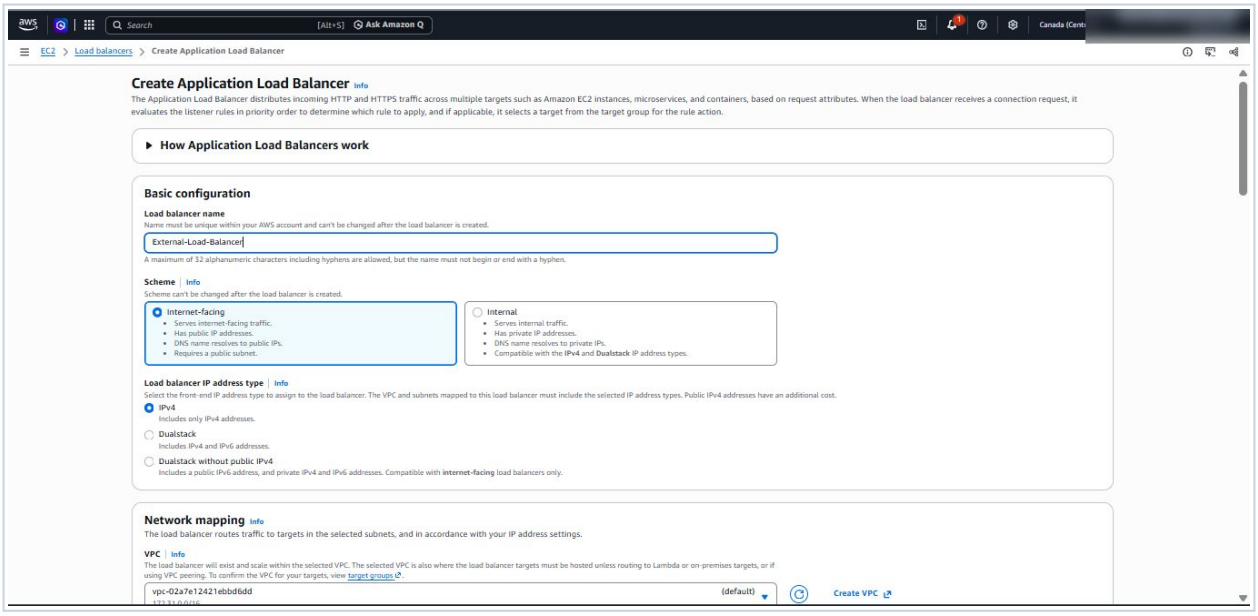


Figure 113. Basic configuration: scheme Internet-facing, IP address type IPv4.

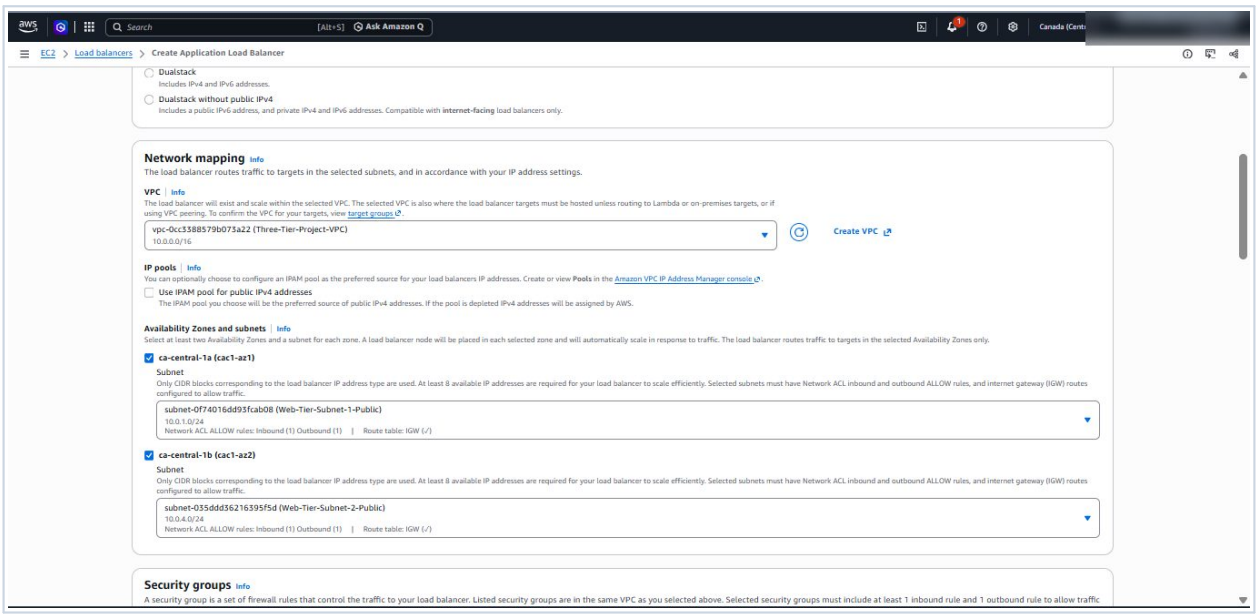


Figure 114. Network mapping: project VPC and a public subnet in each Availability Zone.

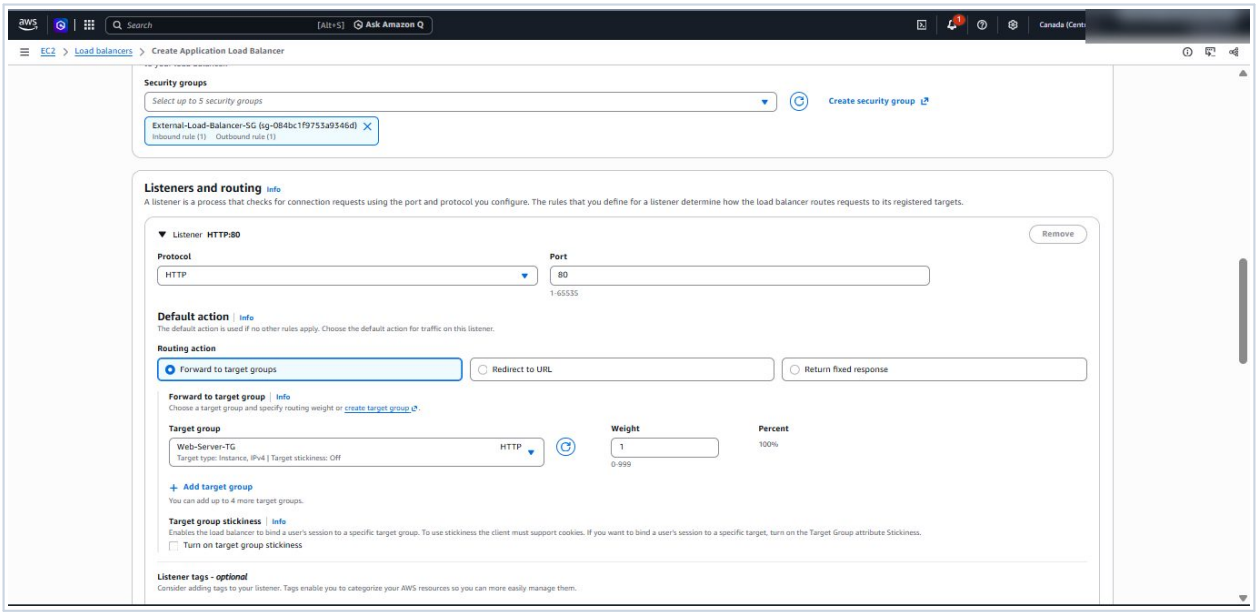


Figure 115. Security group External-Load-Balancer-SG and the HTTP:80 listener forwarding to Web-Server-TG.

CHECKPOINT

Both target groups, both launch templates and both load balancers exist. The next step lets Auto Scaling fill the target groups with servers.

STEP 13 OF 15

SCALE AND LAUNCH

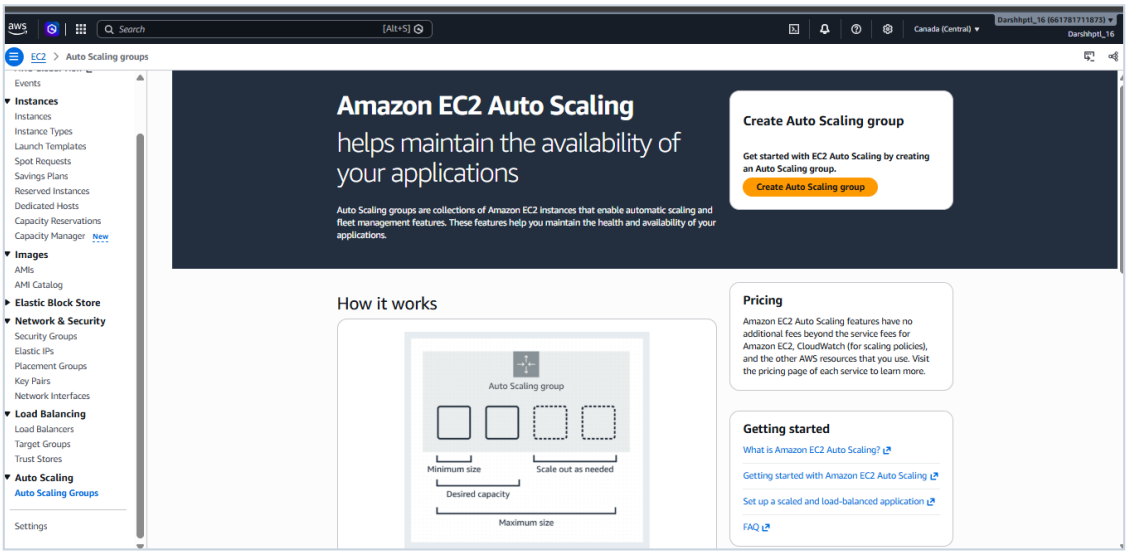
Create the Auto Scaling Groups

Let AWS keep the lights on

WHY THIS STEP?

Auto Scaling groups build servers from the launch templates, register them with the target groups and replace any server that fails its health check, so you never babysit an instance again.

Go to **EC2 > Auto Scaling groups > Create Auto Scaling group** and build one group per tier. The values below are the settings this design calls for.



EC2 > Auto Scaling groups: choose Create Auto Scaling group to start building each group.

A. Auto Scaling group for the app tier

Setting	Value
Name	A clear name of your choice, for example App-Server-ASG
Launch template	App-Server-Launch-Template
VPC / Subnets	Three-Tier-Project-VPC / App-Tier-Subnet-1-Private and App-Tier-Subnet-2-Private
Load balancing	Attach to an existing load balancer: target group App-Server-TG
Health checks	Turn on Elastic Load Balancing health checks
Group size	Desired 2, minimum 2, maximum 2 (or higher for scaling)

SCREENSHOT TO ADD

Auto Scaling group for the app tier: launch template, subnets, target group and group size.

B. Auto Scaling group for the web tier

Setting	Value
Name	A clear name of your choice, for example Web-Server-ASG
Launch template	Web-Server-Launch-Template
VPC / Subnets	Three-Tier-Project-VPC / Web-Tier-Subnet-1-Public and Web-Tier-Subnet-2-Public
Load balancing	Attach to an existing load balancer: target group Web-Server-TG
Health checks	Turn on Elastic Load Balancing health checks
Group size	Desired 2, minimum 2, maximum 2 (or higher for scaling)

SCREENSHOT TO ADD

Auto Scaling group for the web tier: launch template, subnets, target group and group size.

PRO TIP

Use a desired size of at least 2 per tier. With one server in each zone, a zone failure still leaves one healthy server serving traffic.

CHECKPOINT

Each Auto Scaling group launches its servers across both zones and both target groups show the registered targets as **healthy**.

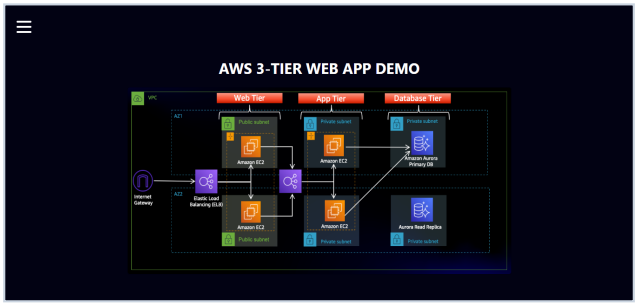
STEP 14 OF 15 SCALE AND LAUNCH

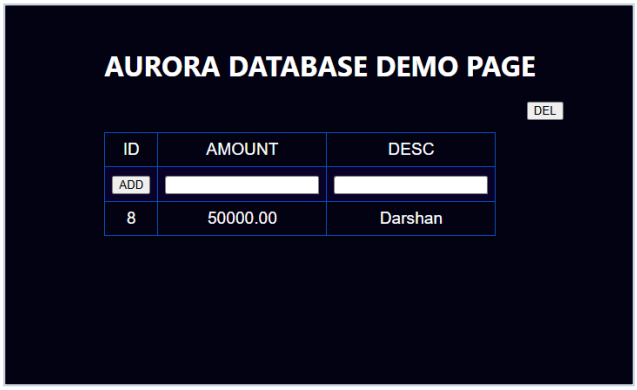
Access the Application

Open the front door and walk in

WHY THIS STEP?
This is the moment of truth: a request from your browser must cross the external load balancer, the web servers, the internal load balancer, the app servers and finally the database.

1. Open **EC2 > Load Balancers** and select the *internet-facing* load balancer.
2. Copy its **DNS name**.
3. Paste it into your browser as **http://<external-load-balancer-dns-name>**.
4. Open the **Database Demo** page, add a transaction and confirm it is listed.
5. Confirm the same row exists in MySQL using the client from Step 7.





The application opened in the browser through the external load balancer DNS name, with a transaction added on the Database Demo page.

Prove the fault tolerance

- Terminate one web or app instance in the EC2 console. The load balancer stops sending it traffic, and the Auto Scaling group launches a replacement from the launch template.
- Refresh the application during this time. It should keep working because the second server in the other zone is still healthy.

CHECKPOINT
The application loads through the external DNS name, a new transaction is stored in MySQL, and the app survives the loss of one server.

STEP 15 OF 15

SCALE AND LAUNCH

Clean Up the Infrastructure

Switch off the meter

COST WARNING

NAT gateways, load balancers, the database and the instances all bill by the hour. Delete everything as soon as you have your screenshots.

Delete in this order so no resource is blocked by something that still depends on it:

1. **Auto Scaling groups** for the web and app tiers (this terminates their instances).
2. **Load balancers**, then **target groups**.
3. **Launch templates**.
4. **AMIs**: deregister both images, then delete their snapshots.
5. Any remaining **test instances**.
6. **RDS database** (choose whether to keep a final snapshot), then the **DB subnet group**.
7. **NAT gateways**, then release both **Elastic IPs**.
8. **Internet gateway**: detach it from the VPC and delete it.
9. **Security groups**, **route tables** (non-main) and **subnets**.
10. The **VPC**.
11. **S3 bucket**: empty it first, then delete it.
12. **IAM role** IAM-ROLE-THREE-TIER-PROJECT.

WARNING

Elastic IPs that are allocated but not attached keep billing. Check **VPC > Elastic IPs** at the end to make sure the list is empty.

CHECKPOINT

The Billing console shows no running resources from this project and the VPC list contains only the default VPC.

Troubleshooting Quick Guide

Symptom	Likely cause	What to check
Private server cannot download packages	No route to a NAT gateway, or the NAT gateway sits in a private subnet	App-Tier-RT-1/2 have 0.0.0.0/0 to the NAT gateway in the same zone; NAT gateways are in the public subnets
Instance is not listed under Session Manager or Fleet Manager	Systems Manager not enabled, or the instance cannot reach the SSM endpoints	Default Host Management Configuration is on; NAT gateway is Available; wait a few minutes after launch
MySQL connection times out	Security group or endpoint mismatch	DB-Tier-SG allows 3306 from App-Tier-SG; you used the RDS endpoint, not an IP; public access is No on purpose
Target group shows unhealthy targets	Wrong port or health check path, or PM2 or nginx not running	App-Server-TG uses port 4000 and /health; run pm2 status; Web-Server-TG uses port 80; run systemctl status nginx
Browser shows 502 or 504	nginx.conf points at the wrong load balancer	proxy_pass uses the internal load balancer DNS name on port 80; Internal-Load-Balancer-SG allows Web-Tier-SG
Target group missing when creating a load balancer	The VPC selector is still on the default VPC	Change VPC to Three-Tier-Project-VPC before choosing subnets
Auto Scaling launches but servers never register	Launch template has the wrong AMI, group or profile	Template uses the right AMI, tier security group and IAM instance profile

Appendix A: Resource Inventory

Resource	Name / value
Region and zones	Canada (Central) ca-central-1: ca-central-1a and ca-central-1b
VPC	Three-Tier-Project-VPC, 10.0.0.0/16 (vpc-0cc3388579b073a22)
Public subnets	Web-Tier-Subnet-1-Public 10.0.1.0/24; Web-Tier-Subnet-2-Public 10.0.4.0/24
Private app subnets	App-Tier-Subnet-1-Private 10.0.0.0/24; App-Tier-Subnet-2-Private 10.0.5.0/24
Private DB subnets	DB-Tier-Subnet-1-Private 10.0.3.0/24; DB-Tier-Subnet-2-Private 10.0.6.0/24
Gateways	Three-Tier-Project-IGW; NAT-GT-1 and NAT-GT-2 (each with an Elastic IP)
Route tables	Web-Tier-RT, App-Tier-RT-1, App-Tier-RT-2
Security groups (5)	External-Load-Balancer-SG, Web-Tier-SG, Internal-Load-Balancer-SG, App-Tier-SG, DB-Tier-SG
Database	RDS for MySQL 8.4.11, identifier webapptier, user admin, database webappdb, table transactions
DB subnet group	Three-Tier-Project-DB-Subnet-Group
IAM role	IAM-ROLE-THREE-TIER-PROJECT (AmazonS3ReadOnlyAccess)
S3 bucket	three-tier-project-28-26 (folder application-code, 52 files)
Test servers	Test-App-Server (i-08591762c5bed6878); Test-Web-Server (i-0d60863f1ba6c7eb8)
AMIs	App-Server-Image; Web-Server-Image
Launch templates	App-Server-Launch-Template; Web-Server-Launch-Template
Target groups	App-Server-TG (HTTP 4000, /health); Web-Server-TG (HTTP 80, /health)
Load balancers	Internal ALB (private app subnets); external ALB (public web subnets)

Appendix B: Ideas to Harden for Production

This project is built to teach the architecture. Before using the same design for a real workload, consider these upgrades:

- **HTTPS everywhere.** Add an ACM certificate and an HTTPS listener on the external load balancer, then redirect HTTP to HTTPS.
- **Multi-AZ database.** Switch the RDS instance to a Multi-AZ deployment so a standby takes over automatically.
- **Secrets Manager.** Let RDS manage the master password in AWS Secrets Manager instead of a hand-typed password in DbConfig.js.
- **Tighter outbound rules.** Replace the allow-all outbound rule on each security group with only the ports that tier needs.
- **VPC endpoints.** Add gateway or interface endpoints for S3 and Systems Manager so private servers need less NAT traffic.
- **Web Application Firewall.** Attach AWS WAF to the external load balancer to block common attacks.
- **Monitoring and alarms.** Create CloudWatch alarms for unhealthy hosts, CPU and database storage, and send them to an SNS topic.
- **Backups.** Keep automated RDS backups on and test a restore.
- **Infrastructure as code.** Rebuild the same stack with CloudFormation or Terraform so the environment is repeatable and reviewable.

References and Credits

Item	Details
Prepared by	Darshan Patil
LinkedIn	https://www.linkedin.com/in/darshan-devops/
GitHub	https://github.com/darshanpatil8799/darshanpatil8799
Application source code	Public three-tier sample project: github.com/Rutuja-Naik07/Project-Three-Tier-Architecture-on-AWS
AWS documentation	Amazon VPC, Elastic Load Balancing, EC2 Auto Scaling, Amazon RDS, Systems Manager
Evidence	Screenshots from my own AWS account, October 2026

Appendix C: Cost Management

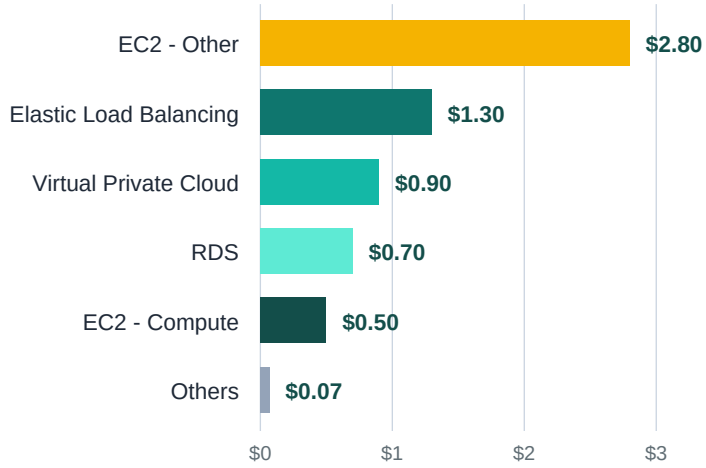
This appendix records what the project cost to run. The figures come from the AWS Billing and Cost Management dashboard for October 2026 (month-to-date): **$6.27** in total, against $0.00 for the same period last month. Running a three-tier stack with a load balancer, NAT gateways and a database costs money even when nobody uses the application, so check this page before you leave the environment running.

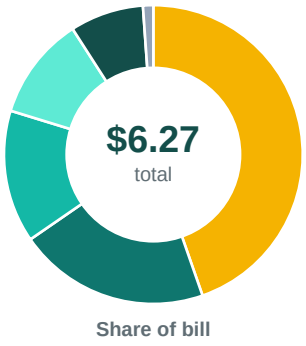
$6.27	$0.00	5	45%
Month-to-date cost	Last month (same period)	Main cost services	EC2 - Other share (est.)

Cost by Service

	Service	Approx. cost (USD)	Share	What it covers
	EC2 - Other	$2.80	45%	Typically NAT Gateway, EBS volumes and data transfer
	Elastic Load Balancing	$1.30	21%	The Application Load Balancer (hourly + usage)
	Virtual Private Cloud	$0.90	14%	Networking charges for the custom VPC
	RDS	$0.70	11%	The database tier
	EC2 - Compute	$0.50	8%	The EC2 instances themselves
	Others	$0.07	1%	Remaining small charges
	Total	**$6.27**	**100%**	Month-to-date, October 2026

Cost Chart





IMPORTANT

Only the $6.27 total and last month's $0.00 are exact figures from the dashboard. The per-service amounts are estimates read from the stacked bar chart and add up to the total. Replace them with exact figures from Cost Explorer (Group by Service). "Others" is the remainder.

NOTE

The load balancer, NAT gateways, RDS database and unused Elastic IPs bill by the hour. Delete them when you finish testing (see Step 15: Clean Up the Infrastructure), and set an AWS Budget alert before you build.